

Section 1 - Easy SQL
Section Summary
No. of Questions: 1
Duration: 20 min
Additional Instructions:

None

Q1. PRODUCT SALES RANKING

An online marketplace stores product sales in a `product_sales` table. The management team wants to compare the performance of products based on their total sales amount.

Task:

Write a SQL query that calculates the total sales for each product and assigns a rank based on total sales, with the highest-selling product receiving rank 1.

Output columns:

`product_name`
`total_sales`
`sales_rank`

Sort the result by `sales_rank` ascending.

Input Format

Table: `product_sales`

Column	Type
<code>sale_id</code>	INT
<code>product_name</code>	VARCHAR(100)
<code>category</code>	VARCHAR(50)
<code>sale_amount</code>	DECIMAL(10,2)

Sample Input

<code>sale_id</code>	<code>product_name</code>	<code>category</code>	<code>sale_amount</code>
1	Laptop	Electronics	60000
2	Phone	Electronics	35000
3	Laptop	Electronics	45000
4	Headset	Accessories	10000
5	Phone	Electronics	30000
6	Headset	Accessories	12000

Output Format

`product_name` | `total_sales` | `sales_rank`

Sample Output

<code>product_name</code>	<code>total_sales</code>	<code>sales_rank</code>
Laptop	105000.00	1
Phone	65000.00	2
Headset	22000.00	3

Explanation

The query first groups sales by product and calculates total sales using `SUM()`. A ranking function is then used to assign a sales rank based on total sales from highest to lowest.

Expected Output

<code>product_name</code>	<code>total_sales</code>	<code>sales_rank</code>
Laptop	145000.00	1
Phone	98000.00	2

Monitor	55000.00	3
Headset	32000.00	4
Keyboard	18000.00	5

Constraints

Use SUM() to calculate product-level sales.
 Group records by product.
 Use a window ranking function.
 Highest total sales must receive rank 1.
 Sort the final result by rank ascending.
 Return the exact output column names specified.

Q2. AVERAGE SALARY FOR WELL-STAFFED DEPARTMENTS

A company stores employee information in an employees table. The HR department wants to compare average salaries among departments that have a reasonable number of active employees.

Only departments having at least 4 active employees should be included.

Task:

Write a SQL query that returns the average salary for every department having at least 4 active employees.

Output columns:

department_name
 avg_salary

Round the average salary to two decimal places.

Input Format

Table: employees

Column	Type
employee_id	INT
employee_name	VARCHAR(100)
department_name	VARCHAR(50)
salary	DECIMAL(10,2)
employment_status	VARCHAR(20)

Sample Input

employee_id	employee_name	department_name	salary	employment_status
1	Nitin	Engineering	70000.00	active
2	Priya	Engineering	72000.00	active
3	Karan	Engineering	68000.00	active
4	Riya	Engineering	76000.00	active
5	Mohit	HR	55000.00	active

Output Format

department_name | avg_salary

Sample Output

department_name	avg_salary
Engineering	71500.00

Explanation

The query first filters active employees. It then groups them by department and uses HAVING COUNT(*) >= 4 to retain only departments with enough active employees.

Expected Output

department_name	avg_salary
Engineering	71500.00
Finance	82500.00
Marketing	64500.00
Support	48500.00

Constraints

Only active employees should contribute to the calculation.
Departments with fewer than 4 active employees must be excluded.
Use GROUP BY.
Use HAVING COUNT(*) >= 4.
Use AVG().
Use ROUND(..., 2).
Sort by department name ascending.

Q3. FIND LOW-PERFORMING APPLICATIONS

A mobile application company maintains application statistics in an applications table. The product team wants to identify applications whose download count is lower than the overall average download count.

Task:

Write a SQL query that returns all applications whose download_count is less than the average download count of all applications.

Output columns:

app_name
category
download_count

Sort by download_count in ascending order.

Input Format

Table: applications

Column	Type
app_id	INT
app_name	VARCHAR(100)
developer_name	VARCHAR(100)
category	VARCHAR(50)
download_count	INT

Sample Input

app_id	app_name	developer_name	category	download_count
1	QuickChat	Nova Labs	Social	150000
2	FitTrack	HealthWorks	Health	100000
3	NoteBox	Bright Apps	Productivity	80000
4	TravelMate	Orbit Apps	Travel	50000
5	RecipeBook	FoodSoft	Lifestyle	30000

Output Format

app_name | category | download_count

Sample Output

app_name	category	download_count
RecipeBook	Lifestyle	30000

Explanation

The overall average download count is calculated using a scalar subquery. Each

application's download count is then compared with that value.

Expected Output

app_name	category	download_count
RecipeBook	Lifestyle	30000
TravelMate	Travel	50000
NoteBox	Productivity	80000

Constraints

Use a scalar subquery with AVG(download_count).

Compare every application's download count against the overall average.

Return only rows satisfying the strict < condition.

Sort by download_count ascending.

Q5. FIND HIGH-VALUE TRAVELERS

A travel booking platform stores every reservation made by its customers in a reservations table. The business team wants to identify travelers whose average reservation cost is greater than the average cost of all reservations.

Task:

Write a SQL query that returns travelers whose average reservation cost is greater than the overall average reservation cost.

The output should also include the traveler's total booking cost and number of reservations.

Output columns:

traveler_name

total_cost

reservation_count

Sort by total_cost descending, followed by traveler_name ascending.

Input Format

Table: reservations

Column Name	Data Type	Constraint
reservation_id	INT	PRIMARY KEY
traveler_name	VARCHAR(50)	â ”
destination	VARCHAR(50)	â ”
booking_cost	INT	â ”
reservation_date	DATE	â ”

Sample Input

	reservation_id	traveler_name	destination	booking_cost	reservation_date
1		Aditi	Goa	15000	2025-02-01
2		Sameer	Delhi	7000	2025-02-03
3		Aditi	Dubai	28000	2025-02-08
4		Vikram	Paris	40000	2025-02-10
5		Sameer	Jaipur	6000	2025-02-14

Output Format

traveler_name | total_cost | reservation_count

Sample Output

traveler_name	total_cost	reservation_count
Vikram	40000	1
Aditi	43000	2

Explanation

The query calculates the average reservation cost for every traveler using AVG(). It then compares this value against the overall reservation average calculated using a scalar subquery.

Expected Output

traveler_name	total_cost	reservation_count
Vikram	68000	2
Aditi	65000	3

Constraints

Group records by traveler.

Calculate traveler-level average using AVG().

Calculate total booking cost using SUM().

Calculate reservation count using COUNT().

Compare against the overall average using a scalar subquery.

Use strict > comparison.

Apply the required sorting.

Q5. CREATE A COMPLETE CUSTOMER DIRECTORY

A company maintains customers who are currently registered in one table and customers from an older archived system in another table.

The reporting team wants a combined list of all unique customer names appearing in either source.

Task:

Write a SQL query that combines the customer names from the two tables while removing duplicate names.

The result should be displayed alphabetically.

Output columns:

customer_name

Input Format

Table: current_customers

Column	Type
--------	------

customer_id	INT
-------------	-----

customer_name	VARCHAR(100)
---------------	--------------

Table: archived_customers

Column	Type
--------	------

archive_id	INT
------------	-----

customer_name	VARCHAR(100)
---------------	--------------

Sample Input

current_customers

customer_id	customer_name
-------------	---------------

1	Aarav
---	-------

2	Neha
---	------

3	Karan
---	-------

4	Priya
---	-------

archived_customers

archive_id	customer_name
101	Rohan
102	Neha
103	Sameer
104	Priya

Output Format
customer_name

Sample Output
customer_name
Aarav
Karan
Neha
Priya
Rohan
Sameer
Explanation

Both tables contain customer names. Some customers may appear in both tables. The query should combine the two result sets and remove duplicate values.

Expected Output
customer_name
Aarav
Karan
Neha
Priya
Rohan
Sameer
Tanvi
Vikram
Constraints

Retrieve customer names from both tables.
Duplicate customer names must appear only once.
Use an appropriate set operation.
Sort the final result alphabetically.
Return exactly one column named customer_name.
QUESTION 6 " AVERAGE MONTHLY PRICE FOR CLOUD PLANS
Section 1 - Easy SQL
Q1. AVERAGE MONTHLY PRICE FOR POPULAR CLOUD PLANS

A cloud service provider stores subscriber information in a cloud_subscribers table. The pricing team wants to compare the average monthly price of plans that have a sufficient number of active subscribers.

Only plans with at least 4 currently active subscribers should be included.

Task:

Write a SQL query that returns the average monthly price for every plan having at least 4 active subscribers.

Output columns:
plan_name
avg_price

The average should be rounded to two decimal places.

Input Format

Table: cloud_subscribers

```

Column      Type
subscriber_id  INT
subscriber_name VARCHAR(100)
plan_name      VARCHAR(50)
monthly_price  DECIMAL(7,2)
subscriber_status VARCHAR(20)

```

Sample Input

subscriber_id	subscriber_name	plan_name	monthly_price	subscriber_status
1	A. Khan	Standard	2100.00	active
2	B. Rao	Standard	2200.00	active
3	C. Shah	Standard	2300.00	active
4	D. Jain	Standard	2400.00	active
5	E. Nair	Premium	4500.00	paused

Output Format

```
plan_name | avg_price
```

Sample Output

```
plan_name  avg_price
Standard   2250.00
```

Explanation

Only active subscribers are considered. The records are grouped by plan and plans with fewer than four active subscribers are removed using HAVING.

Expected Output

```
plan_name  avg_price
Basic      1650.00
Premium    4825.00
Standard   2250.00
```

Constraints

Only active subscribers should contribute.

Use GROUP BY plan_name.

Use COUNT() in the HAVING clause.

Only plans with at least 4 active subscribers should appear.

Use AVG() to calculate the monthly price.

Use ROUND() to two decimal places.

Sort alphabetically by plan_name.

QUESTION 7 â€” FIND HIGH-VALUE RESTAURANT CUSTOMERS

Section 1 - Easy SQL

Q1. FIND HIGH-VALUE RESTAURANT CUSTOMERS

A food delivery platform records every order placed by its customers in a food_orders table. The customer analytics team wants to identify customers whose average order value is greater than the overall average order value across all orders.

This helps the company identify customers who regularly place high-value orders and may be eligible for premium loyalty benefits.

Task:

Write a SQL query on the food_orders table that returns customers whose average order amount is greater than the overall average order amount across all orders.

The output should also contain each qualifying customer's total spending and number of orders.

Output columns:

```
customer_name
total_spent
order_count
```

Sort the output by total_spent in descending order, then by customer_name in ascending order.

Input Format

Table: food_orders

Column Name	Data Type	Constraint
order_id	INT	PRIMARY KEY
customer_name	VARCHAR(50)	â "
restaurant_name	VARCHAR(50)	â "
order_amount	INT	â "
order_date	DATE	â "

Sample Input

order_id	customer_name	restaurant_name	order_amount	order_date
1	Rohan	Spice Hub	1200	2025-01-05
2	Meena	Urban Bites	600	2025-01-06
3	Rohan	Curry House	1800	2025-01-10
4	Arjun	Food Street	2500	2025-01-12
5	Meena	Spice Hub	700	2025-01-13

Output Format

customer_name	total_spent	order_count
---------------	-------------	-------------

Sample Output

customer_name	total_spent	order_count
---------------	-------------	-------------

Arjun 2500 1

Rohan 3000 2

Explanation

The query groups orders by customer and calculates each customer's average order amount. This value is then compared with the overall average order amount calculated from the complete food_orders table.

Only customers whose average order amount is strictly greater than the overall average are returned.

Expected Output

customer_name	total_spent	order_count
---------------	-------------	-------------

Arjun 7200 3

Rohan 5200 3

Constraints

Use GROUP BY.

Use AVG() to calculate the customer-level average.

Use SUM() to calculate total spending.

Use COUNT() to calculate the order count.

Compare the customer average with the overall average using a scalar subquery.

Return only customers with an average greater than the overall average.

Sort by total_spent descending and customer_name ascending.

QUESTION 8 â " FIND CUSTOMERS WITH MULTIPLE PURCHASES

Section 1 - Easy SQL

Q1. CUSTOMERS WITH MULTIPLE PURCHASES

An online electronics store maintains customer information in a customers table and purchase information in an orders table.

The sales team wants to identify customers who have placed at least two completed orders and determine how much they have spent.

Task:

Write a SQL query that returns customers having at least 2 completed orders.

The output should contain the customer's name, number of completed orders, and total amount spent.

Output columns:

customer_name

```

order_count
total_spent
Input Format
Table: customers
Column      Type
customer_id INT
customer_name VARCHAR(100)
city VARCHAR(50)
Table: orders
Column      Type
order_id    INT
customer_id INT
product_name VARCHAR(100)
amount      DECIMAL(10,2)
order_status VARCHAR(20)
Sample Input

```

customers

customer_id	customer_name	city
1	Aarav	Mumbai
2	Neha	Pune
3	Karan	Delhi
4	Priya	Mumbai

orders

order_id	customer_id	product_name	amount	order_status
101	1	Laptop	60000	completed
102	1	Mouse	2000	completed
103	2	Phone	30000	completed
104	3	Tablet	25000	cancelled
105	3	Keyboard	3000	completed

Output Format

customer_name | order_count | total_spent

Sample Output

customer_name	order_count	total_spent
Aarav	2	62000.00

Explanation

The customer and order information is combined using a join. Only completed orders are considered. The records are grouped by customer, and HAVING is used to retain customers with at least two completed orders.

Expected Output

customer_name	order_count	total_spent
Aarav	3	68000.00
Priya	2	45000.00
Neha	2	39000.00

Constraints

Use an INNER JOIN.

Consider only orders having order_status = 'completed'.

Use COUNT() for the order count.

Use SUM() for total spending.

Use GROUP BY.

Use HAVING COUNT(*) >= 2.

Sort by total_spent descending.

Concepts Covered

INNER JOIN, WHERE, GROUP BY, COUNT(), SUM(), HAVING, ORDER BY

QUESTION 9 â ” BOOKS WITH BELOW-AVERAGE SALES

Section 1 - Easy SQL

Q1. BOOKS WITH BELOW-AVERAGE SALES

A bookstore maintains its catalog and sales information in a books table. The management team wants to find books whose sales are below the average number of copies sold across all books.

Task:

Write a SQL query that returns all books whose copies_sold is less than the average copies_sold of all books.

Output columns:

book_title
category
copies_sold

Sort the output by copies_sold in ascending order.

Input Format

Table: books

Column	Type
book_id	INT
book_title	VARCHAR(100)
author	VARCHAR(100)
category	VARCHAR(50)
copies_sold	INT

Sample Input

book_id	book_title	author	category	copies_sold
1	The Silent Road	A. Roy	Fiction	80000
2	Digital World	P. Shah	Technology	60000
3	Hidden Truth	R. Mehta	Mystery	45000
4	Future Code	S. Rao	Technology	30000
5	Morning Light	K. Jain	Fiction	22000

Output Format

book_title | category | copies_sold

Sample Output

book_title	category	copies_sold
Morning Light	Fiction	22000

Explanation

A scalar subquery calculates the average number of copies sold across the entire table. The outer query returns only books whose sales are strictly below that average.

Expected Output

book_title	category	copies_sold
Morning Light	Fiction	22000
Ocean Secrets	Mystery	25000
Future Code	Technology	30000
Hidden Truth	Mystery	45000

Constraints

Use a scalar subquery containing AVG(copies_sold).

Calculate the average across the complete table.

Use a strict < comparison.

Sort by copies_sold ascending.

QUESTION 10 â ” FIND EMPLOYEES EARNING ABOVE DEPARTMENT AVERAGE

Section 1 - Easy SQL

Q1. EMPLOYEES ABOVE DEPARTMENT AVERAGE

A company stores employee and department information in separate tables. The HR team wants to find employees whose salary is greater than the average salary of their own department.

Task:

Write a SQL query that returns employees whose salary is higher than their department's average salary.

Output columns:

employee_name
department_name
salary
department_avg_salary

Sort by department_name ascending, then salary descending.

Input Format

Table: employees

Column Type

employee_id INT

employee_name VARCHAR(100)

department_id INT

salary DECIMAL(10,2)

Table: departments

Column Type

department_id INT

department_name VARCHAR(100)

Sample Input

employees

employee_id	employee_name	department_id	salary
1	Amit	10	70000.00
2	Neha	10	90000.00
3	Rohan	10	65000.00
4	Priya	20	60000.00
5	Karan	20	80000.00

departments

department_id	department_name
10	Engineering
20	Finance

Output Format

employee_name | department_name | salary | department_avg_salary

Sample Output

employee_name	department_name	salary	department_avg_salary
Neha	Engineering	90000.00	75000.00
Karan	Finance	80000.00	70000.00

Explanation

The department average salary can be calculated using a grouped subquery. The resulting average is matched with employees through the department identifier.

Expected Output

employee_name	department_name	salary	department_avg_salary
Neha	Engineering	90000.00	75000.00
Karan	Finance	80000.00	70000.00

Constraints

Join employee and department information.

Calculate department-level average salary.

Return only employees whose salary is strictly greater than their department average.

Use AVG().

Use a subquery or derived table for department averages.

Sort according to the specified order.

Q11. MOST RESERVED HOTEL ROOMS

StayNest, a hotel reservation platform, is preparing a "guest favorites" report to highlight the room types that customers reserve most frequently. The management team wants to include every room type tied for the highest number of reservations, rather than selecting only one when there is a tie.

The report should be displayed alphabetically by room type.

Task:

Write a query against the rooms table that returns the room types with the highest reservation count in the hotel catalog.

The output must contain the following columns:

room_type â " Type of the hotel room
hotel_name â " Name of the hotel
reservation_count â " Total number of reservations
nightly_rate â " Nightly rate of the room

Also display only rooms whose nightly_rate is greater than 3000.

Input Format
TABLE SCHEMA:

Table: rooms

Column	Type	Description
room_id	INT	Unique identifier for the room
room_type	VARCHAR(50)	Type of room
hotel_name	VARCHAR(100)	Hotel offering the room
reservation_count	INT	Number of reservations
nightly_rate	DECIMAL(8,2)	Nightly room price

SAMPLE INPUT

The rooms table is pre-populated.

room_id	room_type	hotel_name	reservation_count	nightly_rate
1	Deluxe	Grand Vista	120	4200.00
2	Standard	City Inn	95	2500.00
3	Suite	Grand Vista	120	7500.00
4	Executive	Metro Palace	80	5500.00
5	Family	Lake View	70	3800.00

Output Format

Return the following columns in this exact order:

room_type | hotel_name | reservation_count | nightly_rate

SAMPLE OUTPUT

room_type	hotel_name	reservation_count	nightly_rate
Deluxe	Grand Vista	120	4200.00
Suite	Grand Vista	120	7500.00

Explanation

The highest reservation count in the sample is 120. Both Deluxe and Suite have this value, so both must be returned.

The Standard room also has fewer reservations and its nightly rate is below the required threshold, so it is excluded.

Expected Output

room_type	hotel_name	reservation_count	nightly_rate
Deluxe	Grand Vista	210	4200.00
Suite	Grand Vista	210	7500.00

Constraints

Only room types matching the maximum reservation_count should appear.

If multiple rooms share the maximum value, include all tied rows.

Only rows where nightly_rate > 3000 should be returned.

Do not use a fixed maximum value.

The output must be ordered by room_type ascending.

Use the exact output column names specified.

Q12. REVENUE SHARE BY DELIVERY ZONE

QuickKart, an online grocery delivery platform, is preparing a monthly performance report. The operations team wants to see how much revenue each delivery zone generated and what percentage of the platform's overall revenue came from each zone.

The report should show the largest revenue contributors first.

Task:

Write a query against the deliveries table that returns each delivery zone along with its total revenue and percentage contribution to total platform revenue.

Additionally, return the number of deliveries made in each zone.

Output columns:

delivery_zone

zone_revenue

revenue_percentage

delivery_count

Input Format

TABLE SCHEMA:

Table: deliveries

Column	Type	Description
delivery_id	INT	Unique delivery identifier
delivery_zone	VARCHAR(50)	Delivery zone
item_category	VARCHAR(50)	Category of item delivered
delivery_amount	DECIMAL(10,2)	Amount earned from the delivery
delivery_date	DATE	Delivery date

SAMPLE INPUT

delivery_id	delivery_zone	item_category	delivery_amount	delivery_date
1	Central	Grocery	12000.00	2025-01-05
2	North	Grocery	9000.00	2025-01-06
3	South	Household	7000.00	2025-01-08
4	West	Grocery	5000.00	2025-01-10
5	East	Household	3000.00	2025-01-12

Output Format

delivery_zone | zone_revenue | revenue_percentage | delivery_count

Sample Output

delivery_zone	zone_revenue	revenue_percentage	delivery_count
Central	12000.00	33.80	1
North	9000.00	25.35	1

Explanation

The total platform revenue in the five sample rows is 36000.00.

Each zone's percentage is calculated as:

$(\text{zone_revenue} / \text{overall_revenue}) \times 100$

The percentage must be rounded to two decimal places.

Expected Output

delivery_zone	zone_revenue	revenue_percentage	delivery_count
Central	295000.00	38.31	18
North	158000.00	20.52	14
South	132000.00	17.14	12
West	98000.00	12.73	10
East	87000.00	11.30	9

Constraints

Use GROUP BY delivery_zone.

Calculate zone revenue using SUM(delivery_amount).

Calculate the overall revenue across all rows.

revenue_percentage must be calculated using zone revenue divided by overall revenue, multiplied by 100.

Round the percentage to two decimal places using ROUND().

Include the count of deliveries in each zone.

Order by revenue_percentage descending.

Use the exact output column names specified.

QUESTION 13 " MOST BORROWED MOVIES

CineVault, a digital movie rental service, wants to prepare a "most watched" section on its homepage. The content team wants every movie that has reached the highest number of rentals to appear, including tied movies.

Only movies released from 2022 onward should be considered.

Task:

Write a query against the movies table that returns movies having the highest rental count among movies released in 2022 or later.

The output must also include the release year.

Output columns:

title
director
genre
rental_count
release_year

Input Format

Table: movies

Column	Type
movie_id	INT
title	VARCHAR(100)
director	VARCHAR(100)
genre	VARCHAR(50)
rental_count	INT
release_year	INT

Sample Input

movie_id	title	director	genre	rental_count	release_year
1	Last Horizon	A. Sen	Drama	85000	2023
2	Silent City	R. Das	Thriller	72000	2024
3	Final Mission	K. Rao	Action	85000	2022
4	Old Memories	P. Shah	Drama	99000	2020
5	Blue Night	M. Jain	Mystery	65000	2024

Output Format

title	director	genre	rental_count	release_year
-------	----------	-------	--------------	--------------

Sample Output

title	director	genre	rental_count	release_year
Final Mission	K. Rao	Action	85000	2022
Last Horizon	A. Sen	Drama	85000	2023

Explanation

Old Memories has a larger rental count, but it was released before 2022 and must not participate in the comparison.

Among the eligible movies, 85000 is the highest rental count, and two movies share that value.

Expected Output

title	director	genre	rental_count	release_year
Final Mission	K. Rao	Action	175000	2022
Last Horizon	A. Sen	Drama	175000	2023

Constraints

Consider only movies with release_year >= 2022.

Find the maximum rental count among the eligible rows.

Return all movies tied at that maximum.

Order the output alphabetically by title.

Use the exact output column names.

QUESTION 14 â€” SALES SHARE BY PRODUCT CATEGORY

ShopSphere is reviewing its sales performance across product categories. Management wants to understand how much each category contributes to the company's overall sales.

The report should show each category's total revenue, its percentage contribution, and the number of products sold.

Task:

Write a query against the sales table that returns each category's total sales and its percentage share of the overall sales.

Output columns:

category
category_total
pct_of_total
items_sold

Input Format

Table: sales

Column	Type
sale_id	INT
product_name	VARCHAR(100)
category	VARCHAR(50)
sale_amount	DECIMAL(10,2)
quantity	INT

```

sale_date    DATE
Sample Input
sale_id      product_name    category    sale_amount quantity    sale_date
1      Alpha Phone Mobile    30000.00    2      2025-02-01
2      Vision Laptop         Laptop      65000.00    1      2025-02-02
3      Tab Max               Tablet      28000.00    2      2025-02-04
4      Smart Fit             Wearable    12000.00    1      2025-02-06
5      Echo Bud              Audio 5000.00    3      2025-02-08

```

Output Format

```
category | category_total | pct_of_total | items_sold
```

Sample Output

```

category    category_total    pct_of_total    items_sold
Laptop      65000.00          46.43 1
Mobile      30000.00          21.43 2
Tablet      28000.00          20.00 2
Wearable    12000.00          8.57 1
Audio 5000.00    3.57 3

```

Explanation

The overall sales amount is obtained by summing sale_amount across the complete table.

The percentage contribution of a category is:

$(\text{category_total} / \text{overall_sales}) \times 100$

Expected Output

```

category    category_total    pct_of_total    items_sold
Laptop      390000.00          41.95 14
Mobile      185000.00          19.89 19
Tablet      150000.00          16.13 11
Wearable    112000.00          12.04 9
Audio 93000.00    9.99 15

```

Constraints

Group records by category.

Use SUM(sale_amount) for category totals.

Use SUM(quantity) for items_sold.

Calculate each category's percentage from the overall sales total.

Round pct_of_total to two decimal places.

Order by pct_of_total descending.

Return one row per category.

QUESTION 15 " MOST WATCHED TRAINING COURSES

LearnEdge, a corporate learning platform, wants to highlight the courses that employees have watched most frequently.

Every course tied for the highest number of views should be displayed.

Only courses with a rating of 4.0 or above should participate in the comparison.

Task:

Write a query against the courses table that returns all eligible courses having the maximum view count.

The result should also include each course's rating.

Output columns:

```

course_name
instructor
category

```

view_count
rating
Input Format

Table: courses

Column	Type
course_id	INT
course_name	VARCHAR(100)
instructor	VARCHAR(100)
category	VARCHAR(50)
view_count	INT
rating	DECIMAL(2,1)

Sample Input

course_id	course_name	instructor	category	view_count	rating
1	SQL Essentials	R. Kumar	Database	95000	4.5
2	Python Basics	A. Shah	Programming	82000	4.3
3	Data Analytics	P. Rao	Analytics	95000	4.7
4	Excel Fundamentals	S. Jain	Productivity	110000	3.8
5	Cloud Basics	N. Verma	Cloud	75000	4.2

Output Format

course_name	instructor	category	view_count	rating
-------------	------------	----------	------------	--------

Sample Output

course_name	instructor	category	view_count	rating
Data Analytics	P. Rao	Analytics	95000	4.7
SQL Essentials	R. Kumar	Database	95000	4.5

Explanation

Excel Fundamentals has the highest view count overall, but its rating is below 4.0, so it is excluded before finding the maximum.

Among eligible courses, SQL Essentials and Data Analytics share the maximum view count.

Expected Output

course_name	instructor	category	view_count	rating
Data Analytics	P. Rao	Analytics	185000	4.7
SQL Essentials	R. Kumar	Database	185000	4.5

Constraints

Only courses with rating ≥ 4.0 should participate.
Find the maximum view_count among eligible courses.
Include every course tied for that maximum.
Sort by course_name ascending.
Use a subquery or equivalent aggregate approach.
Do not hard-code the maximum value.

QUESTION 16 & " REVENUE CONTRIBUTION BY PAYMENT METHOD

PayWave is analyzing transactions made through different payment methods. The finance team wants a report showing how much revenue each payment method generated and what percentage it contributed to the total transaction revenue.

The report should also include the number of transactions processed through each payment method.

Task:

Write a SQL query against the transactions table that returns each payment method's total revenue, percentage contribution, and transaction count.

Output columns:
payment_method

method_total
pct_of_total
transaction_count
Input Format

Table: transactions

Column Type
transaction_id INT
payment_method VARCHAR(50)
transaction_type VARCHAR(50)
transaction_amount DECIMAL(10,2)
transaction_date DATE
Sample Input
transaction_id payment_method transaction_type transaction_amount
transaction_date
1 Card Purchase 18000.00 2025-03-01
2 UPI Purchase 12000.00 2025-03-02
3 Wallet Purchase 7000.00 2025-03-03
4 Card Purchase 15000.00 2025-03-05
5 Bank Transfer Purchase 8000.00 2025-03-06
Output Format
payment_method | method_total | pct_of_total | transaction_count
Sample Output
payment_method method_total pct_of_total transaction_count
Card 33000.00 55.00 2
UPI 12000.00 20.00 1
Bank Transfer 8000.00 13.33 1
Wallet 7000.00 11.67 1
Explanation

The overall transaction amount is calculated across all rows.

For every payment method:

$\text{pct_of_total} = (\text{method_total} / \text{overall_total}) \times 100$

The percentage is rounded to two decimal places.

Expected Output
payment_method method_total pct_of_total transaction_count
Card 420000.00 44.12 31
UPI 255000.00 26.78 25
Bank Transfer 165000.00 17.33 18
Wallet 112000.00 11.77 14

Constraints

Group transactions by payment_method.
Calculate total revenue using SUM(transaction_amount).
Calculate transaction count using COUNT(*).
Calculate the percentage against the overall transaction total.
Use ROUND() to two decimal places.
Order by pct_of_total descending.
Use the exact output column names.

QUESTION 17 â ” SECOND-HIGHEST RATED PRODUCT IN EACH CATEGORY

A product review portal stores product ratings in a product_reviews table. The editorial team wants to identify the second-highest rated product in every category.

If multiple products share the same second-highest rating, all such products

should be included.

Task:

Write a query that returns the second-highest rated product or products from each category.

Output columns:

category
product_name
rating

Input Format

Table: product_reviews

Column	Type
review_id	INT
product_name	VARCHAR(100)
category	VARCHAR(50)
rating	DECIMAL(2,1)

Sample Input

review_id	product_name	category	rating
1	Alpha X	Phones	4.9
2	Beta Pro	Phones	4.7
3	Gamma Lite	Phones	4.7
4	UltraBook	Laptops	4.8
5	NoteBook	Laptops	4.6

Output Format

category | product_name | rating

Sample Output

category	product_name	rating
Laptops	NoteBook	4.6
Phones	Beta Pro	4.7
Phones	Gamma Lite	4.7

Explanation

The first-highest rating within the Phones category is 4.9. The next distinct rating is 4.7, so both products having 4.7 are returned.

The same logic is applied independently to every category.

Expected Output

category	product_name	rating
Accessories	Smart Band	4.5
Laptops	NoteBook	4.6
Phones	Beta Pro	4.7
Phones	Gamma Lite	4.7
Tablets	Tab Air	4.4

Constraints

Determine the second-highest distinct rating within every category.

Include all products tied at that rating.

Do not use a manually entered rating.

Return one or more rows per category.

Sort by category ascending and product_name ascending.

Concepts Covered

QUESTION 18 â ” MONTHLY SALES SHARE

A retail company stores all sales transactions in a regional_sales table. The

management team wants to see how much each region contributed to monthly sales.

For every month and region, calculate the region's total sales and its percentage share of that month's overall sales.

Task:

Write a SQL query that returns the sales contribution of each region for every month.

Output columns:

sales_month
region
region_total
month_percentage
Input Format

Table: regional_sales

Column Type
sale_id INT
region VARCHAR(50)
sale_amount DECIMAL(10,2)
sale_date DATE

Sample Input

sale_id	region	sale_amount	sale_date
1	North	10000.00	2025-01-05
2	South	7000.00	2025-01-06
3	North	8000.00	2025-01-12
4	South	5000.00	2025-01-15
5	West	4000.00	2025-01-18

Output Format

sales_month | region | region_total | month_percentage

Sample Output

sales_month	region	region_total	month_percentage
2025-01	North	18000.00	52.94
2025-01	South	12000.00	35.29
2025-01	West	4000.00	11.76

Explanation

Unlike a simple category percentage report, the overall total must be calculated separately for each month.

For every region:

month_percentage =

(region_total / total_sales_for_that_month) * 100

Expected Output

sales_month	region	region_total	month_percentage
2025-01	North	92000.00	48.42
2025-01	South	51000.00	26.84
2025-01	West	47000.00	24.74
2025-02	North	81000.00	44.02
2025-02	South	59000.00	32.07
2025-02	West	44000.00	23.91

Constraints

Group sales by month and region.

Calculate monthly regional totals.

Calculate the overall sales total separately for each month.

Calculate the percentage contribution of each region within its own month.

Round month_percentage to two decimal places.

Sort by sales_month ascending and month_percentage descending.

QUESTION 19 " TOP-SELLING PRODUCT WITHIN EACH CATEGORY

An online marketplace maintains product sales information in a `product_sales` table. The business team wants to identify the highest-selling product in every product category.

When multiple products have the same highest sales value within a category, all tied products should be included.

Task:

Write a SQL query that returns the top-selling product or products in every category based on total sales amount.

Output columns:

category
product_name
total_sales
Input Format

Table: `product_sales`

Column	Type
sale_id	INT
category	VARCHAR(50)
product_name	VARCHAR(100)
sale_amount	DECIMAL(10,2)

Sample Input

sale_id	category	product_name	sale_amount
1	Phones	Alpha X	45000.00
2	Phones	Beta Pro	60000.00
3	Phones	Gamma Lite	60000.00
4	Laptops	UltraBook	90000.00
5	Laptops	NoteBook	70000.00

Output Format

category | product_name | total_sales

Sample Output

category	product_name	total_sales
Laptops	UltraBook	90000.00
Phones	Beta Pro	60000.00
Phones	Gamma Lite	60000.00

Explanation

Products are first grouped by category and product to calculate total sales.

The highest total within each category is then identified. Any products tied at the highest value must be returned.

Expected Output

category	product_name	total_sales
Accessories	Smart Band	75000.00
Laptops	UltraBook	175000.00
Phones	Beta Pro	135000.00
Phones	Gamma Lite	135000.00
Tablets	Tab Air	92000.00

Constraints

Calculate total sales for each product.

Compare products only with others in the same category.

Return every product tied for the category maximum.

Use aggregation to determine product-level totals.

Sort by category ascending and product name ascending.
Concepts Covered

QUESTION 20 " CATEGORY PERFORMANCE WITH AVERAGE COMPARISON

A retail analytics team is reviewing category performance using a category_sales table.

They want to identify categories whose total sales are greater than the average category sales across all categories.

For every qualifying category, display its total sales and percentage contribution to the company's overall sales.

Task:

Write a SQL query that:

Calculates total sales for each category.

Calculates the average category sales.

Returns only categories whose total sales are greater than that average.

Calculates the category's percentage share of the complete company's sales.

Output columns:

category

category_total

average_category_sales

pct_of_total

Input Format

Table: category_sales

Column	Type
--------	------

sale_id	INT
---------	-----

category	VARCHAR(50)
----------	-------------

sale_amount	DECIMAL(10,2)
-------------	---------------

sale_date	DATE
-----------	------

Sample Input

sale_id	category	sale_amount	sale_date
---------	----------	-------------	-----------

1	Electronics	60000.00	2025-04-01
---	-------------	----------	------------

2	Furniture	35000.00	2025-04-02
---	-----------	----------	------------

3	Clothing	20000.00	2025-04-03
---	----------	----------	------------

4	Electronics	40000.00	2025-04-04
---	-------------	----------	------------

5	Grocery	15000.00	2025-04-05
---	---------	----------	------------

Output Format

category	category_total	average_category_sales	pct_of_total
----------	----------------	------------------------	--------------

Sample Output

category	category_total	average_category_sales	pct_of_total
----------	----------------	------------------------	--------------

Electronics	100000.00	42500.00	76.92
-------------	-----------	----------	-------

Explanation

First, category-level totals are calculated.

For the sample data:

Electronics = 100000

Furniture = 35000

Clothing = 20000

Grocery = 15000

The average category total is calculated from these category totals.

Only categories whose total exceeds that average are returned.

Expected Output

category	category_total	average_category_sales	pct_of_total
Electronics	425000.00	228750.00	46.32
Furniture	310000.00	228750.00	33.77
Grocery	245000.00	228750.00	26.69

Constraints

First calculate total sales for each category.

Calculate the average of the category totals, not the average of individual sales rows.

Return only categories whose total is strictly greater than the average category total.

Calculate pct_of_total using the overall sales amount across the complete table.

Round percentage values to two decimal places.

Sort by category_total descending.

Use exact output column names.

Q1. MONTHLY REVENUE MOVEMENT BY SERVICE

A subscription platform stores service information in a services table and records the monthly revenue generated by each service in a service_revenue table.

The finance team wants to compare every month's revenue with the immediately preceding month for the same service. This will help them identify services whose revenue is increasing or declining over time.

Task

Write an SQL query that joins the services table with the service_revenue table and uses the LAG window function to retrieve the previous month's revenue for each service.

Return one row for every recorded month of every service.

Also calculate the percentage change from the previous month.

For the first recorded month of each service, the previous revenue and percentage change should be NULL.

Output columns

service_name – Name of the service

revenue_month – Month number of the current row

revenue – Revenue generated during the current month

previous_revenue – Revenue generated in the previous month for the same service

revenue_change – Difference between current and previous revenue

change_percentage – Percentage change from the previous month

Input Format

Table 1: services

Column	Type	Description
service_id	INT	Unique service identifier
service_name	VARCHAR(50)	Name of the service
service_type	VARCHAR(50)	Type of service

Table 2: service_revenue

Column	Type	Description
service_id	INT	References services(service_id)
revenue_month	INT	Month number
revenue	INT	Monthly revenue

Sample Input

services

service_id	service_name	service_type
1	Cloud Storage	Storage
2	Video Streaming	Entertainment
3	Music Plus	Entertainment

service_revenue

service_id	revenue_month	revenue
1	1	50000
1	2	56000
1	3	52000
1	4	64000
2	1	30000
2	2	35000
2	3	39000
2	4	37000
3	1	18000
3	2	22000

```
3      3      25000
3      4      28000
```

Output Format

```
service_name | revenue_month | revenue | previous_revenue | revenue_change |
change_percentage
```

Sample Output

```
service_name      revenue_month      revenue      previous_revenue  revenue_change
change_percentage
Cloud Storage     1      50000 NULL      NULL      NULL
Cloud Storage     2      56000 50000 6000      12.00
Cloud Storage     3      52000 56000 -4000      -7.14
Cloud Storage     4      64000 52000 12000      23.08
Explanation
```

For each service, the rows are ordered by revenue_month. LAG() retrieves the revenue of the previous month within the same service.

The revenue difference is:

```
revenue - previous_revenue
```

The percentage change is:

```
((revenue - previous_revenue) / previous_revenue) × 100
```

The first month has no previous value, so both calculated columns must be NULL.

Expected Output

```
service_name      revenue_month      revenue      previous_revenue  revenue_change
change_percentage
Cloud Storage     1      50000 NULL      NULL      NULL
Cloud Storage     2      56000 50000 6000      12.00
Cloud Storage     3      52000 56000 -4000      -7.14
Cloud Storage     4      64000 52000 12000      23.08
Music Plus 1      18000 NULL      NULL      NULL
Music Plus 2      22000 18000 4000      22.22
Music Plus 3      25000 22000 3000      13.64
Music Plus 4      28000 25000 3000      12.00
Video Streaming  1      30000 NULL      NULL      NULL
Video Streaming  2      35000 30000 5000      16.67
Video Streaming  3      39000 35000 4000      11.43
Video Streaming  4      37000 39000 -2000      -5.13
```

Constraints

Use LAG() with OVER(PARTITION BY ... ORDER BY ...).

The previous-value calculation must restart for every service.

Order each service's rows by revenue_month ascending.

The service name must come from the services table.

Return every recorded month, including each service's first month.

previous_revenue, revenue_change, and change_percentage must be NULL for the first month.

Round change_percentage to two decimal places.

Sort by service_name ascending and revenue_month ascending.

QUESTION 2 – ENROLLMENT TOTALS FOR MULTI-SESSION TRAINERS

Section 2 - Medium SQL

Q1. ENROLLMENT TOTALS FOR MULTI-SESSION TRAINERS

A professional training institute maintains trainer information, course information, and learner registrations in three separate tables.

The academic team wants to identify trainers who conduct at least 2 distinct courses and determine their total number of confirmed registrations across those

courses.

Cancelled registrations must not contribute to the total.

Task

Write a SQL query joining the trainers, training_courses, and registrations tables.

Return every trainer who teaches at least 2 distinct courses and has confirmed registrations.

Also display the number of distinct courses taught by each trainer.

Output columns

trainer_name

course_count

confirmed_registrations

Input Format

Table 1: trainers

Column	Type
--------	------

trainer_id	INT
------------	-----

trainer_name	VARCHAR(100)
--------------	--------------

expertise	VARCHAR(100)
-----------	--------------

Table 2: training_courses

Column	Type
--------	------

course_id	INT
-----------	-----

course_name	VARCHAR(100)
-------------	--------------

trainer_id	INT
------------	-----

course_category	VARCHAR(50)
-----------------	-------------

course_level	VARCHAR(20)
--------------	-------------

Table 3: registrations

Column	Type
--------	------

registration_id	INT
-----------------	-----

course_id	INT
-----------	-----

learner_name	VARCHAR(100)
--------------	--------------

registration_date	DATE
-------------------	------

registration_status	VARCHAR(20)
---------------------	-------------

Sample Input

trainers

trainer_id	trainer_name	expertise
------------	--------------	-----------

1	Rahul Mehta	Programming
---	-------------	-------------

2	Sneha Rao	Data
---	-----------	------

3	Kunal Shah	Cloud
---	------------	-------

4	Priya Nair	Testing
---	------------	---------

training_courses

course_id	course_name	trainer_id	course_category	course_level
-----------	-------------	------------	-----------------	--------------

1	Python Core 1	Programming	Beginner
---	---------------	-------------	----------

2	Advanced Python	1	Programming	Advanced
---	-----------------	---	-------------	----------

3	SQL Analytics	2	Data	Intermediate
---	---------------	---	------	--------------

4	Data Modeling	2	Data	Advanced
---	---------------	---	------	----------

5	Cloud Basics	3	Cloud	Beginner
---	--------------	---	-------	----------

registrations

registration_id	course_id	learner_name	registration_date
-----------------	-----------	--------------	-------------------

			registration_status
--	--	--	---------------------

1	1	A. Kumar	2025-05-01	confirmed
---	---	----------	------------	-----------

2	1	B. Shah	2025-05-02	confirmed
---	---	---------	------------	-----------

3	2	C. Rao	2025-05-03	confirmed
---	---	--------	------------	-----------

4	2	D. Jain	2025-05-04	cancelled
5	3	E. Nair	2025-05-05	confirmed

Output Format

trainer_name | course_count | confirmed_registrations

Sample Output

trainer_name	course_count	confirmed_registrations
Rahul Mehta	2	3

Explanation

Only confirmed registrations should be counted. The three tables are joined through trainer and course identifiers.

COUNT(DISTINCT course_id) determines how many distinct courses are taught by each trainer. The HAVING condition ensures that only trainers teaching at least two courses are included.

Expected Output

trainer_name	course_count	confirmed_registrations
Rahul Mehta	3	14
Sneha Rao	2	10
Priya Nair	2	7

Constraints

Join all three tables.

Count only registrations where registration_status = 'confirmed'.

Cancelled registrations must not contribute.

Use COUNT(DISTINCT course_id) to determine the number of courses.

Only trainers teaching at least 2 distinct courses should appear.

Use GROUP BY.

Use HAVING.

Order by confirmed_registrations descending, then trainer_name ascending.

QUESTION 3 – MONTHLY ORDER TREND BY PRODUCT

Section 2 - Medium SQL

Q1. MONTHLY ORDER TREND BY PRODUCT

An online retailer maintains product information in a products table and records monthly order quantities in an order_history table.

The operations team wants to compare each month's order quantity with the previous month for the same product.

Task

Write a SQL query that joins the two tables and uses LAG() to retrieve the previous month's order quantity for every product.

Also calculate the difference in order quantity from the previous month.

Output columns

product_name
month_number
order_quantity
previous_quantity
quantity_change

Input Format

Table: products

Column	Type
product_id	INT
product_name	VARCHAR(100)
category	VARCHAR(50)

Table: order_history

Column	Type
--------	------

product_id INT
month_number INT
order_quantity INT
Sample Input

products

product_id	product_name	category
1	Smart TV	Electronics
2	Air Purifier	Home
3	Gaming Mouse	Accessories

order_history

product_id	month_number	order_quantity
1	1	100
1	2	140
1	3	125
1	4	160
2	1	80
2	2	95
2	3	110

Output Format

product_name | month_number | order_quantity | previous_quantity |
quantity_change

Sample Output

product_name	month_number	order_quantity	previous_quantity	quantity_change
Air Purifier	1	80	NULL	NULL
Air Purifier	2	95	80	15
Air Purifier	3	110	95	15
Smart TV	1	100	NULL	NULL
Smart TV	2	140	100	40

Explanation

LAG(order_quantity) retrieves the immediately preceding quantity within the same product.

The difference is calculated as:

order_quantity - previous_quantity

Expected Output

product_name	month_number	order_quantity	previous_quantity	quantity_change
Air Purifier	1	80	NULL	NULL
Air Purifier	2	95	80	15
Air Purifier	3	110	95	15
Gaming Mouse	1	60	NULL	NULL
Gaming Mouse	2	55	60	-5
Gaming Mouse	3	90	55	35
Smart TV	1	100	NULL	NULL
Smart TV	2	140	100	40
Smart TV	3	125	140	-15
Smart TV	4	160	125	35

Constraints

Use LAG().

Partition by product.

Order within each product by month_number ascending.

Return every month.

The first month of each product must have NULL as previous_quantity and quantity_change.

The product name must come from the products table.

Sort by product_name, then month_number.

QUESTION 4 – TOTAL BOOKINGS FOR MULTI-DESTINATION TRAVEL AGENTS

Section 2 - Medium SQL

Q1. BOOKING TOTALS FOR MULTI-DESTINATION AGENTS

A travel company stores travel agent information, destination packages, and customer bookings in separate tables.

The management team wants to identify agents who handle at least 2 distinct travel packages and determine the total number of confirmed bookings handled by each agent.

Task

Write a SQL query joining the agents, packages, and bookings tables.

Return each qualifying agent along with the number of distinct packages they manage and the total confirmed bookings.

Also include the number of different destinations covered by each agent.

Output columns

agent_name
package_count
destination_count
confirmed_bookings

Input Format

Table: agents

Column	Type
agent_id	INT
agent_name	VARCHAR(100)
branch	VARCHAR(50)

Table: packages

Column	Type
package_id	INT
package_name	VARCHAR(100)
agent_id	INT
destination	VARCHAR(100)
package_type	VARCHAR(50)

Table: bookings

Column	Type
booking_id	INT
package_id	INT
traveler_name	VARCHAR(100)
booking_status	VARCHAR(20)
booking_date	DATE

Sample Input

agents

agent_id	agent_name	branch
1	Amit Verma	Mumbai
2	Neha Kapoor	Delhi
3	Rohan Singh	Pune

packages

package_id	package_name	agent_id	destination	package_type
1	Goa Escape	1	Goa	Beach
2	Dubai Explorer	1	Dubai	International
3	Kerala Trails	2	Kerala	Nature
4	Rajasthan Heritage	2	Rajasthan	Culture
5	Delhi Weekend	3	Delhi	City

bookings

booking_id	package_id	traveler_name	booking_status	booking_date
1	1	A. Shah	confirmed	2025-05-01
2	1	B. Rao	confirmed	2025-05-02
3	2	C. Jain	confirmed	2025-05-03
4	2	D. Nair	cancelled	2025-05-04
5	3	E. Singh	confirmed	2025-05-05

Output Format

agent_name	package_count	destination_count	confirmed_bookings
------------	---------------	-------------------	--------------------

Sample Output

agent_name	package_count	destination_count	confirmed_bookings
Amit Verma	2	2	3

Explanation

Only confirmed bookings are included. The query joins agents with the packages they manage and then connects those packages with their bookings.

COUNT(DISTINCT package_id) gives the number of packages, while COUNT(DISTINCT destination) determines the number of destinations handled by the agent.

Expected Output

agent_name	package_count	destination_count	confirmed_bookings
Amit Verma	4	3	18
Neha Kapoor	3	3	13

Constraints

Join all three tables.

Count only bookings where booking_status = 'confirmed'.

Cancelled bookings must be excluded.

Use COUNT(DISTINCT package_id).

Use COUNT(DISTINCT destination).

Only agents managing at least 2 distinct packages should appear.

Use GROUP BY and HAVING.

Order by confirmed_bookings descending and agent_name ascending.

QUESTION 5 – MONTHLY WEBSITE TRAFFIC COMPARISON

Section 2 - Medium SQL

Q1. MONTHLY WEBSITE TRAFFIC COMPARISON

A digital publishing company maintains website information in a websites table and stores monthly visitor statistics in a traffic_history table.

The analytics team wants to compare each month's visitor count with the immediately preceding month for the same website.

Task

Write an SQL query that joins the two tables and uses LAG() to retrieve the previous month's visitor count.

Also calculate the percentage increase or decrease from the previous month.

Output columns

website_name
traffic_month
visitor_count
previous_visitors
visitor_change
change_percentage

Input Format

Table: websites

```

Column      Type
website_id  INT
website_name VARCHAR(100)
website_category VARCHAR(50)

```

Table: traffic_history

```

Column      Type
website_id  INT
traffic_month INT
visitor_count INT

```

Sample Input

websites

website_id	website_name	website_category
1	TechDaily	Technology
2	TravelScope	Travel
3	FinanceWire	Finance

traffic_history

website_id	traffic_month	visitor_count
1	1	100000
1	2	125000
1	3	115000
2	1	70000
2	2	84000
2	3	90000

Output Format

```

website_name | traffic_month | visitor_count | previous_visitors |
visitor_change | change_percentage

```

Sample Output

website_name	traffic_month	visitor_count	previous_visitors	visitor_change	change_percentage
TechDaily	1	100000	NULL	NULL	NULL
TechDaily	2	125000	100000	25000	25.00
TechDaily	3	115000	125000	-10000	-8.00

Explanation

The LAG() function retrieves the previous visitor count for the same website.

The percentage change is calculated from the current and previous visitor counts.

Expected Output

website_name	traffic_month	visitor_count	previous_visitors	visitor_change	change_percentage
FinanceWire	1	60000	NULL	NULL	NULL
FinanceWire	2	72000	60000	12000	20.00
FinanceWire	3	69000	72000	-3000	-4.17
TravelScope	1	70000	NULL	NULL	NULL
TravelScope	2	84000	70000	14000	20.00
TravelScope	3	90000	84000	6000	7.14
TechDaily	1	100000	NULL	NULL	NULL
TechDaily	2	125000	100000	25000	25.00
TechDaily	3	115000	125000	-10000	-8.00

Constraints

Use the LAG() window function.

Partition by website.

Order by traffic_month ascending.

Return every available month.

First month's previous value and calculated differences must be NULL.

Round change_percentage to two decimal places.

Sort by website_name ascending and traffic_month ascending.

Q6. RUNNING DONATION TOTAL PER MEMBER

GiveHope, a community fundraising platform, is preparing a donor activity report. The organization wants to see every donation made by each member in chronological order along with a running total of that member's donations up to and including the current donation.

The report should also show the campaign to which each donation was made. This will help the fundraising team understand how individual members build their contribution over time.

The report should be sorted alphabetically by member name and chronologically by donation date.

Task:

Write a query joining the members, campaigns, and donations tables to return every donation along with the member name, campaign name, donation date, donation amount, and the member's running donation total.

Additionally, include the member's membership_tier in the output.

The output must contain the following columns:

member_name – Name of the member
membership_tier – Membership tier of the member
campaign_name – Campaign receiving the donation
donation_date – Date of the donation
donation_amount – Amount donated
running_total – Cumulative donation amount for the same member up to and including the current donation
Input Format

The query operates on three tables: members, campaigns, and donations.

TABLE 1 SCHEMA

Table: members

Column	Type	Description
member_id	INT	Unique identifier for the member
member_name	VARCHAR(100)	Name of the member
membership_tier	VARCHAR(20)	Membership tier of the member

TABLE 2 SCHEMA

Table: campaigns

Column	Type	Description
campaign_id	INT	Unique identifier for the campaign
campaign_name	VARCHAR(100)	Name of the campaign
cause	VARCHAR(50)	Cause supported by the campaign

TABLE 3 SCHEMA

Table: donations

Column	Type	Description
donation_id	INT	Unique identifier for the donation
member_id	INT	References members(member_id)
campaign_id	INT	References campaigns(campaign_id)
donation_date	DATE	Date the donation was made
donation_amount	DECIMAL(10,2)	Donation amount

SAMPLE INPUT

members

member_id	member_name	membership_tier
1	A. Mehta	Gold
2	R. Sharma	Silver
3	P. Nair	Gold

campaigns

campaign_id	campaign_name	cause
1	School Support	Education
2	Clean Water Community	
3	Health Drive	Healthcare

donations

donation_id	member_id	campaign_id	donation_date	donation_amount
1	1	1	2024-04-05	500.00
2	1	2	2024-04-18	750.00
3	1	1	2024-05-02	400.00
4	2	3	2024-04-10	300.00
5	2	2	2024-04-25	450.00

Output Format

Return the following columns in this exact order:

member_name | membership_tier | campaign_name | donation_date | donation_amount
| running_total

SAMPLE OUTPUT

member_name	membership_tier	campaign_name	donation_date	donation_amount	running_total
A. Mehta	Gold	School Support	2024-04-05	500.00	500.00
A. Mehta	Gold	Clean Water	2024-04-18	750.00	1250.00
A. Mehta	Gold	School Support	2024-05-02	400.00	1650.00
R. Sharma	Silver	Health Drive	2024-04-10	300.00	300.00
R. Sharma	Silver	Clean Water	2024-04-25	450.00	750.00

EXPLANATION

The running total must be calculated independently for every member.

For example, A. Mehta has donations of 500, 750, and 400, so the running totals are:

500

500 + 750 = 1250

500 + 750 + 400 = 1650

The cumulative calculation must never carry one member's donations into another member's total.

Expected Output

member_name	membership_tier	campaign_name	donation_date	donation_amount	running_total
A. Mehta	Gold	School Support	2024-04-05	500.00	500.00
A. Mehta	Gold	Clean Water	2024-04-18	750.00	1250.00
A. Mehta	Gold	School Support	2024-05-02	400.00	1650.00
P. Nair	Gold	Health Drive	2024-04-08	800.00	800.00
P. Nair	Gold	School Support	2024-05-06	350.00	1150.00
R. Sharma	Silver	Health Drive	2024-04-10	300.00	300.00
R. Sharma	Silver	Clean Water	2024-04-25	450.00	750.00

Constraints

Use joins between all three tables.

The member name and membership tier must come from the members table.

The campaign name must come from the campaigns table.

Use a window function to calculate running_total.
 Partition the running calculation by member_id.
 Order donations chronologically by donation_date.
 Include every donation.
 Sort the final result by member_name ascending, then donation_date ascending.
 Use the exact output column names specified.

QUESTION 7 – RUNNING ORDER VALUE PER CUSTOMER

FreshCart, an online grocery platform, wants to analyze customer purchasing behavior. The analytics team wants a report containing every order placed by a customer along with a running total of the customer's order value in chronological order.

The report should also identify the store from which each order was fulfilled and indicate whether the order was a high-value order.

An order should be considered high-value when its amount is at least 1000.00.

Task:

Write a query joining the customers, stores, and orders tables to return every order along with the customer name, store name, order date, order amount, cumulative spending, and a high-value indicator.

The output must contain:

customer_name – Name of the customer
 store_name – Store fulfilling the order
 order_date – Date of the order
 order_amount – Value of the order
 running_total – Cumulative order value for that customer up to and including the current order
 order_category – High Value when order_amount >= 1000, otherwise Regular
 Input Format

The query operates on three tables: customers, stores, and orders.

TABLE 1 SCHEMA

Table: customers

Column	Type	Description
customer_id	INT	Unique identifier for the customer
customer_name	VARCHAR(100)	Name of the customer
customer_tier	VARCHAR(20)	Customer membership level

TABLE 2 SCHEMA

Table: stores

Column	Type	Description
store_id	INT	Unique identifier for the store
store_name	VARCHAR(100)	Name of the store
city	VARCHAR(50)	City of the store

TABLE 3 SCHEMA

Table: orders

Column	Type	Description
order_id	INT	Unique identifier for the order
customer_id	INT	References customers(customer_id)
store_id	INT	References stores(store_id)

order_date DATE Date the order was placed
order_amount DECIMAL(10,2) Total order value
SAMPLE INPUT

customers

customer_id	customer_name	customer_tier
1	K. Deshmukh	Gold
2	M. Banerjee	Silver
3	P. Chandran	Bronze

stores

store_id	store_name	city
1	Central Market	Mumbai
2	Green Plaza	Mumbai
3	Fresh Point	Pune

orders

order_id	customer_id	store_id	order_date	order_amount
1	1	1	2024-03-05	750.00
2	1	2	2024-03-18	1200.00
3	1	1	2024-04-02	950.00
4	2	3	2024-03-10	1100.00
5	2	2	2024-03-25	650.00

Output Format

Return the following columns in this exact order:

customer_name | store_name | order_date | order_amount | running_total |
order_category

SAMPLE OUTPUT

customer_name	store_name	order_date	order_amount	running_total	order_category
K. Deshmukh	Central Market	2024-03-05	750.00	750.00	Regular
K. Deshmukh	Green Plaza	2024-03-18	1200.00	1950.00	High Value
K. Deshmukh	Central Market	2024-04-02	950.00	2900.00	Regular
M. Banerjee	Fresh Point	2024-03-10	1100.00	1100.00	High Value
M. Banerjee	Green Plaza	2024-03-25	650.00	1750.00	Regular

EXPLANATION

The running total is calculated separately for every customer.

For K. Deshmukh:

March 05 → 750

March 18 → 750 + 1200 = 1950

April 02 → 1950 + 950 = 2900

The order_category is independently determined from each order's amount.

Expected Output

customer_name	store_name	order_date	order_amount	running_total	order_category
K. Deshmukh	Central Market	2024-03-05	750.00	750.00	Regular
K. Deshmukh	Green Plaza	2024-03-18	1200.00	1950.00	High Value
K. Deshmukh	Central Market	2024-04-02	950.00	2900.00	Regular
K. Deshmukh	Fresh Point	2024-04-20	1500.00	4400.00	High Value
M. Banerjee	Fresh Point	2024-03-10	1100.00	1100.00	High Value
M. Banerjee	Green Plaza	2024-03-25	650.00	1750.00	Regular
M. Banerjee	Central Market	2024-04-12	1350.00	3100.00	High Value
P. Chandran	Central Market	2024-03-15	500.00	500.00	Regular
P. Chandran	Fresh Point	2024-04-08	1250.00	1750.00	High Value

Constraints

Join all three tables.

Use a window function to calculate the cumulative order value.

Partition the running total by customer.

Order each customer's rows chronologically by order_date.

Return every order.

Do not combine spending between different customers.

Use CASE to calculate order_category.

An order of exactly 1000.00 must be classified as High Value.

Sort by customer_name ascending and order_date ascending.

Use the exact output column names specified.

QUESTION 8

Q1. DELIVERIES ABOVE THEIR DRIVER'S AVERAGE LOAD

MetroMove Couriers is reviewing the workload assigned to its delivery drivers. Each driver completes multiple deliveries with different package weights, and the operations team wants to identify deliveries that are heavier than what that driver typically handles.

Specifically, the team wants every delivery whose package weight is strictly greater than the average package weight of all deliveries handled by the same driver. The report should be arranged alphabetically by driver name, and within each driver, the heaviest qualifying delivery should appear first.

Task:

Write a query joining the deliveries, depots, and drivers tables to return every delivery whose package weight exceeds the average package weight across all deliveries handled by the same driver.

The result should include the delivery identifier, the depot from which the delivery originated, the driver handling the delivery, and the package weight.

The output must contain the following columns:

delivery_id - Unique identifier for the delivery
depot_name - Name of the depot from which the delivery originated
driver_name - Name of the driver handling the delivery
package_weight_kg - Weight of the delivery package in kilograms
Input Format

The query operates on three tables: depots, drivers, and deliveries.

TABLE 1 SCHEMA

Table: depots

Column	Type	Description
depot_id	INT	Unique identifier for the depot
depot_name	VARCHAR(100)	Name of the depot
city	VARCHAR(50)	City where the depot is located

TABLE 2 SCHEMA

Table: drivers

Column	Type	Description
--------	------	-------------

```
driver_id    INT    Unique identifier for the driver
driver_name  VARCHAR(100)    Name of the driver
vehicle_count INT    Number of vehicles assigned to the driver
TABLE 3 SCHEMA
```

Table: deliveries

```
Column      Type    Description
delivery_id INT    Unique identifier for the delivery
depot_id     INT    References depots(depot_id)
driver_id    INT    References drivers(driver_id)
delivery_date DATE    Date on which the delivery was dispatched
package_weight_kg DECIMAL(7,2)    Weight of the delivery package in kilograms
SAMPLE INPUT
```

All three tables are pre-populated. The first 5 rows of each table are shown below.

Sample rows from depots

```
depot_id  depot_name  city
1         Pune Central  Pune
2         Chennai Hub  Chennai
3         Kolkata Hub  Kolkata
```

Sample rows from drivers

```
driver_id  driver_name  vehicle_count
1          Arrow Delivery    12
2          Swift Carrier     8
3          Urban Express    15
```

Sample rows from deliveries

```
delivery_id depot_id  driver_id  delivery_date  package_weight_kg
1           1        1        2025-05-01    60.00
2           2        1        2025-05-02    80.00
3           1        1        2025-05-03    100.00
4           3        1        2025-05-04    120.00
5           2        1        2025-05-05    140.00
```

Output Format

Return the following columns, in this exact order:

```
delivery_id | depot_name | driver_name | package_weight_kg
SAMPLE OUTPUT
```

Applying the query to the 5-row sample subset shown above, the expected output would be:

```
delivery_id depot_name  driver_name  package_weight_kg
5          Chennai Hub  Arrow Delivery    140.00
4          Kolkata Hub  Arrow Delivery    120.00
Explanation
```

For the five sample deliveries handled by Arrow Delivery, the average package weight is:

$(60 + 80 + 100 + 120 + 140) / 5 = 100$ kg

Only deliveries weighing more than 100 kg qualify. Therefore, deliveries 4 and 5 are returned.

The results are ordered alphabetically by driver_name, and within each driver by package_weight_kg descending.

EXPECTED OUTPUT

The following is the complete expected output from the entire pre-populated

dataset:

delivery_id	depot_name	driver_name	package_weight_kg
20	Pune Central	Arrow Delivery	175.00
5	Chennai Hub	Arrow Delivery	140.00
4	Kolkata Hub	Arrow Delivery	120.00
19	Chennai Hub	Swift Carrier	310.00
9	Pune Central	Swift Carrier	270.00
24	Kolkata Hub	Urban Express	840.00
14	Kolkata Hub	Urban Express	720.00
23	Chennai Hub	Urban Express	670.00
13	Chennai Hub	Urban Express	620.00
22	Pune Central	Urban Express	560.00

Constraints

Keywords, table names, column names, and record values are case-sensitive.

All records are pre-populated. No console input is required.

Use the exact output column names: delivery_id, depot_name, driver_name, and package_weight_kg.

Each delivery must be compared against the average package weight of deliveries handled by the same driver.

Only deliveries whose package weight is strictly greater than the driver-specific average should appear in the result.

The output must be ordered by driver_name in ascending alphabetical order, and within each driver by package_weight_kg in descending order.

QUESTION 9 - PURCHASES ABOVE THEIR CASHIER'S TYPICAL BILL VALUE

QuickMart Retail is analyzing billing activity across its stores. Each cashier processes several customer purchases, and the finance team wants to identify unusually large purchases compared with the cashier's normal transaction value.

Specifically, the report should list every purchase whose total value is greater than the average purchase value for the cashier who processed it. The output should be sorted alphabetically by cashier name, with the largest qualifying purchases listed first for each cashier.

Task:

Write a query joining the purchases, stores, and cashiers tables to return every purchase whose bill amount exceeds the average bill amount of all purchases processed by the same cashier.

The output must contain:

purchase_id - Unique identifier for the purchase
store_name - Name of the store where the purchase was made
cashier_name - Name of the cashier who processed the purchase
bill_amount - Total value of the purchase
Input Format

The query operates on three tables: stores, cashiers, and purchases.

TABLE 1 SCHEMA

Table: stores

Column	Type	Description
store_id	INT	Unique identifier for the store
store_name	VARCHAR(100)	Name of the store
city	VARCHAR(50)	City where the store is located

TABLE 2 SCHEMA

Table: cashiers

Column	Type	Description
cashier_id	INT	Unique identifier for the cashier
cashier_name	VARCHAR(100)	Name of the cashier
shift_hours	INT	Number of working hours in the cashier's shift

TABLE 3 SCHEMA

Table: purchases

Column	Type	Description
purchase_id	INT	Unique identifier for the purchase
store_id	INT	References stores(store_id)
cashier_id	INT	References cashiers(cashier_id)
purchase_date	DATE	Date of the purchase
bill_amount	DECIMAL(8,2)	Total purchase amount

SAMPLE INPUT

Sample rows from stores

store_id	store_name	city
1	Andheri Market	Mumbai
2	Powai Store	Mumbai
3	Vashi Outlet	Navi Mumbai

Sample rows from cashiers

cashier_id	cashier_name	shift_hours
1	Ananya Retail	8
2	Bharat Sales	9
3	Charu Mart	7

Sample rows from purchases

purchase_id	store_id	cashier_id	purchase_date	bill_amount
1	1	1	2025-06-01	120.00
2	2	1	2025-06-02	150.00
3	1	1	2025-06-03	180.00
4	3	1	2025-06-04	210.00
5	2	1	2025-06-05	240.00

Output Format

Return the following columns, in this exact order:

purchase_id | store_name | cashier_name | bill_amount

SAMPLE OUTPUT

purchase_id	store_name	cashier_name	bill_amount
5	Powai Store	Ananya Retail	240.00
4	Vashi Outlet	Ananya Retail	210.00

Explanation

For Ananya Retail, the average bill amount is:

$(120 + 150 + 180 + 210 + 240) / 5 = 180$

Only purchases greater than 180 qualify, so purchases 4 and 5 are returned.

EXPECTED OUTPUT

purchase_id	store_name	cashier_name	bill_amount
20	Vashi Outlet	Ananya Retail	360.00
5	Powai Store	Ananya Retail	240.00
4	Vashi Outlet	Ananya Retail	210.00
19	Powai Store	Bharat Sales	520.00
9	Andheri Market	Bharat Sales	430.00
24	Andheri Market	Charu Mart	780.00
14	Vashi Outlet	Charu Mart	690.00
23	Vashi Outlet	Charu Mart	640.00
13	Powai Store	Charu Mart	590.00
22	Andheri Market	Charu Mart	540.00

Constraints

Keywords, table names, column names, and record values are case-sensitive.
All records are pre-populated. No console input is required.
Use the exact output column names: purchase_id, store_name, cashier_name, and bill_amount.
Every purchase must be evaluated against the average bill amount of purchases processed by the same cashier.
Only purchases whose bill amount is strictly greater than that cashier-specific average should be included.
Sort by cashier_name ascending, followed by bill_amount descending within each cashier.

QUESTION - 10. TECHNICIANS AND THEIR ASSIGNED SUPERVISORS

ServiceCore Solutions is creating an internal staff directory for its technical support teams. Management wants to see every technician who reports to a supervisor, along with the technician's designation, team, and the name of the supervisor they report to.

Employees who do not have an assigned supervisor should not appear in the directory. The final list should be ordered alphabetically by team and then alphabetically by technician name.

Task:

Write a query joining the technicians, designations, and teams tables, while also joining the technicians table to itself to obtain the supervisor's name.

Use an alias such as supervisors for the second instance of the technicians table.

The output must contain:

technician_name - Name of the technician
designation_name - Designation of the technician
team_name - Team to which the technician belongs
supervisor_name - Name of the technician's direct supervisor
Input Format

The query operates on three tables: teams, designations, and technicians.

The technicians table references itself through the supervisor_id column.

TABLE 1 SCHEMA

Table: teams

Column	Type	Description
team_id	INT	Unique identifier for the team
team_name	VARCHAR(100)	Name of the team
location	VARCHAR(50)	Location of the team

TABLE 2 SCHEMA

Table: designations

Column	Type	Description
designation_id	INT	Unique identifier for the designation
designation_name	VARCHAR(100)	Name of the designation
grade_level	INT	Seniority level associated with the designation

TABLE 3 SCHEMA

Table: technicians

Column	Type	Description
technician_id	INT	Unique identifier for the technician
technician_name	VARCHAR(100)	Name of the technician
designation_id	INT	References designations(designation_id)
team_id	INT	References teams(team_id)
supervisor_id	INT	References technicians(technician_id); NULL if no supervisor

SAMPLE INPUT

Sample rows from teams

team_id	team_name	location
1	Network Support	Mumbai
2	Cloud Support	Pune
3	Platform Support	Bengaluru

Sample rows from designations

designation_id	designation_name	grade_level
1	Support Engineer	1
2	Senior Support Engineer	2
3	Technical Lead	3
4	Support Manager	4
5	Service Director	5

Sample rows from technicians

technician_id	technician_name	designation_id	team_id	supervisor_id
1	A. Mehta	5	1	NULL
2	B. Rao	4	1	1
3	C. Shah	3	1	2
4	D. Nair	2	1	3
5	E. Joshi	1	1	3

Output Format

Return the following columns, in this exact order:

technician_name | designation_name | team_name | supervisor_name

SAMPLE OUTPUT

technician_name	designation_name	team_name	supervisor_name
B. Rao	Support Manager	Network Support	A. Mehta
C. Shah	Technical Lead	Network Support	B. Rao
D. Nair	Senior Support Engineer	Network Support	C. Shah
E. Joshi	Support Engineer	Network Support	C. Shah

Explanation

A. Mehta has a NULL supervisor_id, so the self-join does not find a supervisor for that row and the employee is excluded.

The other four technicians have valid supervisor references. For example:

B. Rao → A. Mehta
 C. Shah → B. Rao
 D. Nair → C. Shah
 E. Joshi → C. Shah

The output is ordered by team_name ascending, followed by technician_name ascending.

EXPECTED OUTPUT

technician_name	designation_name	team_name	supervisor_name
B. Rao	Support Manager	Network Support	A. Mehta
C. Shah	Technical Lead	Network Support	B. Rao
D. Nair	Senior Support Engineer	Network Support	C. Shah
E. Joshi	Support Engineer	Network Support	C. Shah
K. Fernandes	Support Engineer	Network Support	C. Shah
L. Kulkarni	Senior Support Engineer	Network Support	C. Shah
M. Das	Technical Lead	Network Support	B. Rao
N. Singh	Support Engineer	Network Support	M. Das
O. Kapoor	Senior Support Engineer	Network Support	M. Das

V. Pillai	Support Engineer	Network Support	M. Das
G. Arora	Technical Lead	Cloud Support	F. Iyer
H. Menon	Support Engineer	Cloud Support	G. Arora
P. Sethi	Support Manager	Cloud Support	F. Iyer
Q. Bansal	Technical Lead	Cloud Support	P. Sethi
R. Verma	Senior Support Engineer	Cloud Support	G. Arora
S. Jain	Support Engineer	Cloud Support	Q. Bansal
W. Shah	Senior Support Engineer	Cloud Support	Q. Bansal

Constraints

Keywords, table names, column names, and record values are case-sensitive.

All records are pre-populated. No console input is required.

Use the exact output column names: technician_name, designation_name, team_name, and supervisor_name.

The technicians table must be joined to itself using an alias such as supervisors to retrieve the supervisor's name.

Use an INNER JOIN for the self-join so technicians with supervisor_id = NULL are excluded.

The output must be ordered by team_name in ascending alphabetical order, followed by technician_name in ascending alphabetical order.

QUESTION 11

Q4. CONSULTANTS AND THEIR REPORTING LEADS

InsightWorks Consulting is preparing an internal directory for its consulting teams. The directory should show every consultant who reports to another employee, together with the consultant's job title, division, and the name of their reporting lead.

Employees positioned at the top of a division who do not have a reporting lead should be excluded.

The report should first be ordered alphabetically by division and then alphabetically by consultant name.

Task:

Write a query joining the consultants, titles, and divisions tables, and join the consultants table to itself to determine each consultant's reporting lead.

Use an alias such as leads for the second instance of the consultants table.

The output must contain:

consultant_name - Name of the consultant
 title_name - Job title of the consultant
 division_name - Division to which the consultant belongs
 lead_name - Name of the consultant's direct reporting lead

Input Format

The query operates on three tables: divisions, titles, and consultants.

The consultants table references itself using the lead_id column.

TABLE 1 SCHEMA

Table: divisions

Column	Type	Description
division_id	INT	Unique identifier for the division
division_name	VARCHAR(100)	Name of the division

office_city VARCHAR(50) City where the division is based
TABLE 2 SCHEMA

Table: titles

Column	Type	Description
title_id	INT	Unique identifier for the title
title_name	VARCHAR(100)	Name of the consultant's title
level_no	INT	Seniority level associated with the title

TABLE 3 SCHEMA

Table: consultants

Column	Type	Description
consultant_id	INT	Unique identifier for the consultant
consultant_name	VARCHAR(100)	Name of the consultant
title_id	INT	References titles(title_id)
division_id	INT	References divisions(division_id)
lead_id	INT	References consultants(consultant_id); NULL if no reporting lead

SAMPLE INPUT

Sample rows from divisions

division_id	division_name	office_city
1	Advisory	Mumbai
2	Analytics	Pune
3	Strategy	Delhi

Sample rows from titles

title_id	title_name	level_no
1	Business Analyst	1
2	Senior Business Analyst	2
3	Project Lead	3
4	Engagement Manager	4
5	Practice Director	5

Sample rows from consultants

consultant_id	consultant_name	title_id	division_id	lead_id
1	A. Kapoor	5	1	NULL
2	B. Malhotra	4	1	1
3	C. Sinha	3	1	2
4	D. Roy	2	1	3
5	E. Gupta	1	1	3

Output Format

Return the following columns, in this exact order:

consultant_name | title_name | division_name | lead_name

SAMPLE OUTPUT

consultant_name	title_name	division_name	lead_name
B. Malhotra	Engagement Manager	Advisory	A. Kapoor
C. Sinha	Project Lead	Advisory	B. Malhotra
D. Roy	Senior Business Analyst	Advisory	C. Sinha
E. Gupta	Business Analyst	Advisory	C. Sinha

Explanation

A. Kapoor has a NULL value in lead_id, so there is no matching lead record and that consultant is omitted.

The remaining four consultants have valid reporting relationships and are returned.

The final output is sorted by division_name ascending, followed by consultant_name ascending.

EXPECTED OUTPUT

consultant_name	title_name	division_name	lead_name
-----------------	------------	---------------	-----------

B. Malhotra	Engagement Manager	Advisory	A. Kapoor
C. Sinha	Project Lead	Advisory	B. Malhotra
D. Roy	Senior Business Analyst	Advisory	C. Sinha
E. Gupta	Business Analyst	Advisory	C. Sinha
K. Rao	Business Analyst	Advisory	C. Sinha
L. Iyer	Senior Business Analyst	Advisory	C. Sinha
M. Shah	Project Lead	Advisory	B. Malhotra
N. Mehta	Business Analyst	Advisory	M. Shah
O. Desai	Senior Business Analyst	Advisory	M. Shah
V. Nair	Business Analyst	Advisory	M. Shah
G. Joshi	Project Lead	Analytics	F. Sharma
H. Jain	Business Analyst	Analytics	G. Joshi
P. Reddy	Engagement Manager	Analytics	F. Sharma
Q. Mehra	Project Lead	Analytics	P. Reddy
R. Bose	Senior Business Analyst	Analytics	G. Joshi
S. Arora	Business Analyst	Analytics	Q. Mehra
W. Khanna	Senior Business Analyst	Analytics	Q. Mehra

Constraints

Keywords, table names, column names, and record values are case-sensitive.

All records are pre-populated. No console input is required.

Use the exact output column names: consultant_name, title_name, division_name, and lead_name.

The consultants table must be joined to itself using an alias such as leads.

Use an INNER JOIN for the self-join so consultants with lead_id = NULL are excluded.

The output must be ordered by division_name in ascending alphabetical order, and within each division by consultant_name in ascending alphabetical order.

QUESTION 12 – TOP-PERFORMING PRODUCTS IN EACH CATEGORY

Q1. TOP PRODUCT BY REVENUE WITHIN EACH CATEGORY

ShopSphere is reviewing product performance across its different merchandise categories. The analytics team wants to identify the highest-revenue product within every category.

If multiple products in the same category have the same revenue, the product with the smaller product_id should be ranked first. Only the top-performing product from each category should appear in the report.

The report should be ordered alphabetically by category name.

Task:

Write a query joining the products and sales tables to calculate the total revenue generated by every product.

Use a window function such as ROW_NUMBER() to identify the highest-revenue product within each category.

The result should contain:

category_name – Name of the product category

product_name – Name of the product

total_revenue – Total revenue generated by the product

Input Format

The query operates on two tables: products and sales.

TABLE 1 SCHEMA

Table: products

Column	Type	Description
product_id	INT	Unique identifier for the product
product_name	VARCHAR(100)	Name of the product
category_name	VARCHAR(100)	Category to which the product belongs

TABLE 2 SCHEMA

Table: sales

Column	Type	Description
sale_id	INT	Unique identifier for the sale
product_id	INT	References products(product_id)
sale_date	DATE	Date of the sale
quantity	INT	Number of units sold
unit_price	DECIMAL(10,2)	Selling price per unit

SAMPLE INPUT

Sample rows from products

product_id	product_name	category_name
1	Wireless Mouse	Accessories
2	Mechanical Keyboard	Accessories
3	Smart Monitor	Electronics
4	Bluetooth Speaker	Electronics
5	Office Chair	Furniture

Sample rows from sales

sale_id	product_id	sale_date	quantity	unit_price
1	1	2025-06-01	10	500.00
2	1	2025-06-05	8	500.00
3	2	2025-06-02	12	800.00
4	3	2025-06-03	5	2000.00
5	4	2025-06-04	8	900.00

Output Format

Return the following columns in this exact order:

category_name	product_name	total_revenue
Accessories	Mechanical Keyboard	9600.00
Electronics	Smart Monitor	10000.00

Explanation

For each product, revenue is calculated as:

quantity × unit_price

The total revenue from all sales is then calculated for each product.

ROW_NUMBER() is used with the category as the partition and total revenue in descending order. Therefore, only the product ranked 1 within each category is returned.

EXPECTED OUTPUT

category_name	product_name	total_revenue
Accessories	Mechanical Keyboard	15600.00
Electronics	Smart Monitor	26800.00
Furniture	Executive Desk	22500.00
Home Appliances	Air Purifier	18400.00

Constraints

All records are pre-populated.

Join the products and sales tables using product_id.

Calculate product revenue using quantity * unit_price.

Use SUM() and GROUP BY to obtain total revenue for each product.

Use ROW_NUMBER() with PARTITION BY category_name.
 Rank products by total_revenue DESC.
 If revenues are equal, use product_id ASC as the tie-breaker.
 Return only the highest-ranked product from each category.
 Sort the final result by category_name ASC.
 QUESTION 13 – EMPLOYEES ABOVE THEIR DEPARTMENT'S SALARY AVERAGE
 Q2. EMPLOYEES EARNING MORE THAN THEIR DEPARTMENT AVERAGE

TechNova Systems wants to conduct a compensation review across its departments.
 The HR team wants to find employees whose salary is higher than the average salary of all employees working in the same department.

Employees earning exactly the department average should not be included.

The report should show the employee's department and position and should be ordered alphabetically by department, followed by salary from highest to lowest.

Task:

Write a query joining the employees, departments, and positions tables to return employees whose salary is strictly greater than the average salary of employees in their department.

The output must contain:

employee_name – Name of the employee
 position_name – Position held by the employee
 department_name – Department of the employee
 salary – Employee's salary
 Input Format

The query operates on three tables: departments, positions, and employees.

TABLE 1 SCHEMA

Table: departments

Column	Type	Description
department_id	INT	Unique department identifier
department_name	VARCHAR(100)	Name of the department
location	VARCHAR(50)	Department location

TABLE 2 SCHEMA

Table: positions

Column	Type	Description
position_id	INT	Unique position identifier
position_name	VARCHAR(100)	Name of the position
level_no	INT	Seniority level

TABLE 3 SCHEMA

Table: employees

Column	Type	Description
employee_id	INT	Unique employee identifier
employee_name	VARCHAR(100)	Name of the employee
position_id	INT	References positions(position_id)
department_id	INT	References departments(department_id)
salary	DECIMAL(10,2)	Employee salary

SAMPLE INPUT

Sample rows from departments

department_id	department_name	location
1	Engineering	Mumbai
2	Finance	Pune

3 Marketing Delhi
Sample rows from positions

position_id	position_name	level_no
1	Software Engineer	1
2	Senior Engineer	2
3	Team Lead	3
4	Manager	4

Sample rows from employees

employee_id	employee_name	position_id	department_id	salary
1	A. Sharma	1	1	50000.00
2	B. Reddy	2	1	70000.00
3	C. Nair	3	1	90000.00
4	D. Iyer	1	2	45000.00
5	E. Khan	2	2	65000.00

Output Format

employee_name | position_name | department_name | salary

SAMPLE OUTPUT

employee_name	position_name	department_name	salary
C. Nair	Team Lead	Engineering	90000.00
E. Khan	Senior Engineer	Finance	65000.00

Explanation

For Engineering:

$(50000 + 70000 + 90000) / 3 = 70000$

Only C. Nair earns more than the Engineering average.

For Finance:

$(45000 + 65000) / 2 = 55000$

E. Khan earns more than the Finance average.

The average must be calculated independently for each department.

EXPECTED OUTPUT

employee_name	position_name	department_name	salary
C. Nair	Team Lead	Engineering	90000.00
H. Mehta	Manager	Engineering	125000.00
L. Kapoor	Senior Engineer	Engineering	95000.00
E. Khan	Senior Engineer	Finance	65000.00
P. Shah	Manager	Finance	105000.00
G. Joshi	Team Lead	Marketing	72000.00

Constraints

Join all three tables.

The employee's department average must be calculated separately for each department.

Use an aggregate function such as AVG().

Only employees whose salary is strictly greater than the department average should be returned.

Employees whose salary equals the average must be excluded.

Use the exact output column names specified.

Sort by department_name ASC, then salary DESC.

QUESTION 14 – CUSTOMERS WITH MULTIPLE COMPLETED ORDERS

Q3. CUSTOMERS WITH REPEATED COMPLETED PURCHASES

QuickBasket is analyzing customer ordering patterns. The business team wants to identify customers who have completed more than one order and calculate their total completed spending.

Orders that are cancelled or pending must not be included in the calculations.

The report should display the customer's membership level, number of completed

orders, and total amount spent.

Task:

Write a query joining the customers, orders, and stores tables to identify customers with at least 2 completed orders.

For each qualifying customer, calculate:

Number of completed orders
Total completed order value

Also display the store associated with the customer's most recent completed order.

The output must contain:

customer_name
membership_level
completed_orders
total_spent
latest_store
Input Format
TABLE 1 SCHEMA

Table: customers

Column	Type	Description
customer_id	INT	Unique identifier for the customer
customer_name	VARCHAR(100)	Name of the customer
membership_level	VARCHAR(30)	Customer membership category

TABLE 2 SCHEMA

Table: orders

Column	Type	Description
order_id	INT	Unique order identifier
customer_id	INT	References customers(customer_id)
store_id	INT	References stores(store_id)
order_date	DATE	Date of the order
order_amount	DECIMAL(10,2)	Order amount
order_status	VARCHAR(20)	Current status of the order

TABLE 3 SCHEMA

Table: stores

Column	Type	Description
store_id	INT	Unique store identifier
store_name	VARCHAR(100)	Name of the store
city	VARCHAR(50)	Store city

SAMPLE INPUT

Customers

customer_id	customer_name	membership_level
1	R. Mehta	Gold
2	S. Rao	Silver
3	K. Shah	Bronze

Stores

store_id	store_name	city
1	Central Market	Mumbai
2	Green Plaza	Pune
3	Fresh Corner	Delhi

Orders

order_id	customer_id	store_id	order_date	order_amount	order_status
1	1	1	2025-05-01	800.00	completed

2	1	2	2025-05-08	1200.00	completed
3	1	1	2025-05-15	500.00	cancelled
4	2	3	2025-05-03	900.00	completed
5	2	2	2025-05-20	700.00	pending

Output Format

customer_name | membership_level | completed_orders | total_spent | latest_store

SAMPLE OUTPUT

customer_name	membership_level	completed_orders	total_spent	latest_store
R. Mehta	Gold	2	2000.00	Green Plaza

Explanation

R. Mehta has two completed orders:

$800 + 1200 = 2000$

The cancelled order is ignored.

S. Rao has only one completed order, so that customer does not satisfy the minimum requirement of two completed orders.

The most recent completed order for R. Mehta was made at Green Plaza.

EXPECTED OUTPUT

customer_name	membership_level	completed_orders	total_spent	latest_store
R. Mehta	Gold	4	4650.00	Fresh Corner
S. Rao	Silver	3	3150.00	Green Plaza
T. Iyer	Gold	2	2800.00	Central Market
V. Nair	Silver	5	6120.00	Fresh Corner

Constraints

Join the three tables through their respective identifiers.

Only orders with order_status = 'completed' should be considered.

Cancelled and pending orders must not affect completed_orders or total_spent.

Use COUNT() to determine completed order count.

Use SUM() to calculate total completed spending.

Only customers with at least 2 completed orders should appear.

The most recent completed order must be used to determine latest_store.

Use GROUP BY and an appropriate subquery or window-function approach where required.

Sort the output by total_spent DESC, then customer_name ASC.

QUESTION 15 – COMPARE CURRENT SALES WITH NEXT MONTH

Q4. MONTHLY SALES OUTLOOK BY STORE

RetailPulse maintains store information along with monthly sales figures. The management team wants to examine how each store's sales compare with the following month.

For every recorded month, the report should display the current month's sales and the sales value of the immediately following month for the same store.

The report should also classify the current month as:

Growing when next month's sales are higher

Declining when next month's sales are lower

Stable when both values are equal

No Future Data when there is no following month

Task:

Write a query joining the stores and monthly_sales tables and use the LEAD() window function to obtain the next month's sales for every store.

Also calculate the difference between next month's sales and the current month's sales.

The output must contain:

store_name
sales_month
current_sales
next_month_sales
sales_difference
trend

Input Format
TABLE 1 SCHEMA

Table: stores

Column	Type	Description
store_id	INT	Unique identifier for the store
store_name	VARCHAR(100)	Name of the store
city	VARCHAR(50)	Store city

TABLE 2 SCHEMA

Table: monthly_sales

Column	Type	Description
store_id	INT	References stores(store_id)
sales_month	INT	Month number
sales_amount	DECIMAL(10,2)	Sales recorded during the month

SAMPLE INPUT

Stores

store_id	store_name	city
1	Phoenix Mall	Mumbai
2	City Square	Pune
3	Metro Plaza	Delhi

Monthly Sales

store_id	sales_month	sales_amount
1	1	100000.00
1	2	125000.00
1	3	118000.00
1	4	140000.00
2	1	80000.00
2	2	80000.00
2	3	92000.00

Output Format

store_name	sales_month	current_sales	next_month_sales	sales_difference	trend
------------	-------------	---------------	------------------	------------------	-------

SAMPLE OUTPUT

store_name	sales_month	current_sales	next_month_sales	sales_difference	trend
City Square	1	80000.00	80000.00	0.00	Stable
City Square	2	80000.00	92000.00	12000.00	Growing
Phoenix Mall	1	100000.00	125000.00	25000.00	Growing
Phoenix Mall	2	125000.00	118000.00	-7000.00	Declining
Phoenix Mall	3	118000.00	140000.00	22000.00	Growing
Phoenix Mall	4	140000.00	NULL	NULL	No Future Data

Explanation

LEAD() retrieves the immediately following month's sales within the same store.

For example:

Month 1 → current = 100000

Month 2 → next = 125000

Difference = 125000 - 100000 = 25000

The final month has no following record, so LEAD() returns NULL and the trend becomes No Future Data.

EXPECTED OUTPUT

store_name	sales_month	current_sales	next_month_sales	sales_difference	trend
City Square 1		80000.00	80000.00	0.00	Stable
City Square 2		80000.00	92000.00	12000.00	Growing
City Square 3		92000.00	87000.00	-5000.00	Declining
City Square 4		87000.00	NULL	NULL	No Future Data
Metro Plaza 1		65000.00	71000.00	6000.00	Growing
Metro Plaza 2		71000.00	71000.00	0.00	Stable
Metro Plaza 3		71000.00	68000.00	-3000.00	Declining
Phoenix Mall	1	100000.00	125000.00	25000.00	Growing
Phoenix Mall	2	125000.00	118000.00	-7000.00	Declining
Phoenix Mall	3	118000.00	140000.00	22000.00	Growing
Phoenix Mall	4	140000.00	NULL	NULL	No Future Data

Constraints

Join the stores and monthly_sales tables.

Use LEAD() with an appropriate PARTITION BY and ORDER BY.

The next month's value must be calculated independently for each store.

Order each store's records by sales_month ASC.

sales_difference should be calculated as:

next_month_sales - current_sales

Use CASE to determine the trend.

A NULL next month's value must be classified as No Future Data.

Equal sales values must be classified as Stable.

Sort by store_name ASC, followed by sales_month ASC.

QUESTION 1 – STUDY SESSION TRACKER

Problem Statement

A learning application helps students maintain records of the time they spend on different study subjects such as Mathematics, Programming, Science, and English. Students currently maintain these records manually, making it difficult to determine their total study time, identify the subject receiving the most attention, or find subjects that exceed a particular study-time target.

The application requires a simple Python-based system that can store study time by subject and provide basic analytical information.

Objective

Design and implement a Python class named `StudyTracker` that models a basic study tracking system. The class should use a dictionary to store subject-wise study data. Students are expected to demonstrate their understanding of class creation, instance attributes, dictionary operations, filtering, and method implementation.

Task

Class Structure

Class Name: `StudyTracker`

Public Attributes:

`study_hours`

Type: dictionary

A dictionary where each key is a subject name (string) and its corresponding value is the total number of hours spent studying that subject (integer).

This attribute must be initialized inside the constructor method.

Methods

Method 1: `add_study(self, subject, hours)`

Signature:

```
def add_study(self, subject, hours)
```

Parameters:

`subject` (string) – The name of the study subject.

`hours` (integer) – The number of study hours to add.

Logic: Check whether the subject already exists in the `study_hours` dictionary. If it exists, add the given hours to the existing total. If it does not exist, create a new entry with the given hours. Return the updated dictionary.

Returns: dictionary

Sample Input and Output

```
tracker = StudyTracker({"Python": 10, "Math": 6})
```

```
print(tracker.study_hours)
```

Output:

```
{'Python': 10, 'Math': 6}
```

Adding a new subject:

```
tracker.add_study("Science", 4)
print(tracker.study_hours)
```

Output:

```
{'Python': 10, 'Math': 6, 'Science': 4}
```

Adding to an existing subject:

```
tracker.add_study("Python", 5)
print(tracker.study_hours)
```

Output:

```
{'Python': 15, 'Math': 6, 'Science': 4}
Method 2: get_total(self)
```

Signature:

```
def get_total(self)
```

Parameters: None

Logic: Calculate and return the total study hours across all subjects. If the dictionary is empty, return zero.

Returns: integer

Example:

```
print(tracker.get_total())
```

Output:

```
25
Method 3: highest_subject(self)
```

Signature:

```
def highest_subject(self)
```

Parameters: None

Logic: Find and return the subject having the highest total study hours. If the dictionary is empty, return an empty string.

Returns: string

Example:

```
print(tracker.highest_subject())
```

Output:

```
Python
Method 4: subjects_above(self, threshold)
```

Signature:

```
def subjects_above(self, threshold)
```

Parameters:

threshold (integer) – The study-hour value to compare against.

Logic: Find all subjects whose total study hours are strictly greater than the given threshold. Return the subject names as a list sorted in alphabetical order. If no subject exceeds the threshold, return an empty list.

Returns: list

Example:

```
print(tracker.subjects_above(5))
```

Output:

```
['Math', 'Python']
```

Note

The constructor must be defined as follows:

```
def __init__(self, study_hours=None)
```

If a dictionary is provided, store it as the study_hours attribute. If no dictionary is provided, initialize study_hours as an empty dictionary.

Write input/output code for testing, making sure it is placed inside:

```
if __name__ == "__main__":
```

```
    # your test input/output code
```

Expected Input and Output

Input:

```
tracker = StudyTracker({"Python": 10, "Math": 6})
```

```
tracker.add_study("Science", 4)
```

```
tracker.add_study("Python", 5)
```

```
print(tracker.study_hours)
```

```
print(tracker.get_total())
```

```
print(tracker.highest_subject())
```

```
print(tracker.subjects_above(5))
```

Expected Output:

```
{'Python': 15, 'Math': 6, 'Science': 4}
```

```
25
```

```
Python
```

```
['Math', 'Python']
```

Constraints

The number of subjects will not exceed 500.

Each subject name is a non-empty string.

Subject names are case-sensitive.

Study hours are always non-negative integers.

Adding to an existing subject accumulates the hours.

subjects_above uses a strict comparison and returns results alphabetically.

QUESTION 2 – FITNESS ACTIVITY ANALYZER

Problem Statement

A fitness application records the number of calories burned by users during different activities. Each activity can be performed multiple times, and every session records the calories burned during that session.

The application needs a programmatic system that can calculate average calories burned per activity, count activities according to their type, calculate total

calories, and identify the activity with the highest total calorie expenditure.

Objective

Design a Python class named `FitnessAnalyzer` that models the fitness analysis system. The class must use dictionaries and lists to organize activity information and perform different analytical operations.

Task

Class Structure

Class Name: `FitnessAnalyzer`

Public Attributes:

`activities`

Type: dictionary

A dictionary whose keys are activity names and whose values are dictionaries containing:

`type` – activity type as a string.

`sessions` – list of integers representing calories burned in individual sessions.

The data must be stored in the order supplied by the caller.

The attribute must be initialized inside the constructor method.

Methods

Method 1: `init(self, activities)`

Signature:

```
def __init__(self, activities)
```

Parameters:

`activities` (dict) – Supplied by the caller during object creation.

Logic: Store the supplied fitness activity data on the instance so that all analytical methods can access it.

Returns: None

Method 2: `average_calories(self)`

Signature:

```
def average_calories(self) -> dict
```

Logic: Calculate the arithmetic mean of calories burned during all sessions for every activity. The result must be expressed as a float.

Returns: dictionary

Method 3: `activity_type_counts(self)`

Signature:

```
def activity_type_counts(self) -> dict
```

Logic: Group activities by their type and determine how many activities belong to each type.

Returns: dictionary

Method 4: total_calories(self)

Signature:

```
def total_calories(self) -> dict
```

Logic: Calculate the total calories burned for every activity by adding all session values.

Returns: dictionary

Method 5: top_activity(self)

Signature:

```
def top_activity(self) -> str
```

Logic: Identify the activity whose session calories add up to the largest total. If two or more activities have the same highest total, the activity appearing first in the dictionary wins.

Returns: string

SAMPLE INPUT AND OUTPUT

```
analyzer = FitnessAnalyzer({
    "Running": {"type": "Cardio", "sessions": [300, 350]},
    "Cycling": {"type": "Cardio", "sessions": [250, 300, 200]},
    "Yoga": {"type": "Flexibility", "sessions": [100, 120]},
    "Swimming": {"type": "Cardio", "sessions": [400, 450]},
    "Weights": {"type": "Strength", "sessions": [200, 250]}
})
```

Average Calories

```
print(analyzer.average_calories())
```

Output:

```
{'Running': 325.0, 'Cycling': 250.0, 'Yoga': 110.0, 'Swimming': 425.0,
'Weights': 225.0}
```

Activity Type Counts

```
print(analyzer.activity_type_counts())
```

Output:

```
{'Cardio': 3, 'Flexibility': 1, 'Strength': 1}
```

Total Calories

```
print(analyzer.total_calories())
```

Output:

```
{'Running': 650, 'Cycling': 750, 'Yoga': 220, 'Swimming': 850, 'Weights': 450}
```

Top Activity

```
print(analyzer.top_activity())
```

Output:

Swimming

Expected Input and Output

Input:

```
analyzer = FitnessAnalyzer({
    "Running": {"type": "Cardio", "sessions": [300, 350]},
    "Cycling": {"type": "Cardio", "sessions": [250, 300, 200]},
```

```

    "Yoga": {"type": "Flexibility", "sessions": [100, 120]},
    "Swimming": {"type": "Cardio", "sessions": [400, 450]},
    "Weights": {"type": "Strength", "sessions": [200, 250]}
})

print("Average Calories:", analyzer.average_calories())
print("Activity Type Counts:", analyzer.activity_type_counts())
print("Total Calories:", analyzer.total_calories())
print("Top Activity:", analyzer.top_activity())
Expected Output:
Average Calories: {'Running': 325.0, 'Cycling': 250.0, 'Yoga': 110.0,
'Swimming': 425.0, 'Weights': 225.0}
Activity Type Counts: {'Cardio': 3, 'Flexibility': 1, 'Strength': 1}
Total Calories: {'Running': 650, 'Cycling': 750, 'Yoga': 220, 'Swimming': 850,
'Weights': 450}
Top Activity: Swimming
Constraints
The number of activities will be between 1 and 1000.
Each activity has at least one session.
Each activity has at most 100 sessions.
Calories are non-negative integers.
Activity names are non-empty and case-sensitive.
Activity types are non-empty strings.
In case of a tie, the activity appearing first wins.

```

QUESTION 3 – MOVIE WATCHLIST ANALYZER

Problem Statement

A streaming service maintains a watchlist containing movies and their viewing information. Each movie belongs to a genre and can have multiple viewing records representing the number of minutes watched during different sessions.

The service wants to analyze the watchlist by calculating the average viewing time, counting movies by genre, calculating total viewing time, and identifying the movie with the highest total viewing time.

Objective

Design a Python class named WatchlistAnalyzer that uses dictionaries and lists to organize movie information and generate analytical results.

Task

Class Structure

Class Name: WatchlistAnalyzer

Public Attributes:

movies

Type: dictionary

A dictionary where each key is a movie name and each value is a dictionary containing:

genre – movie genre.

watch_times – list of integers representing viewing minutes.

The dictionary must preserve the order supplied by the caller.

Methods

Method 1: init(self, movies)

```
def __init__(self, movies)
```

Store the provided movie data in the instance.

Returns: None

Method 2: average_watch_time(self)

```
def average_watch_time(self) -> dict
```

Calculate the average viewing time for every movie.

Returns: dictionary

Method 3: genre_counts(self)

```
def genre_counts(self) -> dict
```

Count how many movies belong to each genre.

Returns: dictionary

Method 4: total_watch_time(self)

```
def total_watch_time(self) -> dict
```

Calculate the total viewing time for every movie.

Returns: dictionary

Method 5: most_watched_movie(self)

```
def most_watched_movie(self) -> str
```

Return the movie having the highest total viewing time.

If there is a tie, the movie appearing first in the original dictionary wins.

Returns: string

SAMPLE INPUT AND OUTPUT

```
analyzer = WatchlistAnalyzer({
    "Inception": {"genre": "Sci-Fi", "watch_times": [120, 30]},
    "Titanic": {"genre": "Drama", "watch_times": [100, 95]},
    "Avatar": {"genre": "Sci-Fi", "watch_times": [80, 90]},
    "Joker": {"genre": "Drama", "watch_times": [70, 80]},
    "Frozen": {"genre": "Animation", "watch_times": [60, 70]}
})
```

Expected Output

Average Watch Time: {'Inception': 75.0, 'Titanic': 97.5, 'Avatar': 85.0, 'Joker': 75.0, 'Frozen': 65.0}

Genre Counts: {'Sci-Fi': 2, 'Drama': 2, 'Animation': 1}

Total Watch Time: {'Inception': 150, 'Titanic': 195, 'Avatar': 170, 'Joker': 150, 'Frozen': 130}

Most Watched Movie: Titanic

Constraints

Number of movies: 1 to 1000.

Each movie contains at least one viewing record.

Viewing time is a non-negative integer.

Movie names are non-empty strings.

Genre names are non-empty strings.

Movie names are case-sensitive.

In case of a tie, the first movie in the dictionary wins.

QUESTION 4 – DELIVERY PERFORMANCE MANAGER

Problem Statement

A delivery company maintains records of packages handled by its delivery agents. Each agent can handle multiple deliveries, and every delivery records the number of packages successfully delivered.

The company wants to analyze the performance of agents and determine total deliveries, average deliveries, agents exceeding a performance target, and the best-performing agent.

In addition to basic calculations, the system must also identify whether an agent has achieved a performance target based on both total deliveries and number of delivery sessions.

Objective

Design a Python class named `DeliveryManager` that uses dictionaries and lists to organize delivery data and perform multiple analytical operations.

Class Structure

Class Name: `DeliveryManager`

Public Attributes:

`agents`

Type: dictionary

Each key is an agent name and its value is a dictionary containing:

`zone` – delivery zone.

`deliveries` – list of integers representing packages delivered during each session.

Methods

Method 1: `init(self, agents)`

`def __init__(self, agents)`

Store the supplied agent information in the instance.

Method 2: `total_deliveries(self)`

`def total_deliveries(self) -> dict`

Calculate the total number of packages delivered by every agent.

Method 3: `average_deliveries(self)`

`def average_deliveries(self) -> dict`

Calculate the average number of packages delivered per session for every agent.

The result must be a float.

Method 4: `zone_counts(self)`

`def zone_counts(self) -> dict`

Count the number of delivery agents working in each zone.

Method 5: `agents_above(self, target)`

`def agents_above(self, target) -> list`

Return all agent names whose total deliveries are strictly greater than the given target.

The result must be sorted alphabetically.

```
Method 6: performance_status(self, target)
def performance_status(self, target) -> dict
```

An agent is considered "Achieved" if:

total deliveries are greater than or equal to the target, and
the agent has completed at least 3 delivery sessions.

Otherwise, the status should be "Pending".

Return a dictionary mapping each agent name to its status.

SAMPLE INPUT AND OUTPUT

```
manager = DeliveryManager({
    "Aman": {"zone": "North", "deliveries": [40, 50, 60]},
    "Riya": {"zone": "South", "deliveries": [30, 35]},
    "Karan": {"zone": "North", "deliveries": [70, 50, 40]},
    "Neha": {"zone": "West", "deliveries": [80, 20, 30]}
})
Total Deliveries
{'Aman': 150, 'Riya': 65, 'Karan': 160, 'Neha': 130}
Average Deliveries
{'Aman': 50.0, 'Riya': 32.5, 'Karan': 53.333333333333336, 'Neha':
43.333333333333336}
Zone Counts
{'North': 2, 'South': 1, 'West': 1}
Agents Above 100
['Aman', 'Karan', 'Neha']
Performance Status for 140
{'Aman': 'Achieved', 'Riya': 'Pending', 'Karan': 'Achieved', 'Neha': 'Pending'}
```

Neha has 130 deliveries, so she does not reach 140.

Expected Input and Output

```
manager = DeliveryManager({
    "Aman": {"zone": "North", "deliveries": [40, 50, 60]},
    "Riya": {"zone": "South", "deliveries": [30, 35]},
    "Karan": {"zone": "North", "deliveries": [70, 50, 40]},
    "Neha": {"zone": "West", "deliveries": [80, 20, 30]}
})
```

```
print("Total:", manager.total_deliveries())
print("Average:", manager.average_deliveries())
print("Zones:", manager.zone_counts())
print("Above Target:", manager.agents_above(100))
print("Status:", manager.performance_status(140))
```

Expected Output

Total: {'Aman': 150, 'Riya': 65, 'Karan': 160, 'Neha': 130}

Average: {'Aman': 50.0, 'Riya': 32.5, 'Karan': 53.333333333333336, 'Neha':
43.333333333333336}

Zones: {'North': 2, 'South': 1, 'West': 1}

Above Target: ['Aman', 'Karan', 'Neha']

Status: {'Aman': 'Achieved', 'Riya': 'Pending', 'Karan': 'Achieved', 'Neha':
'Pending'}

Constraints

Number of agents: 1 to 1000.

Each agent has at least one delivery session.

Each agent can have at most 100 sessions.

Delivery quantities are non-negative integers.

Agent and zone names are non-empty strings.

Agent names are case-sensitive.
agents_above uses a strict > comparison.
An agent needs both conditions to receive "Achieved" status.
QUESTION 5 – CAFE SALES TRACKER
Problem Statement

A café records the number of units sold for different menu items. Each menu item belongs to a category such as Beverage, Snack, Dessert, or Main Course. Sales are recorded during multiple billing periods.

The café requires a Python system that can analyze sales, determine category-wise totals, identify the best-selling item, find items that have crossed a sales target, and calculate the percentage contribution of each item to the overall sales.

Objective

Design a Python class named CafeSalesTracker that uses dictionaries and lists to store sales information and generate useful business insights.

Class Structure

Class Name: CafeSalesTracker

Public Attributes:

sales

Type: dictionary

Each key is a menu item name and its value is a dictionary containing:

category – category of the item.

units – list of integers representing units sold during different billing periods.

Methods

Method 1: init(self, sales)

Store the supplied sales dictionary in the instance.

Method 2: total_units(self)

def total_units(self) -> dict

Calculate the total units sold for every menu item.

Method 3: category_totals(self)

def category_totals(self) -> dict

Calculate the total number of units sold for every category.

For example, if two Beverage items sell 100 and 150 units, the Beverage total should be 250.

Method 4: best_selling(self)

def best_selling(self) -> str

Return the item having the highest total sales.

If two items have the same total, return the item appearing first in the original dictionary.

Method 5: items_above(self, target)

def items_above(self, target) -> list

Return item names whose total sales are strictly greater than the target.

Sort the result alphabetically.

```
Method 6: sales_percentage(self)
def sales_percentage(self) -> dict
```

Calculate each item's percentage contribution to the total units sold.

Formula:

$$(\text{item total} / \text{overall total}) * 100$$

Return each percentage as a float.

SAMPLE INPUT AND OUTPUT

```
tracker = CafeSalesTracker({
    "Coffee": {"category": "Beverage", "units": [50, 70]},
    "Tea": {"category": "Beverage", "units": [30, 40]},
    "Burger": {"category": "Main", "units": [80, 90]},
    "Cake": {"category": "Dessert", "units": [40, 50]}
})
```

Total Units

```
{'Coffee': 120, 'Tea': 70, 'Burger': 170, 'Cake': 90}
```

Category Totals

```
{'Beverage': 190, 'Main': 170, 'Dessert': 90}
```

Best Selling

Burger

Items Above 100

```
['Burger', 'Coffee']
```

Sales Percentage

Total units = 450

```
{
    'Coffee': 26.666666666666668,
    'Tea': 15.555555555555555,
    'Burger': 37.77777777777778,
    'Cake': 20.0
}
```

Expected Input and Output

```
tracker = CafeSalesTracker({
    "Coffee": {"category": "Beverage", "units": [50, 70]},
    "Tea": {"category": "Beverage", "units": [30, 40]},
    "Burger": {"category": "Main", "units": [80, 90]},
    "Cake": {"category": "Dessert", "units": [40, 50]}
})
```

```
print("Total Units:", tracker.total_units())
```

```
print("Category Totals:", tracker.category_totals())
```

```
print("Best Selling:", tracker.best_selling())
```

```
print("Above Target:", tracker.items_above(100))
```

```
print("Sales Percentage:", tracker.sales_percentage())
```

Expected Output

```
Total Units: {'Coffee': 120, 'Tea': 70, 'Burger': 170, 'Cake': 90}
```

```
Category Totals: {'Beverage': 190, 'Main': 170, 'Dessert': 90}
```

```
Best Selling: Burger
```

```
Above Target: ['Burger', 'Coffee']
```

```
Sales Percentage: {'Coffee': 26.666666666666668, 'Tea': 15.555555555555555,
'Burger': 37.77777777777778, 'Cake': 20.0}
```

Constraints

Number of menu items: 1 to 1000.
Each item has at least one sales record.
Sales quantities are non-negative integers.
Category names are non-empty strings.
Item names are case-sensitive.
At least one total unit will be greater than zero.
Items above the target must be sorted alphabetically.
Percentage values must be calculated dynamically.

QUESTION 6 – PET CARE RECORD ANALYZER

Problem Statement

A pet-care center maintains records of appointments for different pets. Each pet belongs to an animal type, and the center records the number of services received during each visit.

The center needs a system that can calculate average services per visit, count pets by animal type, calculate total services, identify the pet with the highest number of services, and determine whether a pet requires additional care.

Objective

Design a Python class named `PetCareAnalyzer` that models the pet-care analysis system using dictionaries and lists.

Class Structure

Class Name: `PetCareAnalyzer`

Public Attributes:

`pets`

Type: dictionary

Each key represents a pet name and each value contains:

`animal` – animal type.

`visits` – list of integers representing services received during each visit.

Methods

Method 1: `init(self, pets)`

Store the supplied pet data in the instance.

Method 2: `average_services(self)`

`def average_services(self) -> dict`

Calculate the average services received per visit for every pet.

Method 3: `animal_counts(self)`

`def animal_counts(self) -> dict`

Count how many pets belong to each animal type.

Method 4: `total_services(self)`

`def total_services(self) -> dict`

Calculate the total services received by each pet.

Method 5: `busiest_pet(self)`

`def busiest_pet(self) -> str`

Return the pet with the highest total number of services.

If two pets have the same total, the first pet in the dictionary wins.

```
Method 6: care_required(self, limit)
def care_required(self, limit) -> list
```

Return pets whose total services are strictly greater than the given limit.

The result must be sorted alphabetically.

SAMPLE INPUT AND OUTPUT

```
analyzer = PetCareAnalyzer({
    "Bruno": {"animal": "Dog", "visits": [3, 4, 2]},
    "Milo": {"animal": "Cat", "visits": [2, 3]},
    "Luna": {"animal": "Dog", "visits": [5, 4]},
    "Coco": {"animal": "Bird", "visits": [1, 2]}
})
Average Services
{'Bruno': 3.0, 'Milo': 2.5, 'Luna': 4.5, 'Coco': 1.5}
Animal Counts
{'Dog': 2, 'Cat': 1, 'Bird': 1}
Total Services
{'Bruno': 9, 'Milo': 5, 'Luna': 9, 'Coco': 3}
Busiest Pet
Bruno
```

Bruno and Luna both have 9 services, but Bruno appears first.

Pets Above 5

```
['Bruno', 'Luna']
```

Expected Input and Output

```
analyzer = PetCareAnalyzer({
    "Bruno": {"animal": "Dog", "visits": [3, 4, 2]},
    "Milo": {"animal": "Cat", "visits": [2, 3]},
    "Luna": {"animal": "Dog", "visits": [5, 4]},
    "Coco": {"animal": "Bird", "visits": [1, 2]}
})
```

```
print("Average Services:", analyzer.average_services())
print("Animal Counts:", analyzer.animal_counts())
print("Total Services:", analyzer.total_services())
print("Busiest Pet:", analyzer.busiest_pet())
print("Care Required:", analyzer.care_required(5))
```

Expected Output

```
Average Services: {'Bruno': 3.0, 'Milo': 2.5, 'Luna': 4.5, 'Coco': 1.5}
```

```
Animal Counts: {'Dog': 2, 'Cat': 1, 'Bird': 1}
```

```
Total Services: {'Bruno': 9, 'Milo': 5, 'Luna': 9, 'Coco': 3}
```

```
Busiest Pet: Bruno
```

```
Care Required: ['Bruno', 'Luna']
```

Constraints

Number of pets: 1 to 1000.

Each pet has at least one visit.

Each pet has at most 100 visits.

Service counts are non-negative integers.

Pet names and animal types are non-empty strings.

Pet names are case-sensitive.

care_required uses a strict comparison.

Ties are resolved using original dictionary order.

QUESTION 7 – PROJECT WORKLOAD ANALYZER

Problem Statement

A software company maintains a record of tasks completed by employees for different projects. Each employee can work on multiple tasks, and every task records the number of hours spent.

The company needs an analytical system that can determine the total hours worked by every employee, calculate average hours per task, count employees by team, identify the employee who worked the most hours, and find employees whose workload exceeds a specified limit.

Objective

Design and implement a Python class named `WorkloadAnalyzer` that uses dictionaries and lists to organize employee workload information.

The exercise evaluates constructor behavior, instance attributes, nested dictionaries, list processing, aggregation, filtering, sorting, and tie handling.

Class Structure

Class Name: `WorkloadAnalyzer`

Public Attributes:

`employees`

Type: dictionary

Each key is an employee name and each value is a dictionary containing:

`team` – team name.

`hours` – list of integers representing hours spent on individual tasks.

Methods

Method 1: `init(self, employees)`

`def __init__(self, employees)`

Store the supplied employee workload information in the instance.

Returns: `None`

Method 2: `total_hours(self)`

`def total_hours(self) -> dict`

Calculate the total number of hours worked by each employee.

Returns: dictionary

Method 3: `average_hours(self)`

`def average_hours(self) -> dict`

Calculate the average number of hours spent per task for every employee.

The result must be a float.

Returns: dictionary

Method 4: `team_counts(self)`

`def team_counts(self) -> dict`

Group employees according to their team and determine the number of employees in every team.

Returns: dictionary

```
Method 5: highest_workload(self)
def highest_workload(self) -> str
```

Find the employee with the highest total number of working hours.

If multiple employees have the same highest total, the employee appearing first in the supplied dictionary wins.

Returns: string

```
Method 6: workload_status(self, limit)
def workload_status(self, limit) -> dict
```

For each employee:

Return "High" if total hours are strictly greater than the given limit.
Return "Normal" otherwise.

Return a dictionary containing the employee name and status.

Returns: dictionary

```
Method 7: overloaded_employees(self, limit)
def overloaded_employees(self, limit) -> list
```

Return all employee names whose total hours are strictly greater than the given limit.

The names must be sorted alphabetically.

Returns: list

SAMPLE INPUT AND OUTPUT

```
analyzer = WorkloadAnalyzer({
    "Arjun": {"team": "Backend", "hours": [6, 7, 5]},
    "Meera": {"team": "Frontend", "hours": [5, 6]},
    "Rahul": {"team": "Backend", "hours": [8, 7, 6]},
    "Priya": {"team": "QA", "hours": [4, 5, 4]},
    "Nisha": {"team": "Frontend", "hours": [7, 6, 5]}
})
Total Hours
{
    'Arjun': 18,
    'Meera': 11,
    'Rahul': 21,
    'Priya': 13,
    'Nisha': 18
}
Average Hours
{
    'Arjun': 6.0,
    'Meera': 5.5,
    'Rahul': 7.0,
    'Priya': 4.333333333333333,
    'Nisha': 6.0
}
Team Counts
{
    'Backend': 2,
    'Frontend': 2,
    'QA': 1
}
Highest Workload
Rahul
```

Workload Status for 17

```
{
  'Arjun': 'High',
  'Meera': 'Normal',
  'Rahul': 'High',
  'Priya': 'Normal',
  'Nisha': 'High'
}
```

Overloaded Employees

['Arjun', 'Nisha', 'Rahul']

Expected Input and Output

Input:

```
analyzer = WorkloadAnalyzer({
  "Arjun": {"team": "Backend", "hours": [6, 7, 5]},
  "Meera": {"team": "Frontend", "hours": [5, 6]},
  "Rahul": {"team": "Backend", "hours": [8, 7, 6]},
  "Priya": {"team": "QA", "hours": [4, 5, 4]},
  "Nisha": {"team": "Frontend", "hours": [7, 6, 5]}
})
```

```
print("Total Hours:", analyzer.total_hours())
```

```
print("Average Hours:", analyzer.average_hours())
```

```
print("Team Counts:", analyzer.team_counts())
```

```
print("Highest Workload:", analyzer.highest_workload())
```

```
print("Workload Status:", analyzer.workload_status(17))
```

```
print("Overloaded Employees:", analyzer.overloaded_employees(17))
```

Expected Output:

Total Hours: {'Arjun': 18, 'Meera': 11, 'Rahul': 21, 'Priya': 13, 'Nisha': 18}

Average Hours: {'Arjun': 6.0, 'Meera': 5.5, 'Rahul': 7.0, 'Priya': 4.333333333333333, 'Nisha': 6.0}

Team Counts: {'Backend': 2, 'Frontend': 2, 'QA': 1}

Highest Workload: Rahul

Workload Status: {'Arjun': 'High', 'Meera': 'Normal', 'Rahul': 'High', 'Priya': 'Normal', 'Nisha': 'High'}

Overloaded Employees: ['Arjun', 'Nisha', 'Rahul']

Constraints

Number of employees will be between 1 and 1000.

Every employee has at least one task.

Every employee has at most 100 task-hour records.

Hours are non-negative integers.

Employee names are non-empty strings.

Team names are non-empty strings.

Employee names are case-sensitive.

workload_status uses a strict > comparison.

overloaded_employees returns names in alphabetical order.

In case of a tie for highest workload, the first employee in the dictionary wins.

Q1. LIBRARY BOOK TRACKER

Problem Statement

A community library maintains a collection of books that members can borrow. The library currently records book information manually, which becomes difficult to manage as the number of books increases. To streamline this process, the library

requires a simple system that can manage book names, record borrow operations, retrieve the current borrow count for any given book, and identify the most borrowed book.

Objective

Design and implement a Python class named `LibraryBookTracker` that models a basic library borrowing system. The class should use a list and dictionary to store and manage book data. Students are expected to demonstrate their understanding of class creation, instance attributes, dictionary operations, and method implementation.

Task

CLASS STRUCTURE

Class Name: `LibraryBookTracker`

Public Attributes

`books`

Type: list

A list of strings where each element represents a book name. Books are stored in the order they are added.

`borrow_counts`

Type: dictionary

A dictionary where each key is a book name and its corresponding value is the number of times the book has been borrowed.

Both attributes must be initialized inside the constructor.

METHODS

Method 1: `__init__(self)`

Signature:

```
def __init__(self)
```

Parameters: None beyond self.

Logic: Initialize `books` as an empty list and `borrow_counts` as an empty dictionary.

Returns: None

Method 2: `add_book(self, book_name)`

Signature:

```
def add_book(self, book_name)
```

Parameters: `book_name` (string) – The name of the book to be added.

Logic: If the book does not exist, append it to `books` and initialize its borrow count to zero. If it already exists, do nothing.

Returns: None

Method 3: `record_borrow(self, book_name)`

Signature:

```
def record_borrow(self, book_name)
```

Logic: If the book exists, increment its borrow count by one. Otherwise, ignore the operation.

Returns: None

Method 4: get_borrow_count(self, book_name)

Signature:

```
def get_borrow_count(self, book_name)
```

Logic: Return the current borrow count. Return 0 if the book does not exist.

Returns: integer

Method 5: most_borrowed_book(self)

Signature:

```
def most_borrowed_book(self)
```

Logic: Return the book with the highest borrow count. If there is a tie, return the book that was added earliest. If there are no books, return None.

Returns: string or None

Sample Input and Output

Adding a new book:

Input: book_name = "Python Basics"

Output:

books becomes ["Python Basics"]

borrow_counts becomes {"Python Basics": 0}

Recording a borrow:

Input: book_name = "Python Basics"

Output:

borrow_counts becomes {"Python Basics": 1}

Getting borrow count:

Input: book_name = "Python Basics"

Output:

1

EXPECTED OUTPUT

```
library = LibraryBookTracker()
```

```
library.add_book("Python Basics")
```

```
library.add_book("Data Structures")
```

```
library.add_book("Clean Code")
```

```
library.record_borrow("Python Basics")
```

```
library.record_borrow("Python Basics")
```

```
library.record_borrow("Data Structures")
```

```
print(library.get_borrow_count("Python Basics"))
```

```
print(library.get_borrow_count("Clean Code"))
```

```
print(library.get_borrow_count("Unknown"))
```

```
print("Most Borrowed Book:", library.most_borrowed_book())
```

Expected Output

2

0

0

Most Borrowed Book: Python Basics

Constraints

The number of books will not exceed 1,000.

Each book name is a non-empty string.

Book names are case-sensitive.

Duplicate additions are silently ignored.

Borrow operations for unknown books are silently ignored.

In case of a tie, the earliest-added book wins.

Q2. FITNESS ACTIVITY ANALYZER

Problem Statement

A fitness center records the activities performed by its members. Each activity has a category and multiple calorie-burning sessions. Manual analysis becomes difficult as the number of activities increases. The center requires a system that can calculate average calories, count activities by category, calculate total calories, and determine the activity with the highest total calorie expenditure.

Objective

Design and implement a Python class named `FitnessActivityAnalyzer` that models a basic fitness analysis system. The class should use dictionaries and lists to organize activity data and compute analytical insights.

CLASS STRUCTURE

Class Name: `FitnessActivityAnalyzer`

Public Attribute

`data`

Type: dictionary

Each key is an activity name and each value is a dictionary containing:

`category` – string

`calories` – list of integers

The activities must remain in the order supplied by the caller.

METHODS

Method 1: `__init__(self, data)`

Store the supplied dictionary in the instance attribute `data`.

Method 2: `average_calories(self)`

Return a dictionary containing the average calorie value for every activity.

The result must contain floating-point values.

Method 3: `category_counts(self)`

Group activities by category and return the number of activities in each category.

Method 4: `total_calories(self)`

Calculate the sum of all calorie values for every activity.

Method 5: `highest_activity(self)`

Return the activity having the highest total calorie value.

If two activities have the same total, the activity appearing first in the original dictionary wins.

Sample Input

```
analyzer = FitnessActivityAnalyzer({
    "Running": {"category": "Cardio", "calories": [300, 350]},
    "Cycling": {"category": "Cardio", "calories": [200, 250, 300]},
    "Yoga": {"category": "Flexibility", "calories": [100, 120]},
    "Swimming": {"category": "Cardio", "calories": [400, 450]}
})
```

EXPECTED OUTPUT

```
Average Calories: {'Running': 325.0, 'Cycling': 250.0, 'Yoga': 110.0,
'Swimming': 425.0}
```

```
Category Counts: {'Cardio': 3, 'Flexibility': 1}
```

```
Total Calories: {'Running': 650, 'Cycling': 750, 'Yoga': 220, 'Swimming': 850}
```

Highest Activity: Swimming

Constraints

Number of activities: 1-1,000.

Each activity contains at least one calorie entry.

Calorie values are non-negative integers.

Activity and category names are non-empty strings.

Ties are resolved using original dictionary order.

Q3. MOVIE NIGHT ORGANIZER

This is a same-logic original-style question, with the original scenario retained.

Problem Statement

A community center hosts weekly movie nights where members suggest movies and vote for their favorites. The center currently tracks suggestions and votes manually. The system must manage movie suggestions, record votes, retrieve vote counts, and announce the winning movie.

Objective

Design and implement:

```
class MovieNightOrganizer
```

using a list and dictionary.

Attributes

movies → list

votes → dictionary

movies

A list of movie names stored in the order they were added.

votes

A dictionary mapping movie names to vote counts.

METHODS

```
__init__(self)
```

Initialize:

```
movies = []
```

```
votes = {}
add_movie(self, movie_name)
```

Add the movie if it does not already exist.

Initialize its vote count to 0.

Duplicate movies must be ignored.

```
register_vote(self, movie_name)
```

If the movie exists, increase its vote count by 1.

Unknown movies are ignored.

```
get_vote_count(self, movie_name)
```

Return the movie's vote count.

Return 0 for an unknown movie.

```
most_voted_movie(self)
```

Return the movie with the highest vote count.

If multiple movies have the same highest count, return the movie added earliest.

Return None when no movie exists.

EXPECTED OUTPUT

```
organizer = MovieNightOrganizer()
```

```
organizer.add_movie("Inception")
organizer.add_movie("The Matrix")
organizer.add_movie("Interstellar")
```

```
organizer.register_vote("Inception")
organizer.register_vote("Inception")
organizer.register_vote("The Matrix")
```

```
print(organizer.get_vote_count("Inception"))
print(organizer.get_vote_count("The Matrix"))
print(organizer.get_vote_count("Interstellar"))
print(organizer.get_vote_count("Unknown"))
```

```
print("Most Voted Movie:", organizer.most_voted_movie())
```

2

1

0

0

Most Voted Movie: Inception

Constraints

Maximum 1,000 movies.

Movie names are non-empty strings.

Names are case-sensitive.

Duplicate movies are ignored.

Votes for unknown movies are ignored.

Earliest-added movie wins a tie.

Q4. RESTAURANT SALES ANALYZER ★ ENHANCED

This question adds discount calculation and category-level totals.

Problem Statement

A restaurant records the dishes sold during the day. Each dish belongs to a category and has several recorded sale quantities. The restaurant wants to analyze its sales, determine average quantities, count dishes by category, calculate total quantities, and identify the best-selling dish. Additionally, management wants to calculate the final sales quantity after applying a category-specific adjustment percentage.

Objective

Design:

```
class RestaurantSalesAnalyzer
Public Attribute
menu_data
```

Dictionary:

```
{
    "Pizza": {
        "category": "Main Course",
        "sales": [20, 30, 25]
    }
}
```

METHODS

1. `__init__(self, menu_data)`

Store the supplied data.

2. `average_sales(self)`

Return the average sale quantity for every dish.

3. `category_counts(self)`

Return the number of dishes belonging to every category.

4. `total_sales(self)`

Return the total quantity sold for every dish.

5. `best_selling_dish(self)`

Return the dish having the highest total sales.

Earliest dish wins a tie.

6. `adjusted_sales(self, percentage)`

Apply the percentage reduction to every dish's total sales.

For example:

100 total sales

10% reduction

final = 90

Return a dictionary containing the adjusted values.

EXPECTED OUTPUT

```
restaurant = RestaurantSalesAnalyzer({
    "Pizza": {"category": "Main", "sales": [20, 30]},
    "Burger": {"category": "Main", "sales": [15, 25]},
    "Soup": {"category": "Starter", "sales": [10, 12]}
})
```

```

print(restaurant.average_sales())
print(restaurant.category_counts())
print(restaurant.total_sales())
print(restaurant.best_selling_dish())
print(restaurant.adjusted_sales(10))
{'Pizza': 25.0, 'Burger': 20.0, 'Soup': 11.0}
{'Main': 2, 'Starter': 1}
{'Pizza': 50, 'Burger': 40, 'Soup': 22}
Pizza
{'Pizza': 45.0, 'Burger': 36.0, 'Soup': 19.8}
Constraints
Maximum 1,000 dishes.
Every dish has at least one sale entry.
Sales quantities are non-negative integers.
Percentage is between 0 and 100.
Ties use original dictionary order.
Q5. INGREDIENT STOCK CHECKER

```

This is the same-logic original-style question.

Problem Statement

A local bakery needs to maintain ingredient stock levels. The bakery requires a system that can manage ingredient names, update stock quantities, retrieve current stock levels, and list ingredients below a reorder threshold.

Objective

Design:

```
class IngredientStockChecker
```

using a list and dictionary.

Attributes

ingredients → list

stock → dictionary

METHODS

```
__init__(self)
```

Initialize:

```
ingredients = []
```

```
stock = {}
```

```
add_ingredient(self, ingredient_name)
```

Add a new ingredient and initialize its stock to 0.

Duplicate ingredients are ignored.

```
update_stock(self, ingredient_name, quantity)
```

If the ingredient exists, add the quantity to its current stock.

Unknown ingredients are ignored.

```
get_stock(self, ingredient_name)
```

Return the current stock.

Return 0 for an unknown ingredient.

```
low_stock_ingredients(self, threshold)
```

Return ingredients whose stock is strictly less than the threshold.

Maintain insertion order.

EXPECTED OUTPUT

```
checker = IngredientStockChecker()
```

```
checker.add_ingredient("Flour")
checker.add_ingredient("Sugar")
checker.add_ingredient("Eggs")
checker.add_ingredient("Butter")
```

```
checker.update_stock("Flour", 50)
checker.update_stock("Flour", 10)
checker.update_stock("Sugar", 25)
```

```
print(checker.get_stock("Flour"))
print(checker.get_stock("Sugar"))
print(checker.get_stock("Eggs"))
print(checker.get_stock("Milk"))
```

```
print("Low Stock Ingredients:", checker.low_stock_ingredients(30))
```

```
60
```

```
25
```

```
0
```

```
0
```

```
Low Stock Ingredients: ['Sugar', 'Eggs', 'Butter']
```

Constraints

Maximum 1,000 ingredients.

Ingredient names are non-empty strings.

Names are case-sensitive.

Duplicate additions are ignored.

Unknown stock updates are ignored.

The threshold comparison is strictly <.

Q6. HOTEL ROOM MANAGER ★ ENHANCED

Problem Statement

A hotel manages rooms across different room types. Each room has a booking history containing the number of nights booked. The hotel requires a system that can calculate total nights, average nights, count rooms by type, identify the most occupied room, and determine rooms that have crossed a specified occupancy limit.

Objective

Design:

```
class HotelRoomManager
Public Attribute
rooms
```

Example:

```
{
    "Room101": {
        "type": "Deluxe",
        "bookings": [2, 3, 1]
    }
}
```

METHODS

```
__init__(self, rooms)
```

Store the supplied room data.

average_nights(self)

Return the average booking nights for every room.

room_type_counts(self)

Return the number of rooms of each type.

total_nights(self)

Return total booked nights for every room.

most_occupied_room(self)

Return the room with the highest total nights.

Earliest room wins a tie.

rooms_above(self, limit)

Return room names whose total booked nights are strictly greater than limit.

Return them in alphabetical order.

EXPECTED OUTPUT

```
hotel = HotelRoomManager({
    "Room101": {"type": "Deluxe", "bookings": [2, 3]},
    "Room102": {"type": "Standard", "bookings": [1, 2]},
    "Room103": {"type": "Deluxe", "bookings": [4, 3]},
    "Room104": {"type": "Standard", "bookings": [2, 1]}
})

print(hotel.average_nights())
print(hotel.room_type_counts())
print(hotel.total_nights())
print(hotel.most_occupied_room())
print(hotel.rooms_above(4))
{'Room101': 2.5, 'Room102': 1.5, 'Room103': 3.5, 'Room104': 1.5}
{'Deluxe': 2, 'Standard': 2}
{'Room101': 5, 'Room102': 3, 'Room103': 7, 'Room104': 3}
Room103
['Room101', 'Room103']
```

Constraints

Maximum 1,000 rooms.

Each room contains at least one booking.

Booking values are non-negative integers.

Room names and room types are non-empty strings.

rooms_above() uses strict > comparison.

Results from rooms_above() are alphabetically sorted.

Q7. BIRDWATCHING SPECIES TRACKER

This is the same-logic original-style question.

Problem Statement

A birdwatching club records different bird species observed during outings. The system must manage species names, record sightings, retrieve sighting counts, and calculate the total number of recorded sightings.

Objective

Design:

```
class BirdwatchingTracker
```

```
using:
```

```
species → list
```

```
sightings → dictionary
```

```
METHODS
```

```
__init__(self)
```

Initialize both attributes as empty:

```
species = []
```

```
sightings = {}
```

```
add_species(self, species_name)
```

Add a new species and initialize its sighting count to 0.

```
record_sighting(self, species_name, count)
```

Add count to the existing sighting count.

Ignore unknown species.

```
get_sighting_count(self, species_name)
```

Return the current count.

Return 0 if unknown.

```
total_sightings(self)
```

Return the sum of all sighting counts.

EXPECTED OUTPUT

```
tracker = BirdwatchingTracker()
```

```
tracker.add_species("Sparrow")
```

```
tracker.add_species("Robin")
```

```
tracker.add_species("Eagle")
```

```
tracker.add_species("Hawk")
```

```
tracker.record_sighting("Sparrow", 5)
```

```
tracker.record_sighting("Sparrow", 3)
```

```
tracker.record_sighting("Robin", 2)
```

```
print(tracker.get_sighting_count("Sparrow"))
```

```
print(tracker.get_sighting_count("Robin"))
```

```
print(tracker.get_sighting_count("Eagle"))
```

```
print(tracker.get_sighting_count("Owl"))
```

```
print("Total Sightings:", tracker.total_sightings())
```

```
8
```

```
2
```

```
0
```

```
0
```

```
Total Sightings: 10
```

Constraints

Maximum 1,000 species.

Species names are non-empty strings.

Species names are case-sensitive.

Duplicate species are ignored.

Unknown species updates are ignored.

Empty tracker total is 0.

Q8. DELIVERY PERFORMANCE ANALYZER ★ ENHANCED

Problem Statement

A delivery company records delivery times for different delivery agents. Each agent belongs to a region and has several delivery-time records. The company needs a system that can calculate average delivery time, total delivery time, group agents by region, find the fastest agent, and identify agents whose average delivery time exceeds a specified limit.

Class

```
class DeliveryPerformanceAnalyzer
```

Attribute

```
agents
```

Example:

```
{
    "AgentA": {
        "region": "North",
        "times": [30, 25, 35]
    }
}
METHODS
__init__(self, agents)
```

Store the provided data.

```
average_time(self)
```

Return the average delivery time for each agent.

```
region_counts(self)
```

Return the number of agents in each region.

```
total_time(self)
```

Return total delivery time for each agent.

```
fastest_agent(self)
```

Return the agent with the lowest average delivery time.

If tied, the earliest agent wins.

```
agents_above(self, limit)
```

Return agents whose average delivery time is strictly greater than limit, sorted alphabetically.

EXPECTED OUTPUT

```
analyzer = DeliveryPerformanceAnalyzer({
    "Aman": {"region": "North", "times": [20, 30]},
    "Ravi": {"region": "South", "times": [40, 50]},
    "Neha": {"region": "North", "times": [25, 25]},
    "Karan": {"region": "West", "times": [60, 40]}
})

print(analyzer.average_time())
print(analyzer.region_counts())
print(analyzer.total_time())
print(analyzer.fastest_agent())
print(analyzer.agents_above(35))
{'Aman': 25.0, 'Ravi': 45.0, 'Neha': 25.0, 'Karan': 50.0}
{'North': 2, 'South': 1, 'West': 1}
```

```
{'Aman': 50, 'Ravi': 90, 'Neha': 50, 'Karan': 100}
```

Aman

```
['Karan', 'Ravi']
```

Constraints

Maximum 1,000 agents.

At least one delivery record per agent.

Delivery times are non-negative integers.

Lower average means better performance.

Ties use insertion order.

agents_above() uses strict >.

Q9. EVENT REGISTRATION MANAGER ★ ENHANCED

Problem Statement

A training organization manages registrations for several events. Each event has a maximum capacity and a list of registered participants. The organization wants to track registrations, prevent duplicate participants, determine remaining seats, identify the most popular event, and find events that are already full.

Class

```
class EventRegistrationManager
```

Attribute

```
events
```

Example:

```
{
    "Python": {
        "capacity": 50,
        "participants": ["Amit", "Riya"]
    }
}
```

METHODS

```
__init__(self, events)
```

Store the supplied event dictionary.

```
register(self, event_name, participant)
```

Add a participant only if:

event exists

participant is not already registered

capacity has not been reached

```
remaining_seats(self)
```

Return remaining seats for every event.

```
participant_count(self)
```

Return participant count for every event.

```
most_popular_event(self)
```

Return the event with the highest participant count.

Earliest event wins ties.

```
full_events(self)
```

Return events where:

```
participants == capacity
```

sorted alphabetically.

EXPECTED OUTPUT

```
manager = EventRegistrationManager({
    "Python": {
        "capacity": 3,
        "participants": ["Amit"]
    },
    "Java": {
        "capacity": 2,
        "participants": ["Riya", "Karan"]
    },
    "SQL": {
        "capacity": 4,
        "participants": []
    }
})
```

```
manager.register("Python", "Neha")
manager.register("Python", "Rahul")
manager.register("Python", "Amit")
```

```
print(manager.participant_count())
print(manager.remaining_seats())
print(manager.most_popular_event())
print(manager.full_events())
{'Python': 3, 'Java': 2, 'SQL': 0}
{'Python': 0, 'Java': 0, 'SQL': 4}
Python
['Java', 'Python']
Constraints
Maximum 1,000 events.
Capacity is a positive integer.
Participant names are non-empty strings.
Duplicate participants must not be added.
Registration beyond capacity is ignored.
Unknown events are ignored.
```

Q10. MUSIC PLAYLIST MANAGER

This is another original-style question based primarily on the Movie Night logic.

Problem Statement

A music application allows users to maintain a playlist and record how many times each song has been played. The application requires a simple system that can add songs, record plays, retrieve play counts, and identify the most played song.

Class

```
class MusicPlaylistManager
```

Attributes

```
songs → list
```

```
plays → dictionary
```

METHODS

```
__init__(self)
```

Initialize:

```
songs = []
```

```
plays = {}
```

```
add_song(self, song_name)
```

Add the song if it does not already exist.

Set its play count to 0.

`record_play(self, song_name)`

Increase play count by 1 if the song exists.

`get_play_count(self, song_name)`

Return play count, or 0 if the song does not exist.

`most_played_song(self)`

Return the song with the highest play count.

If tied, return the song added earliest.

Return None if there are no songs.

EXPECTED OUTPUT

```
playlist = MusicPlaylistManager()
```

```
playlist.add_song("Believer")
```

```
playlist.add_song("Perfect")
```

```
playlist.add_song("Faded")
```

```
playlist.record_play("Believer")
```

```
playlist.record_play("Believer")
```

```
playlist.record_play("Perfect")
```

```
print(playlist.get_play_count("Believer"))
```

```
print(playlist.get_play_count("Faded"))
```

```
print(playlist.get_play_count("Unknown"))
```

```
print("Most Played Song:", playlist.most_played_song())
```

```
2
```

```
0
```

```
0
```

```
Most Played Song: Believer
```

Constraints

Maximum 1,000 songs.

Song names are non-empty strings.

Names are case-sensitive.

Duplicate songs are ignored.

Unknown songs cannot receive plays.

Earliest-added song wins ties.

Q11. STUDENT SCORE ANALYZER ★ ENHANCED

Problem Statement

A training institute stores the marks obtained by students in several assessments. Each student belongs to a course and has multiple scores. The institute wants to calculate averages, determine pass/fail status, identify the top-performing student, count students by course, and list students whose average is above a specified threshold.

Class

```
class StudentScoreAnalyzer
```

Attribute

```
students
```

Example:

```
{
    "Amit": {
        "course": "Python",
```

```

        "scores": [80, 75, 90]
    }
}
METHODS
__init__(self, students)

```

Store the student data.

```
average_scores(self)
```

Return average score for every student.

```
course_counts(self)
```

Return the number of students in each course.

```
result_status(self, passing_score)
```

Return:

```

{
    "Amit": "Pass",
    "Riya": "Fail"
}

```

A student passes when their average is greater than or equal to passing_score.

```
top_student(self)
```

Return the student with the highest average.

Earliest student wins a tie.

```
above_average(self, threshold)
```

Return students whose average score is strictly greater than the threshold, sorted alphabetically.

EXPECTED OUTPUT

```

analyzer = StudentScoreAnalyzer({
    "Amit": {"course": "Python", "scores": [80, 90, 70]},
    "Riya": {"course": "Java", "scores": [60, 65, 70]},
    "Karan": {"course": "Python", "scores": [90, 95, 85]}
})

```

```

print(analyzer.average_scores())
print(analyzer.course_counts())
print(analyzer.result_status(70))
print(analyzer.top_student())
print(analyzer.above_average(75))
{'Amit': 80.0, 'Riya': 65.0, 'Karan': 90.0}
{'Python': 2, 'Java': 1}
{'Amit': 'Pass', 'Riya': 'Fail', 'Karan': 'Pass'}
Karan
['Amit', 'Karan']

```

Constraints

Maximum 1,000 students.

Every student has at least one score.

Scores are integers from 0 to 100.

Course and student names are non-empty strings.

Passing score uses >=.

above_average() uses strict >.

Q12. PET CARE RECORD TRACKER

This is a same-logic original-style question.

Problem Statement

A veterinary center maintains records of pets brought in by customers. The center wants a simple system that can manage pet names, record visit counts, retrieve the current number of visits for a pet, and determine the pet with the highest number of visits.

Class

```
class PetCareTracker
```

Attributes

pets → list

visits → dictionary

METHODS

```
__init__(self)
```

Initialize empty list and dictionary.

```
add_pet(self, pet_name)
```

Add the pet if it does not already exist.

Initialize visits to 0.

```
record_visit(self, pet_name)
```

Increase visit count by 1.

Ignore unknown pets.

```
get_visit_count(self, pet_name)
```

Return visit count.

Return 0 for an unknown pet.

```
most_frequent_pet(self)
```

Return the pet with the highest visit count.

Earliest-added pet wins a tie.

Return None for an empty system.

EXPECTED OUTPUT

```
tracker = PetCareTracker()
```

```
tracker.add_pet("Bruno")
```

```
tracker.add_pet("Milo")
```

```
tracker.add_pet("Coco")
```

```
tracker.record_visit("Bruno")
```

```
tracker.record_visit("Bruno")
```

```
tracker.record_visit("Milo")
```

```
print(tracker.get_visit_count("Bruno"))
```

```
print(tracker.get_visit_count("Milo"))
```

```
print(tracker.get_visit_count("Coco"))
```

```
print(tracker.get_visit_count("Rocky"))
```

```
print("Most Frequent Pet:", tracker.most_frequent_pet())
```

2

1

```
0
0
Most Frequent Pet: Bruno
Constraints
Maximum 1,000 pets.
Pet names are non-empty strings.
Pet names are case-sensitive.
Duplicate pets are ignored.
Visits for unknown pets are ignored.
Earliest-added pet wins ties.
Q13. ELECTRICITY USAGE ANALYZER ★ ENHANCED
Problem Statement
```

A utility company records monthly electricity consumption for different households. Each household belongs to a locality and has several monthly usage readings. The company needs a system that can calculate average usage, total consumption, count households by locality, identify the household with the highest consumption, and identify households whose total consumption exceeds a specified limit.

```
Class
class ElectricityUsageAnalyzer
Attribute
households
```

Example:

```
{
    "House101": {
        "locality": "North",
        "usage": [120, 140, 160]
    }
}
METHODS
__init__(self, households)
```

Store the supplied data.

```
average_usage(self)
```

Calculate the average monthly usage for each household.

```
locality_counts(self)
```

Count the number of households in every locality.

```
total_usage(self)
```

Calculate total electricity usage for every household.

```
highest_usage_household(self)
```

Return the household with the highest total usage.

If there is a tie, the earliest household wins.

```
high_consumption(self, limit)
```

Return household names whose total usage is strictly greater than limit.

Sort the result alphabetically.

EXPECTED OUTPUT

```
analyzer = ElectricityUsageAnalyzer({
```

```

    "House101": {
        "locality": "North",
        "usage": [120, 140, 160]
    },
    "House102": {
        "locality": "South",
        "usage": [100, 110, 90]
    },
    "House103": {
        "locality": "North",
        "usage": [200, 180, 220]
    }
})

print(analyzer.average_usage())
print(analyzer.locality_counts())
print(analyzer.total_usage())
print(analyzer.highest_usage_household())
print(analyzer.high_consumption(400))
{'House101': 140.0, 'House102': 100.0, 'House103': 200.0}
{'North': 2, 'South': 1}
{'House101': 420, 'House102': 300, 'House103': 600}
House103
['House101', 'House103']
Constraints
Number of households will not exceed 1,000.
Each household has at least one usage reading.
Usage values are non-negative integers.
Household and locality names are non-empty strings.
high_consumption() uses strict >.
Results from high_consumption() must be alphabetically sorted.
Ties in highest_usage_household() use original dictionary order.

```

Q1. INVENTORY PERFORMANCE ANALYZER

Problem Statement

A retail warehouse records the stock quantities of a product over a sequence of time periods. The warehouse team needs a tool that can prepare the raw stock data for analysis, validate the values, calculate important inventory statistics, classify stock levels against a reorder threshold, identify the longest period of consecutive stock growth, and produce a human-readable representation of the stock values.

The data is provided as a 1D sequence of inventory quantities in chronological order.

Objective

Design and implement a Python class named `InventoryAnalyzer` that uses NumPy to perform analytical operations on a 1D inventory dataset. You are expected to demonstrate NumPy array creation with an explicit data type, multi-condition validation, aggregation into a metrics tuple, conditional categorisation, a sequential algorithmic computation, and element-wise string formatting.

Task

Class Structure

Class Name: `InventoryAnalyzer`

The class is stateless: every method receives the data it needs as an argument. Do NOT store the array on the instance.

Methods

Method 1: `create_inventory_array(self, inventory_list)`

Signature:

```
def create_inventory_array(self, inventory_list)
```

Parameters:

`inventory_list` (list) – A list of stock quantities in chronological order.

Sample Input

```
inventory_list = [250, 180, 220, 300, 270]
```

Sample Output

```
[250. 180. 220. 300. 270.]
```

The input list is converted into a 1D NumPy array using a floating-point data type.

Logic: Convert the input list into a NumPy array with `float64` dtype and return it.

Returns: `numpy.ndarray (float)`

Method 2: `validate_inventory_array(self, inventory_array)`

Signature:

```
def validate_inventory_array(self, inventory_array)
```

Parameters:

`inventory_array` (`numpy.ndarray`) – The inventory data.

Sample Input

```
inventory_array = [250. 180. 220. 300. 270.]
```

Sample Output
True

The array is valid because it is non-empty and contains no negative values.

Logic: Return False if the array is empty OR if it contains any negative value.
Otherwise return True.

Returns: bool

Method 3: compute_inventory_metrics(self, inventory_array)

Signature:

```
def compute_inventory_metrics(self, inventory_array)
```

Parameters:

inventory_array (numpy.ndarray) - The inventory data.

Sample Input

```
inventory_array = [250. 180. 220. 300. 270.]
```

Sample Output

```
(1220, 244.0, 300)
```

Logic: Compute and return a tuple of three values:

total - Sum of all stock quantities as a Python int

average - Mean inventory quantity rounded to 2 decimal places as a float

peak - Maximum inventory quantity as a Python int

Returns:

tuple -> (total, average, peak)

Method 4: classify_stock(self, inventory_array, reorder_level)

Signature:

```
def classify_stock(self, inventory_array, reorder_level)
```

Parameters:

inventory_array (numpy.ndarray) - The inventory data.

reorder_level (number) - Minimum stock level.

Sample Input

```
inventory_array = [250. 180. 220. 300. 270.]
```

```
reorder_level = 220
```

Sample Output

```
['Sufficient' 'Low' 'Sufficient' 'Sufficient' 'Sufficient']
```

A value is labelled "Sufficient" when it is greater than or equal to the reorder level. Otherwise it is labelled "Low".

Logic: For each value, return "Sufficient" if it is $\geq$ reorder_level, otherwise return "Low". Return the labels as a 1D NumPy array.

Returns: numpy.ndarray (strings)

Method 5: longest_stock_growth(self, inventory_array)

Signature:

```
def longest_stock_growth(self, inventory_array)
```

Parameters:

inventory_array (numpy.ndarray) – The inventory data.

Sample Input

```
inventory_array = [250. 180. 220. 300. 270.]
```

Sample Output

3

The longest strictly increasing sequence is:

180 -> 220 -> 300

which contains 3 periods.

Logic: If the array is empty, return 0.

Use variables to track the current increasing streak and maximum streak. Compare each value with the previous value. If the current value is greater, increase the current streak. Otherwise reset the current streak to 1.

Return the longest strictly increasing consecutive streak.

Returns: int

Method 6: format_inventory_report(self, inventory_array)

Signature:

```
def format_inventory_report(self, inventory_array)
```

Parameters:

inventory_array (numpy.ndarray) – The inventory data.

Sample Input

```
inventory_array = [250. 180. 220. 300. 270.]
```

Sample Output

```
['250 units' '180 units' '220 units' '300 units' '270 units']
```

Logic: Convert each inventory value to its integer form followed by " units".

Return the results as a 1D NumPy array of strings.

Returns: numpy.ndarray (strings)

Constraints

The input list contains numeric inventory values in chronological order.

create_inventory_array must produce a floating-point NumPy array.

validate_inventory_array returns False for an empty array or any negative value.

compute_inventory_metrics returns (int total, float average, int peak).

classify_stock uses >= for comparison.

longest_stock_growth counts consecutive strictly increasing values.

An empty array returns 0 for the growth streak.

format_inventory_report uses the integer form followed by " units".

Every result must be derived from the input array.

Execution Instructions

To run and check the result:

Click the Run Test Case button on the interface.

To run from the terminal:

```
python main.py
```

Q2. ELECTRICITY CONSUMPTION ANALYZER

Problem Statement

A utility company maintains a customer registry and daily electricity-consumption records. The usage data may contain blank readings, negative values caused by faulty meters, and records belonging to customers who are not present in the registry.

The operations team needs to clean the raw datasets, associate each valid reading with its service zone, identify days with unusually high electricity consumption, calculate average consumption by zone, and determine the day with the highest total consumption.

Objective

Implement six Python functions using Pandas to analyze customer and electricity-consumption data from CSV files.

The solution should demonstrate:

Reading CSV files

Handling missing values using dropna

Type conversion using astype

Boolean filtering

Inner merging with pd.merge

groupby

sum and mean

Sorting

Rounding

Dictionary construction

FILE STRUCTURE

Project/

```
| -- data/  
|   |-- Customers.csv  
|   |-- ElectricityUsage.csv  
|  
|-- main.py
```

Dataset Description

Customers.csv – Customer Registry

Column Name Description

CustomerID Unique identifier for each customer

Zone Service zone: North, South, East, or West

Category Customer category: Residential or Commercial

ElectricityUsage.csv – Daily Electricity Readings

Column Name Description

Date Date of electricity reading

CustomerID References a customer

UsageUnits Electricity consumed on that date

The usage file may contain missing values, negative values, and unknown customer IDs. These must be handled during the analysis.

Function 1: load_and_clean_data(customers_path, usage_path)

Signature:

```
def load_and_clean_data(customers_path, usage_path)
```

Parameters:

customers_path (str) – Path to Customers.csv.

usage_path (str) – Path to ElectricityUsage.csv.

Sample Input

```
data/Customers.csv
data/ElectricityUsage.csv
Sample Output
customers_df -> shape (4, 3)
usage_df -> shape (7, 3)
```

Logic: Read both CSV files into Pandas DataFrames. Remove rows containing missing values from the usage DataFrame.

Returns:

```
(customers_df, usage_df)
Function 2: clean_usage_values(usage_df)
```

Signature:

```
def clean_usage_values(usage_df)
```

Parameters:

usage_df (DataFrame) - Cleaned usage DataFrame returned by Function 1.

Sample Input

Date	CustomerID	UsageUnits
2025-08-01	C001	450
2025-08-01	C002	620
2025-08-02	C003	-50
2025-08-02	C004	700

Sample Output

UsageUnits converted to float
Negative reading removed

Logic:

Convert the UsageUnits column to a numeric floating-point type.
Keep only rows where UsageUnits >= 0.
Return the resulting DataFrame.

Returns: DataFrame

```
Function 3: merge_usage_with_zones(customers_df, usage_df)
```

Signature:

```
def merge_usage_with_zones(customers_df, usage_df)
```

Parameters:

customers_df (DataFrame) - Customer registry.

usage_df (DataFrame) - Valid usage data.

Sample Output

Date	CustomerID	UsageUnits	Zone	Category
2025-08-01	C001	450.0	North	Residential
2025-08-01	C002	620.0	East	Commercial
2025-08-02	C004	700.0	South	Residential

Logic: Combine the two DataFrames using an inner join on CustomerID.

Only usage records having a matching customer in the registry should remain.

Returns: Merged DataFrame

```
Function 4: detect_high_usage_days(merged_df, threshold)
```

Signature:

```
def detect_high_usage_days(merged_df, threshold)
```

Parameters:

merged_df (DataFrame) – Merged DataFrame.

threshold (number) – Daily electricity-consumption threshold.

Sample Input

threshold = 1000

Sample Output

['2025-08-01', '2025-08-03']

Logic:

Calculate total UsageUnits for each Date.

Keep only dates whose total is strictly greater than the threshold.

Return the dates sorted in ascending order.

Returns: Sorted list of date strings

Function 5: zone_wise_average_usage(merged_df)

Signature:

```
def zone_wise_average_usage(merged_df)
```

Parameters:

merged_df (DataFrame) – Merged DataFrame.

Sample Output

{'East': 735.0, 'North': 520.25, 'South': 410.0}

Logic:

Calculate mean UsageUnits for each Zone.

Round each average to 2 decimal places.

Sort the zones in descending order of average consumption.

Convert the result into a dictionary.

Returns: Dictionary mapping zone to average usage.

Function 6: peak_usage_day(merged_df)

Signature:

```
def peak_usage_day(merged_df)
```

Parameters:

merged_df (DataFrame) – Merged DataFrame.

Sample Output

'2025-08-03'

Logic: Calculate the total electricity usage for every date and return the date having the highest total consumption.

The comparison must use the total consumption for each date, not an individual reading.

Returns: Date string

Constraints

Both CSV files are located inside the data/ directory.

UsageUnits must be converted to a numeric type.

Rows containing missing usage values must be removed.
Negative usage values must be removed.
The merge must be an inner join on CustomerID.
High-usage dates must use a strict > comparison.
High-usage dates must be sorted in ascending order.
Zone averages must be rounded to 2 decimal places.
Zone averages must be sorted in descending order.
All calculations must be derived from the provided DataFrames.
Execution Instructions

To run and check the result:

Click the Run Test Case button.

To run from the terminal:

```
python main.py
```

Q3. TEMPERATURE TREND ANALYZER

Problem Statement

A weather station records temperatures at regular intervals. The readings are stored as a chronological sequence. The analysis team wants to validate the readings, calculate temperature statistics, classify readings according to a comfort threshold, determine the longest period of continuous temperature increase, and generate formatted temperature labels.

Objective

Design and implement a Python class named TemperatureAnalyzer using NumPy to perform analytical operations on a 1D temperature dataset.

The implementation should demonstrate array creation, validation, aggregation, conditional categorisation, sequential processing, and element-wise string formatting.

Task

Class Structure

Class Name: TemperatureAnalyzer

The class is stateless. Do not store the input array as an instance variable.

Methods

Method 1: create_temperature_array(self, temperature_list)

```
def create_temperature_array(self, temperature_list)
```

Sample Input

```
temperature_list = [24, 21, 25, 29, 27]
```

Sample Output

```
[24. 21. 25. 29. 27.]
```

Logic: Convert the list into a 1D NumPy array using float64.

Returns: numpy.ndarray

Method 2: validate_temperature_array(self, temperature_array)

```
def validate_temperature_array(self, temperature_array)
```

Sample Input

[24. 21. 25. 29. 27.]

Sample Output

True

Logic: Return False if the array is empty or contains a negative temperature. Otherwise return True.

Returns: bool

Method 3: compute_temperature_metrics(self, temperature_array)
def compute_temperature_metrics(self, temperature_array)

Sample Input

[24. 21. 25. 29. 27.]

Sample Output

(126, 25.2, 29)

Logic: Return:

(total, average, peak)

where:

total is the sum as an integer.
average is the mean rounded to 2 decimal places.
peak is the maximum value as an integer.

Returns: tuple

Method 4: classify_temperature(self, temperature_array, threshold)
def classify_temperature(self, temperature_array, threshold)

Sample Input

temperature_array = [24. 21. 25. 29. 27.]
threshold = 25

Sample Output

['Cool' 'Cool' 'Warm' 'Warm' 'Warm']

Logic: Label each temperature "Warm" when it is $\geq$ threshold; otherwise label it "Cool".

Return a 1D NumPy string array.

Returns: numpy.ndarray

Method 5: longest_rising_streak(self, temperature_array)
def longest_rising_streak(self, temperature_array)

Sample Input

[24. 21. 25. 29. 27.]

Sample Output

The longest increasing sequence is:

21 -> 25 -> 29

Logic: Find the longest strictly increasing sequence of consecutive readings.

An empty array returns 0.

Returns: int

```
Method 6: format_temperature_report(self, temperature_array)
def format_temperature_report(self, temperature_array)
```

Sample Input

[24. 21. 25. 29. 27.]

Sample Output

['24 C' '21 C' '25 C' '29 C' '27 C']

Logic: Convert each value to its integer form followed by " C".

Returns: numpy.ndarray of strings

Constraints

Input values are numeric and chronologically ordered.

The created array must use float64.

Empty or negative-containing arrays are invalid.

Metrics must return (int, float, int).

Classification uses >=.

Rising streak means strictly increasing consecutive values.

Empty input returns 0 for the streak.

Formatted values must use integer representation followed by " C".

Execution Instructions

Click the Run Test Case button.

Alternatively:

```
python main.py
```

Q4. FITNESS ACTIVITY DATA ANALYZER

Problem Statement

A fitness center maintains member information and daily activity records.

Activity records can contain missing step counts, invalid negative values, and entries belonging to members who are no longer present in the member registry.

The fitness team wants to clean the data, associate each activity record with the member's fitness category, identify high-activity days, calculate average activity by category, and determine the day with the highest total number of steps.

Objective

Implement six Python functions using Pandas to analyze member and activity data stored in CSV files.

The implementation should demonstrate CSV loading, missing-value handling, numeric conversion, Boolean filtering, inner merging, grouping, aggregation, sorting, rounding, and dictionary construction.

FILE STRUCTURE

Project/

| -- data/

| | -- Members.csv

```
| `-- ActivityLogs.csv
```

```
| `-- main.py
```

Dataset Description

Members.csv – Member Registry

Column Name Description

MemberID Unique member identifier

Category Fitness category: Beginner, Regular, Advanced

Plan Membership plan: Basic or Premium

ActivityLogs.csv – Daily Activity Records

Column Name Description

Date Activity date

MemberID References a member

Steps Number of steps recorded

Function 1: load_and_clean_data(members_path, activity_path)

```
def load_and_clean_data(members_path, activity_path)
```

Logic: Read both CSV files into DataFrames and remove rows containing missing values from the activity DataFrame.

Returns:

```
(members_df, activity_df)
```

Sample Output

```
members_df -> shape (5, 3)
```

```
activity_df -> shape (8, 3)
```

Function 2: clean_activity_values(activity_df)

```
def clean_activity_values(activity_df)
```

Logic:

Convert Steps to a numeric floating-point type.

Remove rows where Steps is negative.

Return only valid activity records.

Sample Input

Date	MemberID	Steps
2025-08-10	M001	6500
2025-08-10	M002	-200
2025-08-11	M003	8200

Sample Output

Date	MemberID	Steps
2025-08-10	M001	6500.0
2025-08-11	M003	8200.0

Returns: DataFrame

Function 3: merge_activity_with_members(members_df, activity_df)

```
def merge_activity_with_members(members_df, activity_df)
```

Logic: Perform an inner merge using MemberID.

Only activity records belonging to registered members should remain.

Sample Output

Date	MemberID	Steps	Category	Plan
2025-08-10	M001	6500.0	Regular	Premium
2025-08-11	M003	8200.0	Advanced	Premium

Returns: Merged DataFrame

Function 4: detect_high_activity_days(merged_df, threshold)

```
def detect_high_activity_days(merged_df, threshold)
```

Sample Input

threshold = 15000

Sample Output

```
['2025-08-10', '2025-08-12']
```

Logic:

Calculate total Steps for every date.
Keep only dates where the total is strictly greater than the threshold.
Sort the dates in ascending order.

Returns: List of date strings

Function 5: category_wise_average_steps(merged_df)
def category_wise_average_steps(merged_df)
Sample Output
{'Advanced': 9125.0, 'Regular': 7350.5, 'Beginner': 5210.0}

Logic:

Group records by Category.
Calculate the mean of Steps.
Round averages to 2 decimal places.
Sort categories by average steps in descending order.
Convert the result to a dictionary.

Returns: Dictionary

Function 6: peak_activity_day(merged_df)
def peak_activity_day(merged_df)
Sample Output
'2025-08-12'

Logic: Calculate the total steps for every date and return the date with the highest total.

Returns: Date string

Constraints

- Both CSV files are stored in data/.
- Steps must be converted to a numeric type.
- Missing activity records must be removed.
- Negative step counts must be removed.
- The merge must use an inner join on MemberID.
- High-activity detection must use strict >.
- Dates must be sorted ascending.
- Category averages must be rounded to 2 decimal places.
- Category averages must be sorted descending.
- Peak activity must be based on daily totals.

Execution Instructions

python main.py

Q5. PRODUCT ORDER MANAGEMENT SYSTEM

Problem Statement

A small online store maintains customer orders in memory. Each order contains an order ID, customer name, product, quantity, price, and order status.

The store needs a Python class that can validate orders, calculate order values, maintain unique order records, update order status, summarize sales by product, and identify the most valuable completed order.

The solution should work with standard Python data structures and should not

depend on NumPy or Pandas.

Objective

Design and implement a Python class named OrderManager that demonstrates:

Lists
Dictionaries
Sets
Tuples
Loops
Conditional statements
Sorting
Dictionary aggregation
String formatting

The class must maintain its order collection internally.

Task

Class Structure

Class Name: OrderManager

The constructor should initialize an empty collection of orders.

Methods

Method 1: add_order(self, order)

Signature:

```
def add_order(self, order)
```

Parameters:

order is a dictionary containing:

```
{
    "order_id": "0101",
    "customer": "Amit",
    "product": "Keyboard",
    "quantity": 2,
    "price": 1200,
    "status": "Completed"
}
```

Logic:

Add the order only when its order_id does not already exist.

Return:

True

when successfully added, otherwise:

False

Returns: bool

Method 2: validate_order(self, order)

```
def validate_order(self, order)
```

Logic: An order is valid only when:

order_id is present.
customer is not empty.
product is not empty.
quantity is greater than 0.
price is greater than 0.
status is one of:

Pending

Completed

Cancelled

Sample Input

```
{
  "order_id": "0102",
  "customer": "Riya",
  "product": "Mouse",
  "quantity": 3,
  "price": 500,
  "status": "Completed"
}
```

Sample Output

True

Returns: bool

Method 3: calculate_order_total(self, order_id)

```
def calculate_order_total(self, order_id)
```

Logic: Find the order using its ID and calculate:

quantity × price

Sample Output

1500

If the order does not exist, return:

0

Returns: number

Method 4: update_order_status(self, order_id, new_status)

```
def update_order_status(self, order_id, new_status)
```

Logic: Update the status only when:

The order exists.

new_status is valid.

Valid statuses:

Pending

Completed

Cancelled

Sample Output

True

Return False if the order does not exist or the status is invalid.

Returns: bool

Method 5: product_sales_summary(self)

```
def product_sales_summary(self)
```

Logic: Consider only orders with status "Completed".

Calculate the total sales amount for every product.

For example:

Keyboard -> 5000
Mouse -> 3000
Monitor -> 12000

Return a dictionary sorted from highest sales amount to lowest.

Sample Output

```
{  
    "Monitor": 12000,  
    "Keyboard": 5000,  
    "Mouse": 3000  
}
```

Returns: dict

Method 6: `highest_value_order(self)`
`def highest_value_order(self)`

Logic: Consider only completed orders.

Calculate each order's total value and return a tuple containing:

`(order_id, total_value)`
Sample Output
`('0105', 18000)`

If there are no completed orders, return:

None

Returns: tuple or None

Constraints

Every order must contain all required fields.

Order IDs must be unique.

Quantity and price must be positive.

Only Pending, Completed, and Cancelled are valid statuses.

Sales summaries must consider completed orders only.

Product sales must be sorted in descending order.

The highest-value order must be calculated dynamically.

No hard-coded output values should be used.

Standard Python collections must be used.

Execution Instructions

To run:

`python main.py`

Q6. COURSE ENROLLMENT AND RESULT MANAGER

Problem Statement

A training institute maintains student enrollment and course-result information. Each student can enroll in multiple courses. The institute wants to process the records to identify valid enrollments, calculate grades, determine course-wise statistics, find students who completed all required courses, and generate a concise student report.

The data is represented using standard Python collections.

Objective

Design and implement a Python class named CourseManager that uses dictionaries, lists, sets, loops, conditional statements, sorting, and string formatting to manage course enrollment and results.

Task

Class Structure

Class Name: CourseManager

The class should maintain student and course information internally.

Methods

Method 1: add_student(self, student_id, name)

```
def add_student(self, student_id, name)
```

Logic: Add a student only if the student_id does not already exist.

Sample Input

```
student_id = "S101"
```

```
name = "Neha"
```

Sample Output

```
True
```

If the ID already exists:

```
False
```

Returns: bool

Method 2: enroll_student(self, student_id, course)

```
def enroll_student(self, student_id, course)
```

Logic: Add the student to the specified course only when:

The student exists.

The student is not already enrolled in the course.

Sample Input

```
student_id = "S101"
```

```
course = "Python"
```

Sample Output

```
True
```

If the student does not exist or is already enrolled:

```
False
```

Returns: bool

Method 3: record_result(self, student_id, course, marks)

```
def record_result(self, student_id, course, marks)
```

Logic: Record marks only when:

The student exists.

The student is enrolled in the course.

Marks are between 0 and 100.

Sample Input

```
student_id = "S101"
```

```
course = "Python"
```

```
marks = 86
```

Sample Output

```
True
```

Returns: bool

Method 4: calculate_grade(self, marks)
def calculate_grade(self, marks)

Use the following grading rules:

Marks Grade
90-100 A+
80-89 A
70-79 B
60-69 C
50-59 D
Below 50 F
Sample Input
marks = 86
Sample Output
'A'

Returns: str

Method 5: course_average(self, course)
def course_average(self, course)

Logic: Calculate the average marks of all students who have a recorded result for the specified course.

Round the result to 2 decimal places.

Sample Input
course = "Python"
Sample Output
78.67

If no results exist for the course, return:

0.0

Returns: float

Method 6: student_report(self, student_id)
def student_report(self, student_id)

Logic: Generate a report for the specified student.

For every course having a recorded result, display:

Course: Python, Marks: 86, Grade: A
Course: Java, Marks: 74, Grade: B

Return the report as a list of strings.

Sample Output
[
 'Python: 86 - A',
 'Java: 74 - B'
]

The courses should appear in alphabetical order.

If the student does not exist, return:

[]

Returns: list of strings

Constraints

Student IDs must be unique.

A student must exist before enrollment.

Duplicate course enrollment is not allowed.

Results can only be recorded for enrolled courses.

Marks must be between 0 and 100.

Grades must be calculated using the specified ranges.

Course averages must be rounded to 2 decimal places.

Student reports must be sorted alphabetically by course name.

Missing students must be handled without crashing.

All results must be calculated dynamically from the stored data.

Do not hard-code expected outputs.

Use standard Python data structures and control flow.

A1. Weather Station Rainfall Analyzer

Problem Statement

A weather station records its **daily rainfall** and needs a small analytics helper built with **NumPy**. The station wants the raw daily readings organised into a **NumPy array**, checked for validity, summarised into a few headline figures, labelled against a **rainfall threshold**, examined for the longest stretch of continuously rising rainfall, and presented as a clean, readable report. All of these related operations are grouped together inside a single analyzer class.

Objective

Design a Python class named **`RainfallAnalyzer`** that analyses daily rainfall using NumPy. Assesses class definition and instance methods, NumPy array creation and dtype handling, basic aggregate operations, comparison and labelling, a single sequential scan over an array, and string formatting. The class is stateless and defines no constructor; every method receives the data it needs as an argument.

File Structure

...

Project

```
|— main.py
|— tests.py
|— installation.txt
|— ...
```

- `main.py` – Contains the `RainfallAnalyzer` class implementation and a runnable example inside the main block.

- `tests.py` – Holds the automated test cases used to evaluate the implementation.

- `installation.txt` – Lists the Python dependencies required to run the program.

Task – Method Overview

Implement a class named `RainfallAnalyzer` that groups six related operations on a day-by-day rainfall series.

#	Method	Signature
1	<code>create_rainfall_array</code>	<code>`create_rainfall_array(self, rainfall_list)`</code>
2	<code>validate_rainfall_array</code>	<code>`validate_rainfall_array(self, rainfall_array)`</code>

```
|
| 3 | compute_rainfall_metrics | `compute_rainfall_metrics(self,
rainfall_array)` |
| 4 | flag_heavy_rainfall_days | `flag_heavy_rainfall_days(self, rainfall_array,
threshold)` |
| 5 | longest_rising_streak | `longest_rising_streak(self, rainfall_array)` |
| 6 | format_rainfall_report | `format_rainfall_report(self, rainfall_array)` |
```

****Method 1 – create_rainfall_array****

Parameters: `rainfall\_list` – Python list of daily rainfall values (mm), each a non-negative int or float.

Sample I/O – Input: `[12, 18, 15, 25, 30, 40]` → Output: `[12. 18. 15. 25. 30. 40.]`

Logic: Produce a NumPy representation of the daily rainfall held as floating-point numbers.

Returns: A NumPy array of dtype `float64`.

****Method 2 – validate_rainfall_array****

Parameters: `rainfall\_array` – NumPy array of daily rainfall, typically the result of Method 1.

Sample I/O – Input: `[12.0, 18.0, 15.0, 25.0, 30.0, 40.0]` → Output: `True`

Logic: The array is valid only when it contains at least one value and no value is negative; zero is acceptable.

Returns: Boolean.

****Method 3 – compute_rainfall_metrics****

Sample I/O – Input: `[12.0, 18.0, 15.0, 25.0, 30.0, 40.0]` → Output: `(140.0, 23.33, 40.0)`

Logic: Summarise into three headline figures – total across all days, average across the days, and the highest single day – each rounded to two decimal places.

Returns: Tuple of `(total, average, highest)`.

****Method 4 – flag_heavy_rainfall_days****

Parameters: `rainfall\_array`; `threshold` – numeric rainfall threshold in mm.

Sample I/O – Input: `rainfall=[12,18,15,25,30,40]`, `threshold=20` → Output: `['Light' 'Light' 'Light' 'Heavy' 'Heavy' 'Heavy']`

Logic: A day is `Heavy` when rainfall meets or exceeds the threshold, `Light` otherwise (a day exactly equal to the threshold is `Heavy`).

Returns: NumPy array of label strings, one per day.

****Method 5 – longest_rising_streak****

Sample I/O – Input: `[12.0, 18.0, 15.0, 25.0, 30.0, 40.0]` → Output: `4`

Logic: Report the length of the longest run of consecutive days whose rainfall strictly increases from one day to the next. A single day counts as a run of length one; an empty array gives zero; equal consecutive values do not extend a run.

Returns: Integer.

****Method 6 – format_rainfall_report****

Sample I/O – Input: `[12.0, 18.0, 15.0, 25.0, 30.0, 40.0]` → Output: `['12.00 mm' '18.00 mm' '15.00 mm' '25.00 mm' '30.00 mm' '40.00 mm']`

Logic: Present each day's value as a readable string shown to exactly two decimal places, followed by the suffix `mm`.

Returns: NumPy array of formatted strings.

****Main Block****

Under `if \_\_name\_\_ == "\_\_main\_\_"`, create an instance of `RainfallAnalyzer`, build a sample daily-rainfall array, and call each method in turn, printing the validity result, the summary metrics, the heavy-rainfall labels, the longest rising streak, and the formatted report.

****Expected Output****

```
```
```

```
Valid: True
Metrics (total, average, highest): (140.0, 23.33, 40.0)
Heavy Rainfall Days (threshold=20): ['Light' 'Light' 'Light' 'Heavy' 'Heavy'
'Heavy']
Longest Rising Streak: 4
Report: ['12.00 mm' '18.00 mm' '15.00 mm' '25.00 mm' '30.00 mm' '40.00 mm']
```
```

****Constraints****

- Use NumPy arrays to hold and process the rainfall data.
- Rainfall values are non-negative; validation must reject an empty array or any negative value, while zero is allowed.
- ``compute_rainfall_metrics`` rounds each figure to two decimal places;
- ``format_rainfall_report`` displays each value to exactly two decimal places.
- ``flag_heavy_rainfall_days`` uses a greater-than-or-equal comparison against the threshold.
- ``longest_rising_streak`` uses a strict increase; equal consecutive values do not extend a streak, and an empty array returns zero.
- Use only basic NumPy operations; advanced helpers are not required.

A2. Fitness Tracker Step Count Analyzer

****Problem Statement****

A fitness tracker app records a user's ****daily step count**** and needs a small analytics helper built with ****NumPy****. The app wants the raw daily counts organised into a ****NumPy array****, checked for validity, summarised into headline figures, labelled against a ****daily step goal****, examined for the longest streak of rising activity, and presented as a clean, readable summary. All of these related operations are grouped together inside a single analyzer class.

****Objective****

Design a Python class named ****`StepCountAnalyzer`**** that analyses daily step counts using NumPy. Assesses class definition and instance methods, NumPy array creation and dtype handling, basic aggregate operations, comparison and labelling, a single sequential scan over an array, and string formatting. The class is stateless and defines no constructor; every method receives the data it needs as an argument.

****File Structure****

```

#### **Project**

```
|— main.py
|— tests.py
|— installation.txt
```
```

****Task – Method Overview****

#	Method	Signature
1	<code>create_steps_array</code>	<code>`create_steps_array(self, steps_list)`</code>
2	<code>validate_steps_array</code>	<code>`validate_steps_array(self, steps_array)`</code>
3	<code>compute_steps_metrics</code>	<code>`compute_steps_metrics(self, steps_array)`</code>
4	<code>flag_goal_days</code>	<code>`flag_goal_days(self, steps_array, goal)`</code>
5	<code>longest_activity_streak</code>	<code>`longest_activity_streak(self, steps_array)`</code>
6	<code>format_steps_summary</code>	<code>`format_steps_summary(self, steps_array)`</code>

****Method 1 – create_steps_array****

Parameters: ``steps_list`` – Python list of daily step counts, each a non-negative int or float.

Sample I/O – Input: ``[8000, 9500, 8700, 10200, 11000, 12500]`` → Output: ``[8000.`

9500. 8700. 10200. 11000. 12500.]`
Returns: A NumPy array of dtype `float64`.

****Method 2 – validate_steps_array****

Sample I/O – Input: `[8000.0, 9500.0, 8700.0, 10200.0, 11000.0, 12500.0]` →
Output: `True`

Logic: Valid only when the array has at least one value and no value is negative; zero is acceptable.

Returns: Boolean.

****Method 3 – compute_steps_metrics****

Sample I/O – Input: `[8000.0, 9500.0, 8700.0, 10200.0, 11000.0, 12500.0]` →
Output: `(59900.0, 9983.33, 12500.0)`

Logic: Total across all days, average across days, and the highest single day – each rounded to two decimal places.

Returns: Tuple `(total, average, highest)`.

****Method 4 – flag_goal_days****

Parameters: `steps\_array`; `goal` – numeric daily step goal.

Sample I/O – Input: `steps=[8000,9500,8700,10200,11000,12500]`, `goal=9500` →
Output: `['Missed' 'Met' 'Missed' 'Met' 'Met' 'Met']`

Logic: `Met` when the day's steps meet or exceed the goal, `Missed` otherwise; a day exactly equal to the goal is `Met`.

Returns: NumPy array of labels.

****Method 5 – longest_activity_streak****

Sample I/O – Input: `[8000.0, 9500.0, 8700.0, 10200.0, 11000.0, 12500.0]` →
Output: `4`

Logic: Longest run of consecutive days whose steps strictly increase from one day to the next.

Returns: Integer.

****Method 6 – format_steps_summary****

Sample I/O – Input: `[8000.0, 9500.0, 8700.0, 10200.0, 11000.0, 12500.0]` →
Output: `['8000.00 steps' '9500.00 steps' '8700.00 steps' '10200.00 steps' '11000.00 steps' '12500.00 steps']`

Logic: Each value shown to two decimal places, followed by the suffix `steps`.

Returns: NumPy array of formatted strings.

****Expected Output****

...

Valid: True

Metrics (total, average, highest): (59900.0, 9983.33, 12500.0)

Goal Days (goal=9500): ['Missed' 'Met' 'Missed' 'Met' 'Met' 'Met']

Longest Activity Streak: 4

Summary: ['8000.00 steps' '9500.00 steps' '8700.00 steps' '10200.00 steps' '11000.00 steps' '12500.00 steps']

...

****Constraints****

Same constraint pattern as A1 (non-negative validation with rejection of empty/negative arrays; rounding to two decimal places; `>=` comparison against the goal; strict increase for the streak; basic NumPy operations only).

A3. Warehouse Inventory Level Analyzer

****Problem Statement****

A warehouse records its ****daily stock level**** for one product line and needs a small analytics helper built with ****NumPy****. The warehouse wants the raw daily readings organised into a ****NumPy array****, checked for validity, summarised into headline figures, labelled against a ****storage limit****, examined for the longest

continuous overstock stretch, and expressed as a daily utilization rate against the warehouse's capacity. All of these related operations are grouped together inside a single analyzer class.

****Objective****

Design a Python class named **`**`InventoryLevelAnalyzer`**`** that analyses one product line's daily stock level using NumPy. Assesses class definition and instance methods, NumPy array creation and dtype handling, basic aggregate operations, comparison and labelling, a single sequential scan over an array, and broadcasting and vectorized arithmetic. The class is stateless and defines no constructor; every method receives the data it needs as an argument.

****File Structure****

```
Project
├── main.py
├── tests.py
└── installation.txt
```

****Task – Method Overview****

#	Method	Signature
1	<code>create_inventory_array</code>	<code>`create_inventory_array(self, inventory_list)`</code>
2	<code>validate_inventory_array</code>	<code>`validate_inventory_array(self, inventory_array)`</code>
3	<code>compute_inventory_metrics</code>	<code>`compute_inventory_metrics(self, inventory_array)`</code>
4	<code>flag_overstock_days</code>	<code>`flag_overstock_days(self, inventory_array, limit)`</code>
5	<code>longest_overstock_stretch</code>	<code>`longest_overstock_stretch(self, inventory_array, threshold)`</code>
6	<code>stock_utilization_rate</code>	<code>`stock_utilization_rate(self, inventory_array, capacity)`</code>

****Method 1 – create_inventory_array****

Sample I/O – Input: ``[22, 30, 28, 35, 40, 38, 18, 25]`` → Output: ``[22. 30. 28. 35. 40. 38. 18. 25.]``
Returns: NumPy array of dtype ``float64``.

****Method 2 – validate_inventory_array****

Sample I/O – Input: ``[22.0, 30.0, 28.0, 35.0, 40.0, 38.0, 18.0, 25.0]`` → Output: ``True``
Logic: Valid only when at least one value exists and no value is negative; zero is acceptable.

****Method 3 – compute_inventory_metrics****

Sample I/O – Input: ``[22.0, 30.0, 28.0, 35.0, 40.0, 38.0, 18.0, 25.0]`` → Output: ``(236.0, 29.5, 40.0)``
Logic: Total, average, and peak stock level, each rounded to two decimal places.
Returns: Tuple ``(total, average, peak)``.

****Method 4 – flag_overstock_days****

Sample I/O – Input: ``inventory=[22,30,28,35,40,38,18,25]`, `limit=40`` → Output: ``['Normal' 'Normal' 'Normal' 'Normal' 'Overstock' 'Normal' 'Normal' 'Normal']``
Logic: ``Overstock`` when a day's stock meets or exceeds the limit, ``Normal`` otherwise.
Returns: NumPy array of labels.

****Method 5 – longest_overstock_stretch****

Sample I/O – Input: ``inventory=[22,30,28,35,40,38,18,25]`, `threshold=30`` → Output: ``3``
Logic: Longest run of consecutive days whose stock is at or above the threshold;

a day below the threshold breaks the run; an empty array gives zero.
Returns: Integer.

****Method 6 – stock_utilization_rate****

Sample I/O – Input: `inventory=[22,30,28,35,40,38,18,25]`, `capacity=40` →

Output: `[55. 75. 70. 87.5 100. 95. 45. 62.5]`

Logic: Each day's stock expressed as a percentage of capacity – divide by capacity and multiply by 100 – rounded to two decimal places.

Returns: NumPy array of per-day utilization rates.

****Expected Output****

Valid: True

Metrics (total, average, peak): (236.0, 29.5, 40.0)

Overstock Days (limit=40): ['Normal' 'Normal' 'Normal' 'Normal' 'Overstock'
'Normal' 'Normal' 'Normal']

Longest Overstock Stretch (>=30): 3

Utilization Rate (capacity=40): [55. 75. 70. 87.5 100. 95. 45. 62.5]

****Constraints****

Non-negative validation (reject empty/negative, allow zero);

`compute\_inventory\_metrics` and `stock\_utilization\_rate` round to two decimal places; `flag\_overstock\_days` uses `>=` against the limit;

`longest\_overstock\_stretch` uses `>=` against the threshold; basic NumPy operations only.

A4. Call Center Daily Ticket Volume Analyzer

****Problem Statement****

A call center records the ****number of support tickets**** it receives each day and needs a small analytics helper built with ****NumPy****. The center wants the raw daily counts organised into a ****NumPy array****, checked for validity, summarised into headline figures, labelled against a ****staffing capacity****, examined for the longest continuous overload stretch, and expressed as a daily resolution-capacity rate. All of these related operations are grouped together inside a single analyzer class.

****Objective****

Design a Python class named ****`TicketVolumeAnalyzer`**** that analyses one team's daily ticket volume using NumPy. Assesses class definition and instance methods, NumPy array creation and dtype handling, basic aggregate operations, comparison and labelling, a single sequential scan over an array, and broadcasting and vectorized arithmetic. The class is stateless and defines no constructor.

****File Structure****

Project

```
|— main.py
|— tests.py
|— installation.txt
---
```

****Task – Method Overview****

#	Method	Signature
1	create_ticket_array	`create_ticket_array(self, ticket_list)`
2	validate_ticket_array	`validate_ticket_array(self, ticket_array)`
3	compute_ticket_metrics	`compute_ticket_metrics(self, ticket_array)`
4	flag_overload_days	`flag_overload_days(self, ticket_array, capacity)`
5	longest_overload_stretch	`longest_overload_stretch(self, ticket_array,

```

threshold)` |
| 6 | resolution_capacity_rate | `resolution_capacity_rate(self, ticket_array,
capacity)` |

**Method 1 – create_ticket_array**
Sample I/O – Input: `[22, 30, 28, 35, 40, 38, 18, 25]` → Output: `[22. 30. 28.
35. 40. 38. 18. 25.]`
Returns: NumPy array of dtype `float64`.

**Method 2 – validate_ticket_array**
Sample I/O – Input: `[22.0, 30.0, 28.0, 35.0, 40.0, 38.0, 18.0, 25.0]` → Output:
`True`
Logic: Valid only when at least one value exists and none is negative.

**Method 3 – compute_ticket_metrics**
Sample I/O – Output: `(236.0, 29.5, 40.0)`
Logic: Total, average, and busiest day, rounded to two decimal places.

**Method 4 – flag_overload_days**
Sample I/O – Input: `capacity=40` → Output: `['Manageable' 'Manageable'
'Manageable' 'Manageable' 'Overloaded' 'Manageable' 'Manageable' 'Manageable']`
Logic: `Overloaded` when the day's tickets meet or exceed capacity, `Manageable`
otherwise.

**Method 5 – longest_overload_stretch**
Sample I/O – Input: `threshold=30` → Output: `3`
Logic: Longest run of consecutive days at or above the threshold.

**Method 6 – resolution_capacity_rate**
Sample I/O – Input: `capacity=40` → Output: `[55. 75. 70. 87.5 100. 95. 45.
62.5]`
Logic: Each day's ticket volume as a percentage of staffing capacity, rounded to
two decimal places.

**Expected Output**
...
Valid: True
Metrics (total, average, busiest): (236.0, 29.5, 40.0)
Overload Days (capacity=40): ['Manageable' 'Manageable' 'Manageable'
'Manageable' 'Overloaded' 'Manageable' 'Manageable' 'Manageable']
Longest Overload Stretch (>=30): 3
Resolution Capacity Rate (capacity=40): [ 55.  75.  70.  87.5 100.  95.  45.
62.5]
...

**Constraints**
Same pattern as A3: non-negative validation, two-decimal rounding, `>=`
comparisons throughout, basic NumPy operations only.

---
```

A5. Student Enrollment & Fee Payment Analyzer

Problem Statement

A college wants to understand student engagement and fee-collection patterns across different courses. The college has provided two datasets: one containing student enrollment information and the other containing fee payment history. The goal is to load, clean, and merge these datasets, then analyze payment engagement metrics, identify revenue patterns by course, and detect students who enrolled but never made a fee payment.

Objective

Implement six Python functions using Pandas to analyze student and fee-payment data from CSV files. Demonstrates reading CSV files, handling missing values

with ``dropna``, type conversion with ``astype``, merging DataFrames, aggregation, groupby operations, and membership filtering using ``isin``.

****File Structure****
...

```
Project/
├── data/
│   ├── Students.csv
│   └── FeePayments.csv
├── main.py
└── installation.txt
...
```

All six functions must be defined at the module level in ``main.py``. Each function must use Pandas operations.

****Dataset Description****

``Students.csv`` – Student enrollment records

- ``StudentID`` – Unique identifier (e.g. S001)
- ``StudentName`` – Full name
- ``Course`` – Course the student is enrolled in
- ``EnrollDate`` – Date of enrollment (YYYY-MM-DD)

``FeePayments.csv`` – Payment transaction records

- ``PaymentID`` – Unique identifier (e.g. P001)
- ``StudentID`` – References a student from `Students.csv`
- ``PaymentDate`` – Date of payment (YYYY-MM-DD)
- ``Amount`` – Payment amount in numeric format

Both datasets may contain rows with missing values that need to be cleaned before analysis.

****Function 1 – `load_and_clean_data(students_path, fees_path)`****
Logic: Read both CSV files into DataFrames. Remove rows with any missing values from both DataFrames.
Returns: Tuple of two cleaned DataFrames ``(students_df, fees_df)``.

****Function 2 – `convert_amounts(fees_df)`****
Logic: Convert the ``Amount`` column to a numeric (float) type so it can be aggregated.
Returns: The fees DataFrame with a float ``Amount`` column.

****Function 3 – `merge_data(students_df, fees_df)`****
Logic: Perform an inner merge on the ``StudentID`` column. This keeps only students who have made at least one payment.
Returns: A merged DataFrame containing student details alongside their payment records.

****Function 4 – `calculate_metrics(students_df, fees_df, merged_df)`****
Logic: Report the shapes of all three DataFrames plus total student count, total payment count, and total revenue.
Returns: A dictionary with keys such as ``students_shape``, ``payments_shape``, ``merged_shape``, ``total_students``, ``total_payments``, ``total_revenue``.

****Function 5 – `find_non_paying_students(students_df, fees_df)`****
Logic: Identify students who enrolled but never made a payment. Collect the StudentIDs that appear in the payments data, keep the students whose IDs are not among them, and return their StudentIDs sorted alphabetically.
Returns: A sorted list of non-paying StudentID strings.

****Function 6 – `course_wise_summary(merged_df)`****
Logic: Group the merged DataFrame by the ``Course`` column. For each course, compute the total number of payments (count of rows, as an integer) and the total revenue (sum of ``Amount``, rounded to 1 decimal place, as a float).

Returns: A dictionary like `{"BSc CS": {"payments": 6, "revenue": 1881.2}, "BCom": {"payments": 7, "revenue": 2307.5}, ...}`.

Note: Write input/output code for testing inside `if __name__ == "__main__":`.

Expected Output (shape values will vary with your own sample data)

```
Students shape: (35, 4)
Payments shape: (45, 4)
Merged shape: (45, 6)
Total Students: 35
Total Payments: 45
Total Revenue: 14741.5
Non-Paying Students: ['S004', 'S012', 'S019', 'S027', 'S033']
```

A6. Employee Attendance & Payroll Analyzer

Problem Statement

An HR department wants to understand employee activity and payroll patterns across departments. Two datasets are provided: one containing employee master records and the other containing monthly payslip history. The goal is to load, clean, and merge these datasets, then analyze payroll engagement metrics, identify salary patterns by department, and detect employees who were onboarded but never received a payslip.

Objective

Implement six Python functions using Pandas to analyze employee and payslip data from CSV files. Demonstrates reading CSV files, handling missing values with `dropna`, type conversion with `astype`, merging DataFrames, aggregation, groupby operations, and membership filtering using `isin`.

File Structure

```
Project/
├── data/
│   ├── Employees.csv
│   └── Payslips.csv
├── main.py
└── installation.txt
```

Dataset Description

`Employees.csv` – Employee master records

- `EmployeeID` – Unique identifier (e.g. E001)
- `EmployeeName` – Full name
- `Department` – Department the employee belongs to
- `JoinDate` – Date of joining (YYYY-MM-DD)

`Payslips.csv` – Monthly payroll records

- `PayslipID` – Unique identifier (e.g. PS001)
- `EmployeeID` – References an employee from `Employees.csv`
- `PayDate` – Date of payment (YYYY-MM-DD)
- `Salary` – Salary amount in numeric format

Function 1 – `load_and_clean_data(employees_path, payslips_path)`

Logic: Read both CSV files into DataFrames. Remove rows with any missing values from both DataFrames.

Returns: Tuple `(employees_df, payslips_df)`.

```

**Function 2 – convert_amounts(payslips_df)**
Logic: Convert the `Salary` column to a numeric (float) type.
Returns: The payslips DataFrame with a float `Salary` column.

**Function 3 – merge_data(employees_df, payslips_df)**
Logic: Perform an inner merge on `EmployeeID`, keeping only employees who have
received at least one payslip.
Returns: Merged DataFrame of employee details alongside payslip records.

**Function 4 – calculate_metrics(employees_df, payslips_df, merged_df)**
Logic: Report shapes of all three DataFrames plus total employee count, total
payslip count, and total salary paid.
Returns: A dictionary of these metrics.

**Function 5 – find_unpaid_employees(employees_df, payslips_df)**
Logic: Identify employees who were onboarded but never received a payslip, using
`isin` against the EmployeeIDs present in the payslip data.
Returns: A sorted list of unpaid EmployeeID strings.

**Function 6 – department_wise_summary(merged_df)**
Logic: Group the merged DataFrame by `Department`. For each department, compute
the total number of payslips (count, as an integer) and the total salary paid
(sum of `Salary`, rounded to 1 decimal place, as a float).
Returns: A dictionary like `{"Engineering": {"payslips": 6, "salary_paid":
188120.0}, "Sales": {"payslips": 7, "salary_paid": 230750.0}, ...}`.

**Expected Output (shape values will vary with your own sample data)**
...
Employees shape: (35, 4)
Payslips shape: (45, 4)
Merged shape: (45, 6)
Total Employees: 35
Total Payslips: 45
Total Salary Paid: 1474150.0
Unpaid Employees: ['E004', 'E012', 'E019', 'E027', 'E033']
...

```

B1. Energy Consumption Analyzer

****Problem Statement****

A smart meter records a household's ****daily energy consumption**** in kWh. Unlike a simple sales tracker, the utility company also enforces a ****maximum sensor reading**** the meter must never exceed, tracks the ****cheapest (minimum) usage day**** rather than the busiest, flags days as efficient relative to the ****array's own average**** (not an external target), and reports how far each day's usage deviates – up or down – from the mean.

****Objective****

Design a Python class named ****`EnergyConsumptionAnalyzer`**** that analyses daily energy usage using NumPy. This exercise differs from a basic aggregation exercise: validation enforces an ****upper bound as well as a lower bound****, the summary reports the ****minimum**** rather than the maximum, day-labelling compares each value against a ****computed average**** (not a caller-supplied target) using a ****strict**** comparator, the streak method tracks days ****at or below a threshold**** (not simply increasing), and the final method returns ****signed percentage deviations****, which can be negative. The class is stateless and defines no constructor.

****File Structure****

```

...
Project
├─ main.py

```

```
└─ tests.py
└─ installation.txt
...
```

****Task – Method Overview****

#	Method	Signature
1	create_usage_array	<code>`create_usage_array(self, usage_list)`</code>
2	validate_usage_array	<code>`validate_usage_array(self, usage_array, max_capacity)`</code>
3	compute_usage_metrics	<code>`compute_usage_metrics(self, usage_array)`</code>
4	flag_efficient_days	<code>`flag_efficient_days(self, usage_array)`</code>
5	longest_efficient_streak	<code>`longest_efficient_streak(self, usage_array, threshold)`</code>
6	compute_usage_deviation	<code>`compute_usage_deviation(self, usage_array)`</code>

****Method 1 – create_usage_array****

Sample I/O – Input: ``[18, 15, 12, 25, 10, 8, 30]`` → Output: ``[18. 15. 12. 25. 10. 8. 30.]``

Returns: NumPy array of dtype ``float64``.

****Method 2 – validate_usage_array****

Parameters: ``usage_array``; ``max_capacity`` – the meter's maximum rated reading.
Sample I/O – Input: ``usage=[18,15,12,25,10,8,30]``, ``max_capacity=35`` → Output: ``True``

Logic: ****Different from a simple non-negative check**** – the array is valid only when it has at least one value, no value is negative, ****and no value exceeds ``max_capacity``****.

Returns: Boolean.

****Method 3 – compute_usage_metrics****

Sample I/O – Input: ``[18.0, 15.0, 12.0, 25.0, 10.0, 8.0, 30.0]`` → Output: ``(118.0, 16.86, 8.0)``

Logic: Total, average, and the ****minimum**** (lowest) single day's usage – each rounded to two decimal places. (Note: minimum, not maximum.)

Returns: Tuple ``(total, average, minimum)``.

****Method 4 – flag_efficient_days****

Parameters: ``usage_array`` only – ****no external target is passed in****.

Sample I/O – Input: ``[18.0, 15.0, 12.0, 25.0, 10.0, 8.0, 30.0]`` → Output: ``['Excessive' 'Efficient' 'Efficient' 'Excessive' 'Efficient' 'Efficient' 'Excessive']``

Logic: Compute the array's own average internally, then label each day ``Efficient`` when its usage is ****strictly less than**** that average, ``Excessive`` otherwise (a day exactly equal to the average is ``Excessive``).

Returns: NumPy array of labels.

****Method 5 – longest_efficient_streak****

Parameters: ``usage_array``; ``threshold`` – a caller-supplied ceiling.

Sample I/O – Input: ``usage=[18,15,12,25,10,8,30]``, ``threshold=20`` → Output: ``3``

Logic: Longest run of consecutive days whose usage is ****at or below**** the threshold (``<=``); a day above the threshold breaks the run; an empty array gives zero. (Runs are ``[18,15,12]`` → length 3, and ``[10,8]`` → length 2; the longer run wins.)

Returns: Integer.

****Method 6 – compute_usage_deviation****

Sample I/O – Input: ``[18.0, 15.0, 12.0, 25.0, 10.0, 8.0, 30.0]`` → Output: ``[6.78 -11.02 -28.81 48.3 -40.68 -52.53 77.96]``

Logic: For each day, compute the ****percentage deviation from the array's mean****: ``((value - mean) / mean) * 100``, rounded to two decimal places. Values above the mean are positive; values below are negative. This uses vectorized broadcasting arithmetic across the whole array in one expression.

Returns: NumPy array of signed percentage deviations.

****Expected Output****

...

Valid: True

Metrics (total, average, minimum): (118.0, 16.86, 8.0)

Efficient Days: ['Excessive' 'Efficient' 'Efficient' 'Excessive' 'Efficient' 'Efficient' 'Excessive']

Longest Efficient Streak (<=20): 3

Usage Deviation (%): [6.78 -11.02 -28.81 48.3 -40.68 -52.53 77.96]

...

****Constraints****

- Validation must reject an empty array, any negative value, ****or any value above `max\_capacity`****.

- `compute\_usage\_metrics` and `compute\_usage\_deviation` round to two decimal places.

- `flag\_efficient\_days` computes its own comparison baseline from the data (the mean) – it does ****not**** take a target parameter, unlike the original exercise.

- `longest\_efficient\_streak` uses ` $\leq$ ` against a threshold, tracking a "stays low" run rather than a "keeps rising" run.

- `compute\_usage\_deviation` must produce signed (positive and negative) results, not absolute values.

B2. Reservoir Water Level Analyzer

****Problem Statement****

A dam operator records the ****daily water level**** of a reservoir (in metres) and needs an analytics helper that works in the ***opposite direction*** of a simple occupancy tracker: it watches for levels that are ****too low**** rather than too high, tracks the longest stretch of ****continuously falling**** levels rather than a sustained busy period, and reports how far each day falls ****short of**** capacity rather than how full it is.

****Objective****

Design a Python class named ****`ReservoirLevelAnalyzer`**** using NumPy. Validation enforces both a lower bound (no negative levels) and an upper bound (``max_level``, the physical capacity a sensor reading cannot exceed). The summary reports the ****lowest**** recorded level rather than the peak. Day-labelling flags ****critical**** (dangerously low) days using a ****strict less-than**** comparison against a safety threshold – the opposite direction from a "full/overstock" check. The streak method finds the longest run of ****strictly decreasing**** consecutive days, and the final method returns each day's ****deficit percentage**** – how empty the reservoir is relative to full capacity. The class is stateless and defines no constructor.

****File Structure****

...

Project

- ├─ main.py
- ├─ tests.py
- └─ installation.txt

...

****Task – Method Overview****

#	Method	Signature
1	create_level_array	<code>`create_level_array(self, level_list)`</code>
2	validate_level_array	<code>`validate_level_array(self, level_array, max_level)`</code>

```
| 3 | compute_level_metrics | `compute_level_metrics(self, level_array)` |
| 4 | flag_critical_days | `flag_critical_days(self, level_array,
min_safe_level)` |
| 5 | longest_declining_stretch | `longest_declining_stretch(self, level_array)` |
| 6 | deficit_percentage | `deficit_percentage(self, level_array, capacity)` |
```

****Method 1 – create_level_array****

Sample I/O – Input: `[80, 65, 55, 40, 60, 30, 20]` → Output: `[80. 65. 55. 40. 60. 30. 20.]`

Returns: NumPy array of dtype `float64`.

****Method 2 – validate_level_array****

Parameters: `level\_array`; `max\_level` – the reservoir's physical maximum.

Sample I/O – Input: `level=[80,65,55,40,60,30,20]`, `max\_level=100` → Output: `True`

Logic: Valid only when there is at least one value, no value is negative, **and no value exceeds `max\_level`**.

Returns: Boolean.

****Method 3 – compute_level_metrics****

Sample I/O – Input: `[80.0, 65.0, 55.0, 40.0, 60.0, 30.0, 20.0]` → Output: `(350.0, 50.0, 20.0)`

Logic: Total, average, and the **lowest** single day's level, rounded to two decimal places.

Returns: Tuple `(total, average, lowest)`.

****Method 4 – flag_critical_days****

Parameters: `level\_array`; `min\_safe\_level` – the minimum safe operating level.

Sample I/O – Input: `level=[80,65,55,40,60,30,20]`, `min\_safe\_level=35` →

Output: `['Safe' 'Safe' 'Safe' 'Safe' 'Safe' 'Critical' 'Critical']`

Logic: A day is `Critical` when its level is **strictly below** `min\_safe\_level` (a day exactly at the minimum is still `Safe` – the opposite direction and boundary rule from a `>=` "overstock" check).

Returns: NumPy array of labels.

****Method 5 – longest_declining_stretch****

Sample I/O – Input: `[80.0, 65.0, 55.0, 40.0, 60.0, 30.0, 20.0]` → Output: `4`

Logic: Longest run of consecutive days whose level **strictly decreases** from one day to the next (the mirror image of a "rising streak"). A single day counts as length one; an empty array gives zero; equal consecutive values do not extend a run. (Here: `80>65>55>40` gives length 4; `60>30>20` gives length 3 – the first run wins.)

Returns: Integer.

****Method 6 – deficit_percentage****

Sample I/O – Input: `level=[80,65,55,40,60,30,20]`, `capacity=100` → Output: `[20. 35. 45. 60. 40. 70. 80.]`

Logic: For each day, compute how far the level falls short of capacity:

`((capacity - value) / capacity) \* 100`, rounded to two decimal places – the inverse of an occupancy-rate calculation.

Returns: NumPy array of per-day deficit percentages.

****Expected Output****

...

Valid: True

Metrics (total, average, lowest): (350.0, 50.0, 20.0)

Critical Days (min_safe_level=35): ['Safe' 'Safe' 'Safe' 'Safe' 'Safe' 'Critical' 'Critical']

Longest Declining Stretch: 4

Deficit Percentage (capacity=100): [20. 35. 45. 60. 40. 70. 80.]

...

****Constraints****

- Validation rejects an empty array, any negative value, or any value above ``max_level``.
- ``compute_level_metrics`` and ``deficit_percentage`` round to two decimal places.
- ``flag_critical_days`` uses a `**strict less-than**` comparison (the reverse boundary rule of the original's ``>=``).
- ``longest_declining_stretch`` uses a `**strict decrease**`, not an increase.
- ``deficit_percentage`` measures emptiness, not fullness.

B3. Library Membership & Borrowing Analyzer

****Problem Statement****

A public library wants to understand member engagement and borrowing patterns across cities. Two datasets are provided: member registration records and book-loan history. Unlike a simple "who never transacted" analysis, the library specifically wants to know ****which members are repeat borrowers****, wants every member kept in the merged view (even those with zero loans, appearing as missing loan data) rather than dropped, and wants ****average borrowing duration**** by city rather than total revenue.

****Objective****

Implement six Python functions using Pandas to analyze member and loan data from CSV files. This exercise deliberately changes three pieces of logic from a standard clean-merge-aggregate exercise: the cleaning step uses a ****mixed strategy**** (drop only where the join key is missing, fill other missing fields with a placeholder) rather than a blanket ``dropna``; the merge is a ****left join**** (keeping every member) rather than an inner join; and the final aggregation reports a ****mean**** rather than a ****sum****, alongside identifying ****repeat**** (active) members rather than inactive ones.

****File Structure****

```

Project/
├── data/
│   ├── Members.csv
│   └── Loans.csv
├── main.py
└── installation.txt

```

****Dataset Description****

``Members.csv`` – Member registration records

- ``MemberID`` – Unique identifier (e.g. M001)
- ``MemberName`` – Full name
- ``City`` – City where the member is located (may be missing)
- ``JoinDate`` – Date of registration (YYYY-MM-DD)

``Loans.csv`` – Book-loan records

- ``LoanID`` – Unique identifier (e.g. L001)
- ``MemberID`` – References a member from `Members.csv` (may be missing)
- ``LoanDate`` – Date the book was borrowed (YYYY-MM-DD)
- ``DaysBorrowed`` – Number of days the book was kept, in numeric-as-string format (e.g. ``"14"``)

****Function 1 – `load_and_clean_data(members_path, loans_path)`****

Logic: ****Different cleaning rule from a blanket `dropna`**** – read both CSV files. In ``Members.csv``, drop a row only if ``MemberID`` itself is missing; for any other missing field (e.g. ``City``), fill it with the string ``"Unknown"`` instead of dropping the row. In ``Loans.csv``, drop any row where ``MemberID`` is missing (an orphaned loan cannot be attributed).
Returns: Tuple ``(members_df, loans_df)``.

****Function 2 – convert_durations(loans_df)****

Logic: Convert the `DaysBorrowed` column from string to a numeric (integer) type.

Returns: The loans DataFrame with an integer `DaysBorrowed` column.

****Function 3 – merge_data(members_df, loans_df)****

Logic: ****Different join type from the original**** – perform a ****left merge**** of `members\_df` with `loans\_df` on `MemberID`, keeping every member regardless of whether they have any loans. Members with no loans will show missing values in the loan-related columns.

Returns: Merged DataFrame.

****Function 4 – calculate_metrics(members_df, loans_df, merged_df)****

Logic: Report the shapes of all three DataFrames, total member count, total loan count, and the count of members who have ****zero loans**** (identified via missing values in the merged DataFrame's loan columns) – this last figure has no equivalent in the original exercise.

Returns: A dictionary including a `members\_with\_zero\_loans` key.

****Function 5 – find_repeat_borrowers(members_df, loans_df)****

Logic: ****Inverted logic from the original's inactivity check**** – instead of finding members who never borrowed, find members who appear ****more than once**** in the loans data (i.e., borrowed more than one book). Use a

`groupby`/`value\_counts`-style count on `MemberID` in `loans\_df`, keep IDs with a count greater than one, and return them sorted alphabetically.

Returns: A sorted list of repeat-borrower MemberID strings.

****Function 6 – city_wise_summary(merged_df)****

Logic: ****Different aggregation from the original**** – group the merged DataFrame by `City`. For each city, compute the number of loans (count of non-missing `LoanID`, as an integer) and the ****average**** `DaysBorrowed` (mean, not sum, rounded to 1 decimal place) among that city's loans.

Returns: A dictionary like `{"Mumbai": {"loans": 6, "avg\_days\_borrowed": 12.3}, "Delhi": {"loans": 4, "avg\_days\_borrowed": 9.8}, ...}`.

****Expected Output (values will vary with your own sample data)****

...

Members shape: (38, 4)

Loans shape: (45, 4)

Merged shape: (49, 7)

Total Members: 38

Total Loans: 45

Members with zero loans: 6

Repeat Borrowers: ['M003', 'M011', 'M017', 'M022']

...

****Constraints****

- The cleaning step must not use a blanket `dropna()` on `Members.csv`; only the join-key column is a hard drop condition there.

- `merge\_data` must use `how="left"`, not an inner join.

- `find\_repeat\_borrowers` must identify members with ****more than one**** loan, not members with zero loans.

- `city\_wise\_summary` must report a ****mean****, not a sum, for `DaysBorrowed`.